



Universidade Estadual de Feira de Santana
Programa de Pós-Graduação em Ciência da Computação

Improving JavaScript Test Quality with Large Language Models: Lessons from Test Smell Refactoring

Gabriel Amaral Sousa

Feira de Santana

2026



Universidade Estadual de Feira de Santana
Programa de Pós-Graduação em Ciência da Computação

Gabriel Amaral Sousa

Improving JavaScript Test Quality with Large Language Models: Lessons from Test Smell Refactoring

Dissertation submitted to the State University of Feira de Santana as part of the requirements for the degree of Master of Science in Computer Science.

Advisor: Larissa Rocha Soares Bastos

Co-advisor: Eduardo Figueiredo

Feira de Santana

2026

Abstract

Automated tests are essential for supporting software evolution, regression prevention, and confidence in maintenance activities. However, test suites may degrade over time due to test smells, which affect readability, maintainability, reliability, and diagnostic effectiveness. Although previous studies have proposed catalogs and detection tools for test smells, less attention has been given to their automated correction, especially in JavaScript projects. This dissertation investigates the effectiveness and limitations of Large Language Models (LLMs) for automated refactoring of JavaScript test smells through a controlled empirical study. The evaluation includes six LLMs, three prompting strategies, ten smell categories, and 200 validated smell instances extracted from real JavaScript projects, resulting in 3,600 refactoring attempts. Refactoring quality is assessed through a multi-dimensional perspective that considers Smell Removal Rate (SRR), Pass Preservation Rate (PPR), coverage impact, unintended structural side effects, and an integrated Refactoring Success Rate (RSR). The results show that LLMs can remove JavaScript test smells in a substantial proportion of cases, achieving an overall SRR of approximately 76.7% and a global PPR of 86.3%. However, when smell removal, behavioral preservation, and coverage stability are jointly required, the RSR decreases to 64.9%, indicating that smell removal alone overestimates refactoring quality. The findings also show that model choice has a stronger influence on outcomes than prompting strategy, while refactoring difficulty varies considerably across smell types. Smells requiring localized and well-defined transformations tend to be handled more effectively, whereas smells involving broader structural changes, assertion design, control flow, or test intent remain more challenging. Overall, this dissertation provides empirical evidence that LLMs can support JavaScript test smell refactoring, but should be integrated into validation-aware workflows that combine generated refactorings with automated testing, coverage analysis, and post-refactoring smell detection.

Keywords: Large Language Models, JavaScript, test smells, automated refactoring, software testing, software maintenance, prompt engineering.

Resumo

Testes automatizados são essenciais para apoiar a evolução de software, a prevenção de regressões e a confiança em atividades de manutenção. No entanto, suítes de teste podem se degradar ao longo do tempo devido à presença de *test smells*, que afetam sua legibilidade, manutenibilidade, confiabilidade e capacidade de apoiar o diagnóstico de falhas. Embora estudos anteriores tenham proposto catálogos e ferramentas para a detecção de *test smells*, menor atenção tem sido dedicada à sua refatoração automatizada, especialmente em projetos JavaScript. Esta dissertação investiga a efetividade e as limitações de *Large Language Models* (LLMs) na refatoração automatizada de *test smells* em JavaScript por meio de um estudo empírico controlado. A avaliação inclui seis LLMs, três estratégias de *prompting*, dez categorias de *smells* e 200 instâncias validadas extraídas de projetos reais, resultando em 3.600 tentativas de refatoração. A qualidade da refatoração é avaliada por uma perspectiva multidimensional que considera *Smell Removal Rate* (SRR), *Pass Preservation Rate* (PPR), impacto na cobertura, efeitos colaterais estruturais não intencionais e *Refactoring Success Rate* (RSR). Os resultados mostram que LLMs removem *test smells* em JavaScript em uma proporção substancial dos casos, alcançando SRR geral de aproximadamente 76,7% e PPR global de 86,3%. No entanto, quando a remoção do *smell*, a preservação comportamental e a estabilidade da cobertura são exigidas conjuntamente, a RSR diminui para 64,9%, indicando que a remoção isolada do *smell* superestima a qualidade da refatoração. Os achados também mostram que a escolha do modelo exerce maior influência sobre os resultados do que a estratégia de *prompting*, enquanto a dificuldade da refatoração varia consideravelmente entre os tipos de *smell*. *Smells* que exigem transformações localizadas e bem definidas tendem a ser tratados com maior efetividade, ao passo que *smells* que envolvem mudanças estruturais mais amplas, projeto de asserções, fluxo de controle ou intenção do teste permanecem mais desafiadores. De modo geral, esta dissertação fornece evidências empíricas de que LLMs podem apoiar a refatoração de *test smells* em JavaScript, mas devem ser integrados a fluxos de trabalho orientados por validação, combinando refatorações geradas com testes automatizados, análise de cobertura e redetecção de *smells* após a refatoração.

Palavras-chave: *Large Language Models*, JavaScript, *test smells*, refatoração automatizada, testes de software, manutenção de software, *prompt engineering*.

Preface

This master's dissertation was submitted to the State University of Feira de Santana (UEFS) as a partial requirement for obtaining the degree of Master's in Computer Science.

The dissertation was developed within the Graduate Program in Computer Science (PGCC), with Prof. Dr. **Larissa Rocha Soares Bastos** as advisor.

Prof. Dr. **Eduardo Figueiredo** was co-advisor of this work.

This research was funded by FAPESB.

Acknowledgements

First and foremost, I thank God and the Lord Jesus Christ for the grace, strength, and guidance that sustained me throughout this challenging journey. Without Their blessings, this achievement would not have been possible.

I extend my deepest thanks to my family, especially my mother, Lídia Amaral, for their unconditional love, sacrifices, and constant support. You have always been my foundation, and your presence and example guided me through each stage of this path. I also offer a very special thanks to my future wife, Marcely Ribeiro, for her love, encouragement, patience, and constant presence throughout this journey. Your support and belief in me were fundamental in helping me face the most difficult moments, and you were not only present during this process, but also an essential part of it.

I am profoundly grateful to my advisor, Professor Dr. Larissa Rocha, and to my co-advisor, Professor Dr. Eduardo Figueiredo, for their guidance, expertise, encouragement, and invaluable support throughout this research. Professor Larissa's mentorship was fundamental not only to the development of this work but also to my growth as a researcher, while Professor Eduardo's generosity, availability, careful contributions, and dedication were essential to the review, writing, conception, and refinement of this dissertation.

I would also like to express my deep gratitude to the Christian movements CRU and LUZ at UEFS. When I arrived in Feira de Santana to begin my Master's program, far from home and facing a new stage of life, these communities received me in a way that I will never forget. Through their support, encouragement, prayers, friendship, and acceptance, I did not face this period alone; instead, I found a community that supported me as family. The people who are part of these movements strengthened my faith and were fundamental during my adaptation and throughout the period I lived in Feira de Santana.

To my friends and classmates, thank you for the companionship, support, and the many shared moments that made this academic journey lighter. I also gratefully acknowledge the financial support provided by FAPESB (Bahia Research Foundation), which made this research possible. Finally, to everyone who contributed directly or indirectly to this achievement, my sincere thanks. This accomplishment reflects the collective support, faith, and generosity of those who walked this journey with me.

*“There is some good in this world,
and it’s worth fighting for.”*

– J. R. R. Tolkien

Summary

Abstract	i
Resumo	ii
Preface	iii
Alignment with Research Line	xi
Publications and Technical Productions	xii
List of Tables	xiii
List of Figures	xiv
List of Abbreviations	xv
List of Symbols	xvi
1 Introduction	1
1.1 Research Objectives	3
1.2 Research Questions	3
1.3 Research Methodology	4
1.4 Overview of Main Results	5
1.5 Contributions	6
1.6 Document Organization	7
2 Background and Related Work	9
2.1 Software Testing	10
2.2 Test smells	11
2.3 Tools for <i>Test Smell</i> Detection	12
2.4 Code Refactoring	14
2.5 Large Language Models	15
2.6 Prompt Engineering	17
2.6.1 Zero-Shot Prompting	18
2.6.2 Few-Shot Prompting with Contextual Examples	18

2.6.3	Chain-of-Thought Prompting	18
2.7	Related Work	19
2.8	Conclusion	21
3	Methodology	23
3.1	Research Objective and Questions	24
3.2	Pre-Refactoring Activities	25
3.2.1	Selection of Test Smell Detection Tools	26
3.2.2	Selection of Repositories	26
3.2.3	Selection of Test Smells	28
3.2.4	Selected LLMs	30
3.3	Refactoring Process	32
3.3.1	Overview of the Refactoring Pipeline	32
3.3.2	Prompting Strategies	33
3.3.3	Zero-Shot Prompt Structure	34
3.3.4	Few-Shot Prompt Structure	34
3.3.5	Chain-of-Thought Prompt Structure	34
3.3.6	Parameter Configuration and Execution Constraints	35
3.3.7	Extraction and Validation of LLM-Generated Code	38
3.3.8	Results Integration	39
3.4	Post-Refactoring Analysis	39
3.4.1	Functional Behavior Validation	39
3.4.2	Reapplication of Smell Detection Tools	39
3.5	Conclusion	42
4	Results	43
4.1	RQ1 – Effectiveness of Smell Removal	44
4.1.1	Experimental Scope and Global Smell Removal Effectiveness	44
4.1.2	Effectiveness Across Smell Types	44
4.1.3	Removal Effectiveness by Prompting Strategy	46
4.1.4	Model-Level Performance and Interaction Effects	47
4.1.5	Variance Decomposition Insight	48
4.1.6	Answer to RQ1	48
4.2	RQ2 – Behavioral Preservation of Refactored Tests	48
4.2.1	Global Behavioral Preservation	49
4.2.2	Behavioral Preservation Across Smell Types	49
4.2.3	Behavioral Preservation by Prompting Strategy	50
4.2.4	Model-Level Performance and Interaction Effects	51
4.2.5	Failure Taxonomy and Regression Patterns	52
4.2.6	Answer to RQ2	56
4.3	RQ3 – Coverage Impact of Refactoring	56
4.3.1	Overall Coverage Effects	57
4.3.2	Model-Level Coverage Behavior	58
4.3.3	Coverage Risk by Smell Type	59

4.3.4	Interpretation in Light of Coverage Theory	62
4.3.5	Answer to RQ3	62
4.4	RQ4 – Unintended Structural Side Effects	63
4.4.1	Global New-Smell Introduction	63
4.4.2	Removal vs. New-Smell Introduction by Smell Type	64
4.4.3	Taxonomy of Newly Introduced Smells	65
4.4.4	Smell Interaction Matrix	66
4.4.5	Smell Introduction by Model and Prompt Strategy	66
4.4.6	Answer to RQ4	69
4.5	Influence of Experimental Factors on Refactoring Outcomes	69
4.5.1	Global Integrated Success	70
4.5.2	Influence of Prompting Strategy	70
4.5.3	Influence of Model Choice	71
4.5.4	Model x Prompt Interaction Effects	71
4.5.5	Failure Cause Diagnosis	72
4.5.6	Explanatory Synthesis	73
4.5.7	Answer to RQ5	73
4.6	Conclusion of Results	74
5	Discussion	76
5.1	A Multi-Dimensional Interpretation of Refactoring Success	77
5.2	Smell Removal Effectiveness as a Function of Refactoring Complexity	79
5.3	The Structural and Semantic Gap in Behavioral Preservation	81
5.4	Coverage Reduction as an Independent Quality Signal	82
5.5	Model Capability as the Primary Driver of Refactoring Outcomes . .	84
5.6	Smell-Dependent Risk and New-Smell Introduction	85
5.7	Threats to Validity	87
5.8	Implications for Research and Practice	89
5.9	Conclusion of the Discussion	90
6	Conclusion	92
6.1	Integrated Synthesis of Findings	92
6.2	Scientific Contributions	94
6.3	Future Work	95
	Referências	97
A	Catalog	107
A.1	Anonymous Test	107
A.1.1	Definition	107
A.1.2	Consequences	107
A.1.3	Detection	107
A.1.4	Example in JavaScript	108
A.1.5	Refactoring Strategies	108
A.2	Conditional Test Logic	108

A.2.1	Definition	108
A.2.2	Consequences	109
A.2.3	Detection	109
A.2.4	Example in JavaScript	109
A.2.5	Refactoring Strategies	110
A.3	Duplicate Assert	110
A.3.1	Definition	110
A.3.2	Consequences	110
A.3.3	Detection	111
A.3.4	Example in JavaScript	111
A.3.5	Refactoring Strategies	111
A.4	Exception Handling	112
A.4.1	Definition	112
A.4.2	Consequences	112
A.4.3	Detection	113
A.4.4	Example in JavaScript	113
A.4.5	Refactoring Strategies	113
A.5	Magic Number	114
A.5.1	Definition	114
A.5.2	Consequences	114
A.5.3	Detection	115
A.5.4	Example in JavaScript	115
A.5.5	Refactoring Strategies	115
A.6	Overcommented Test	116
A.6.1	Definition	116
A.6.2	Consequences	116
A.6.3	Detection	117
A.6.4	Example in JavaScript	117
A.6.5	Refactoring Strategies	117
A.7	Sleepy Test	118
A.7.1	Definition	118
A.7.2	Consequences	118
A.7.3	Detection	119
A.7.4	Example in JavaScript	119
A.7.5	Refactoring Strategies	119
A.8	Suboptimal Assertion	121
A.8.1	Definition	121
A.8.2	Consequences	121
A.8.3	Detection	121
A.8.4	Example in JavaScript	121
A.8.5	Refactoring Strategies	122
A.9	Unknown Test	122
A.9.1	Definition	122
A.9.2	Consequences	123

A.9.3	Detection	123
A.9.4	Example in JavaScript	123
A.9.5	Refactoring Strategies	123
A.10	Verbose Test	124
A.10.1	Definition of Verbose Test	124
A.10.2	Consequences	125
A.10.3	Detection	125
A.10.4	Example in JavaScript	125
A.10.5	Refactoring Strategies	126
B	Repositories	127

Alignment with Research Line

Research Line: Software Engineering

This dissertation aligns with the Software Engineering research line through its focus on automated software maintenance and the empirical evaluation of AI-assisted tools, contributing directly to software testing and test code quality improvement.

The research advances the study of JavaScript test smell refactoring by investigating the effectiveness of Large Language Models (LLMs) in improving test code. Preliminary results, published in a peer-reviewed venue, explored 148 test smell instances across ten real-world JavaScript projects using zero-shot prompting with GitHub Copilot Chat and Amazon CodeWhisperer. These early findings provided initial evidence about LLM performance and limitations, and laid the groundwork for the broader and more rigorous investigation presented in this dissertation.

Expanding upon this foundation, the present work evaluates six LLMs and incorporates three prompting strategies, namely zero-shot, few-shot, and chain-of-thought prompting, employing a multi-dimensional empirical evaluation that assesses refactoring quality across multiple complementary dimensions. It proposes a comprehensive and replicable framework for assessing AI-based test smell refactoring in dynamic programming environments such as JavaScript.

By providing empirical evidence on AI-assisted test improvement, this research contributes to the understanding of both the capabilities and limitations of LLMs in real-world development contexts. Methodologically, it emphasizes systematic evaluation, controlled comparisons, and the integration of practical software engineering concerns into empirical research, reinforcing its alignment with core principles of the Software Engineering research line.

Publications and Technical Productions

List of the author's bibliographic and technical productions during the master's program.

- **Paper Title:** *Identifying and Addressing Test Smells in JavaScript: A Developer-Centric Study*
Conference or Journal: SBES Research Track 2025
List of Authors: Jhonatan Oliveira (UEFS), Luigi Mateus (UNEB), **Gabriel Amaral (UEFS)**, Tássio Virgínio (UFBA), Carla Bezerra (UFC), Ivan Machado (UFBA), Larissa Rocha (UNEB/PGCCUEFS)
- **Paper Title:** *Improving JavaScript Test Quality with Large Language Models: Lessons from Test Smell Refactoring*
Conference or Journal: SBES-IIER 2025 - Insightful Ideas and Emerging Results Track
List of Authors: **Gabriel Amaral (PGCCUEFS)**, Henrique Gomes (UFMG), Eduardo Figueiredo (UFMG), Carla Bezerra (UFC), Larissa Rocha (UNEB/PGCCUEFS)

List of Tables

2.1	Test Smells Adopted in This Research	13
4.1	Failure Taxonomy	53
4.2	Intra-suite Regressions by Smell Type	54
4.3	Intra-suite Regressions by Model	54
4.4	Suite-level Regressions by Smell Type	55
4.5	Suite-level Regressions by Model	55
4.6	Coverage Delta by Smell Type	60
4.7	Coverage Delta by Model	60
4.8	Smell Removal and New-Smell Introduction by Smell Type	64
4.9	Taxonomy of Newly Introduced Smells	65
4.10	Smell Removal Results by Prompt Strategy	70
4.11	Smell Removal Results by Model	71
4.12	Distribution of Refactoring Failure Causes	72
B.1	Technical Characteristics of Selected Projects	127

List of Figures

3.1	Research Design Flow	26
3.2	Interactive Web-based Tool for Exploring and Selecting Test Smells. .	30
3.3	CLI for Configuring and Executing the LLM-based Refactoring Pipeline	33
4.1	SRR by Smell Type	45
4.2	SRR by Prompt Strategy	46
4.3	Heatmap — Model $\times$ Prompt Removal Matrix	47
4.4	PPR by Smell Type	49
4.5	PPR by Prompt Strategy	50
4.6	Heatmap — Model by Prompt PPR Matrix	52
4.7	Overall Coverage Classification After Refactoring.	58
4.8	Mean Statement Coverage Change by Model.	59
4.9	Heatmap – Model by Mean Statement Coverage Matrix	61
4.10	Smell Removal and New Smell Introduction Rates by Smell Type . .	64
4.11	Smell Interaction Matrix Showing How Refactoring One Smell Intro- duces Others	67
4.12	New Smell Introduction Rate by Model	67
4.13	Heatmap – New Smell Introduction Rate by Model and Prompt Strategy	68
4.14	Heatmap of RSR by Model and Prompting Strategy	72

List of Abbreviations

Abbreviation	Description
AI	Artificial Intelligence
BDD	Behavior-driven development
CLI	Command-line Interface
CoT	Chain-of-Thought
CSV	Comma-Separated Values
IDE	Integrated Development Environment
LLM	Large Language Model
LM	Language Model
NLP	Natural Language Processing
PPR	Pass Preservation Rate
RQ	Research Question
RSR	Refactoring Success Rate
SRR	Smell Removal Rate
VLM	Vision-Language Model

List of Symbols

Symbol	Description
c_i	Categorical variable: coverage outcome of experiment
Δ	Difference between post-refactoring and baseline values
Δ_{coverage}	Change in test coverage after refactoring ($\text{coverage}_{\text{post}} - \text{coverage}_{\text{baseline}}$)
pp	Percentage points (unit for expressing coverage variation)
τ	Tolerance threshold for coverage variation (0.5 pp)
T	LLM sampling temperature (fixed at 0.3)
p_{top}	Nucleus sampling parameter top_p (fixed at 0.7)

Chapter 1

Introduction

Software testing is a fundamental pillar in the software development lifecycle, playing a crucial role in ensuring the delivery of high-quality software products (Burnstein et al., 1996; Ramler and Wolfmaier, 2006). By identifying and correcting defects early in the development process, testing significantly reduces development and maintenance costs while mitigating the risk of production failures (Bavota et al., 2012). This critical activity can be performed either manually or through automated approaches. In manual testing, human testers simulate end-user experiences by interacting with the software system to identify defects and inconsistencies. Automated testing, on the other hand, employs scripts or specialized tools to execute test cases (Amannejad et al., 2014), representing a widely adopted practice that reduces the inherent costs of manual testing while maintaining system quality. However, test suites are themselves software artifacts and are therefore subject to degradation over time. Poorly structured tests may become difficult to understand, maintain, and evolve, reducing their effectiveness as quality assurance mechanisms and increasing the cost of software maintenance.

One important manifestation of test code degradation is the presence of *test smells*, which are poor design patterns or antipatterns that affect the quality, maintainability, readability, or reliability of automated tests (van Deursen et al., 2001; Meszaros, 2007; Palomba et al., 2016; Tufano et al., 2016). Test smells may appear in different forms, including unclear test intent, missing or redundant assertions, excessive control logic, fragile timing dependencies, verbose test structures, and exception handling patterns that obscure the behavior being verified. Although these smells do not necessarily cause immediate test failures, they can reduce the diagnostic value of the test suite, increase maintenance effort, and make tests less effective at supporting software evolution (Bavota et al., 2015; Tufano et al., 2016). For this reason, detecting and refactoring test smells is an important activity for preserving test quality.

Prior research on test smells has evolved along complementary research directions. Foundational studies proposed catalogs and conceptual classifications that charac-

terize recurring problems in test code and describe their effects on maintainability, readability, and reliability (van Deursen et al., 2001; Meszaros, 2007; Aljedaani et al., 2021). Subsequent work developed automated tools and approaches for detecting test smells across different programming languages (Virgínio et al., 2021; Wang et al., 2021; Jorge et al., 2021; Silva, 2022; Oliveira et al., 2024), while other studies investigated refactoring strategies and tool-supported corrections aimed at improving the structure and maintainability of test suites (Fowler, 2018; van Deursen et al., 2001; Martins et al., 2024). Nevertheless, existing support remains more mature for smell characterization and detection than for automated correction, and many refactoring approaches still depend on predefined rules or language-specific transformation patterns. This limitation motivates further investigation into automated test smell refactoring, especially in contexts where test code is dynamic, heterogeneous, and difficult to transform through fixed rules alone.

JavaScript represents a particularly relevant context for this problem. It is one of the most widely used programming languages and is extensively adopted in web, server-side, and full-stack development environments (Cass, 2025). Its dynamic nature, flexible syntax, and diverse testing practices make the maintenance of test suites especially challenging. At the same time, prior research on test smells has historically focused more heavily on statically typed languages such as Java, Scala, Smalltalk, and C++ (Virgínio et al., 2021; De Bleser et al., 2019; Reichhart et al., 2007; Breugelmans and Van Rompaey, 2008). Recent JavaScript-oriented tools such as Steel (Jorge et al., 2021), TestSmellDetectorJs (Silva, 2022), and SNUTS.JS (Oliveira et al., 2024) have expanded detection support for JavaScript test smells, but the automated refactoring of these smells remains comparatively less explored. Beyond detection, recent research has also examined the automation and semi-automation of test smell refactoring, including approaches based on refactoring tools, IDE support, and structured correction strategies (Soares et al., 2023; Santana et al., 2024; Pizzini et al., 2023). These studies reinforce that improving test quality requires not only identifying problematic patterns, but also applying corrections that preserve the purpose of the test suite. In parallel, the rapid progress of generative AI has expanded interest in using LLMs for software engineering tasks such as code generation, test generation, debugging, and refactoring (Fan et al., 2023; Naveed et al., 2024; Martinez et al., 2025). Compared with predefined transformation rules, LLM-based approaches can be guided by natural language instructions and source code context, which has motivated their application to code improvement and refactoring tasks (Liu et al., 2024; Pomian et al., 2024).

This capability is especially relevant for test smell refactoring, since correcting a smell requires more than applying a syntactic change: the generated test must preserve the intended behavior, remain compatible with the testing framework, and avoid reducing the effectiveness or maintainability of the test suite (Fowler, 2018; van Deursen et al., 2001; Meszaros, 2007). Recent studies have begun to examine language models in contexts related to test smell detection, code improvement, and automated refactoring (Melo et al., 2025; Cordeiro et al., 2024; Liu et al., 2024;

Pomian et al., 2024). However, the use of LLMs for JavaScript test smell refactoring still requires systematic evaluation, particularly because JavaScript’s dynamic features and diverse testing conventions may influence both how smells appear and how generated refactorings should be validated.

Building on this context, this dissertation investigates the use of LLMs for automated refactoring of JavaScript test smells. The central concern of the study is not only whether these models can generate cleaner test code, but whether their refactorings satisfy the broader expectations of software maintenance: removing problematic test structures while preserving the behavior and usefulness of the test suite. By focusing on real JavaScript projects and on multiple dimensions of refactoring quality, this research seeks to clarify both the potential and the limitations of LLM-based assistance for improving test code. In doing so, the dissertation contributes empirical evidence on how LLMs can support test maintenance and on which validation concerns must guide their adoption in practical software engineering workflows.

1.1 Research Objectives

The primary objective of this research is to evaluate the effectiveness and limitations of LLM-based refactoring for improving JavaScript test code affected by test smells. Rather than treating refactoring as a simple code generation task, the study examines whether LLM-generated changes satisfy the core expectation of refactoring, namely improving internal structure while preserving externally observable behavior. In the context of test code, this requirement is particularly important because tests must remain reliable quality assurance artifacts after being modified (van Deursen et al., 2001; Meszaros, 2007).

A complementary objective is to analyze the conditions under which LLM-based refactoring succeeds or fails. For this purpose, the study investigates how refactoring outcomes vary across models, prompting strategies, and smell categories, allowing the analysis to distinguish between general performance trends and smell-specific or model-specific limitations. This perspective supports a more precise understanding of LLMs as assistive tools for test maintenance, since it examines their behavior under controlled and comparable experimental conditions rather than only reporting aggregate effectiveness. By combining structural, behavioral, coverage-based, and side-effect analyses, the research aims to provide empirical evidence about the reliability of LLM-based JavaScript test smell refactoring and the constraints that should guide its adoption in software maintenance workflows.

1.2 Research Questions

To guide the empirical investigation, this dissertation defines five research questions that progressively examine the quality of LLM-based refactoring from complementary perspectives. Rather than treating refactoring success as a single outcome,

the study separates the analysis into structural, behavioral, adequacy-related, side-effect, and explanatory dimensions. This organization reflects the central premise of the dissertation: a refactoring should not be considered effective only because the targeted smell is removed, since the resulting test must also preserve its original behavior, maintain adequate coverage, and avoid the introduction of new structural problems. The final research question then integrates these dimensions by examining how experimental factors, especially model choice and prompting strategy, influence the combined success of the refactoring process.

1. **RQ1:** To what extent can LLMs successfully remove JavaScript test smells?
2. **RQ2:** Do LLM-generated refactorings preserve the functional behavior of the test suite?
3. **RQ3:** How does LLM-based refactoring affect test coverage?
4. **RQ4:** What unintended structural side effects emerge from LLM-based refactoring?
5. **RQ5:** How do experimental factors influence LLM-based refactoring outcomes?

Together, these research questions provide a multi-dimensional structure for evaluating LLM-based test refactoring. RQ1 addresses the direct structural objective of removing the targeted smell, while RQ2 examines whether the generated refactoring preserves the original behavior of the test suite. RQ3 extends the analysis by considering coverage variation as an indicator of test adequacy, and RQ4 investigates whether the refactoring introduces new structural problems after the targeted smell is addressed. Finally, RQ5 synthesizes these dimensions by analyzing how model choice and prompting strategy influence refactoring outcomes across the previous questions. This progression allows the dissertation to move from isolated measures of success toward an integrated assessment of refactoring quality.

1.3 Research Methodology

This research adopts a controlled empirical methodology to evaluate LLM-based refactoring of JavaScript test smells under comparable experimental conditions. The methodological process is organized into three main phases: pre-refactoring activities, LLM-based refactoring, and post-refactoring analysis. In the pre-refactoring phase, the study defines the experimental corpus by selecting JavaScript repositories, choosing test smell detection tools, identifying smell categories, and validating concrete smell instances. The final corpus comprises 200 manually validated test smell instances distributed across ten smell categories and extracted from real JavaScript projects. This balanced design supports comparisons across smell types and prevents the results from being dominated by categories that occur more frequently in the analyzed repositories.

The refactoring phase is conducted through an automated pipeline that applies six LLMs to each validated smell instance under three prompting strategies: *Zero-shot*, *Few-shot*, and Chain-of-Thought (CoT). The evaluated models include both proprietary and open-weight systems, enabling the study to compare different model families and deployment paradigms under the same experimental setup. For each refactoring attempt, the pipeline constructs a prompt based on the target smell, invokes the selected model, extracts the generated JavaScript code, and stores the resulting refactoring for later execution and analysis. This process results in 3,600 refactoring attempts, allowing the research to analyze how model choice, prompting strategy, and smell category influence refactoring outcomes.

The post-refactoring phase evaluates each generated refactoring through multiple complementary quality dimensions. First, smell detection tools are reapplied to determine whether the targeted smell was removed and whether new smells were introduced. Second, the original test suite is executed after applying the generated refactoring to assess whether functional behavior is preserved. Third, coverage information is collected and compared before and after refactoring to identify whether the structural reach of the test suite is preserved, improved, or reduced. These analyses support the main evaluation indicators used in the study: *Smell Removal Rate* (SRR), *Pass Preservation Rate* (PPR), coverage variation, and the integrated *Refactoring Success Rate* (RSR), while new smell introduction is examined separately as an unintended structural side effect. Together, these measurements make it possible to evaluate whether LLM-generated refactorings effectively improve JavaScript test code and to identify the trade-offs that emerge after refactoring.

1.4 Overview of Main Results

The empirical results indicate that LLMs can support the automated refactoring of JavaScript test smells, although their effectiveness varies according to the quality aspect being evaluated. In terms of smell removal, the study achieved an overall SRR of 76.7%, showing that the targeted smell was removed in a substantial proportion of cases. However, this result was not uniform across smell categories. Smells associated with localized and well-defined transformations, such as *Unknown Test*, *Exception Handling*, and *Magic Number*, were removed more consistently, whereas smells requiring broader restructuring, especially *Verbose Statement*, remained more difficult to eliminate.

The behavioral analysis produced an overall PPR of 86.3%, indicating that most generated refactorings preserved the original execution outcome of the test suite. Nevertheless, the comparison between SRR and PPR shows that smell removal and behavioral preservation do not always coincide. This divergence is especially visible in smells whose refactoring may affect execution-sensitive behavior, such as *Exception Handling* and *Sleepy Test*. In these cases, a generated transformation may remove the detectable smell while altering exception propagation, asynchronous timing, or the conditions under which the test passes. This reinforces that smell

removal alone is insufficient to characterize refactoring quality, since a test may become cleaner from the perspective of smell detection while no longer preserving the behavior expected from the original test suite.

Coverage analysis adds a complementary perspective on refactoring quality by assessing whether refactored tests continue to exercise the codebase to a similar extent after transformation. The results indicate that coverage remained overwhelmingly stable, with preservation observed in 97.0% of the experiments and improvement in 1.3%. Meaningful coverage reduction occurred in only 1.7% of the experiments, concentrated mainly in smells whose refactoring may alter exception propagation or test structure, such as *Exception Handling*, *Overcommented Test*, and *Anonymous Test*. This result shows that, even when a refactoring is structurally valid, it may still reduce the portion of production code exercised by the resulting tests, making coverage variation an important complementary signal in the evaluation.

The analysis of unintended test smell introduction shows that LLM-generated refactorings may create new antipatterns after addressing the original smell. Overall, 13.4% of the analyzed experiments introduced at least one new smell after refactoring. These occurrences were concentrated mainly in *Duplicate Assert* and *Conditional Test Logic*, indicating that the most frequent side effects are associated with duplicated assertions and additional branching logic. This pattern suggests that the removal of a targeted smell may be accompanied by new test design problems, particularly when the refactoring requires substantial modifications to assertions or test logic.

When the main quality requirements were considered jointly, the integrated RSR reached 64.9%. This stricter result shows that fewer refactorings satisfy smell removal, behavioral preservation, and coverage stability at the same time. The analysis also indicates that model choice had a stronger influence on outcomes than prompting strategy, suggesting that the underlying capability of the LLM was more decisive than prompt variation in this experimental setting. Overall, the results show that LLMs are promising tools for JavaScript test smell refactoring, but their outputs require systematic validation before being treated as reliable improvements to test code.

1.5 Contributions

This research contributes to Software Engineering by providing a large-scale empirical evaluation of LLM-based JavaScript test smell refactoring. The study extends prior work on test smell detection, automated refactoring, and LLM-assisted software maintenance by focusing on the correction of real JavaScript test smells under controlled experimental conditions. The evaluation covers six LLMs, three prompting strategies, ten smell categories, and 3,600 refactoring attempts, enabling systematic comparison across multiple experimental dimensions. Rather than treating refactoring success as a single outcome, the study analyzes smell removal, be-

havioral preservation, coverage impact, and unintended side effects, producing a detailed characterization of when LLM-based refactoring is effective and when it remains limited.

A second contribution is methodological. The dissertation proposes and implements a multi-stage experimental pipeline that integrates smell detection, prompt construction, LLM-based refactoring, execution-based validation, coverage analysis, and post-refactoring smell detection. This pipeline supports reproducible experimentation and enables the evaluation of refactoring outcomes under comparable conditions across models and prompts. The study also defines an integrated success criterion that combines the main quality requirements of refactoring into a single measure, allowing a more realistic assessment of whether generated refactorings satisfy the combined expectations of structural correction, behavioral preservation, and coverage stability.

A third contribution is the construction and use of a literature-grounded test smell catalog, presented in Appendix A. For each investigated smell, the catalog consolidates formal definitions, detection cues, representative manifestations, JavaScript examples, and operational refactoring guidance based on prior work. This catalog serves as the primary knowledge source for prompt construction and ensures that the instructions provided to the LLMs are grounded in curated software engineering knowledge rather than only in implicit model assumptions. By standardizing smell descriptions and refactoring guidance across all models and prompting strategies, the catalog improves comparability and reduces ambiguity in the experimental process.

Finally, the dissertation contributes to Software Engineering conceptually and practically by providing a large-scale empirical evaluation and by clarifying how LLM-based refactoring behaves in the context of JavaScript test smell correction. The results show that LLMs are effective in many localized JavaScript test refactoring scenarios, but also reveal structural and semantic limitations, coverage effects, and side effects that must be considered before adopting these tools in test maintenance workflows. This contribution is relevant for researchers interested in AI for Software Engineering and for practitioners considering the use of LLMs as assistive tools for JavaScript test maintenance. The findings support the view that LLMs should be integrated into validation-aware workflows, where automated testing, coverage analysis, and smell re-detection complement generated refactorings.

1.6 Document Organization

The remainder of this dissertation is organized as follows:

- Chapter 2 presents the theoretical foundation and related work. It covers software testing, test smells, test smell detection tools, code refactoring, Large Language Models, prompt engineering, and prior studies on automated test smell detection and refactoring.

-
- Chapter 3 describes the design of the empirical study. It details the repository selection process, smell instance validation, model selection, prompt construction, experimental execution, and the metrics used to evaluate refactoring quality.
 - Chapter 4 presents the empirical results organized according to the five research questions. It reports the findings on smell removal effectiveness, behavioral preservation, coverage impact, unintended structural side effects, and the influence of models and prompting strategies on refactoring outcomes.
 - Chapter 5 provides a transversal interpretation of the results. It discusses the main patterns observed across the evaluation dimensions, including the relationship between structural effectiveness, behavioral preservation, coverage variation, model capability, and unintended side effects. It also presents the threats to validity.
 - Chapter 6 summarizes the main findings of the dissertation, presents its scientific contributions and outlines future research directions.

Chapter 2

Background and Related Work

This chapter presents the theoretical and empirical foundations that support this research. It first introduces the role of software testing in software quality assurance and then discusses test smells as recurring design problems that affect the maintainability and effectiveness of test suites. The chapter also reviews tools for detecting test smells, the principles of code and test refactoring, the use of Large Language Models (LLMs) in software engineering, and the role of prompt engineering in guiding model behavior. Finally, it discusses related work on automated test smell detection, test refactoring, and LLM-based software maintenance, positioning this dissertation within the current research landscape.

The chapter is organized as follows:

- Section 2.1 introduces software testing and discusses its relevance for software quality and evolution.
- Section 2.2 presents the concept of test smells and explains how these antipatterns may compromise the quality and maintainability of automated tests.
- Section 2.3 reviews tools for detecting test smells, with particular attention to JavaScript testing environments.
- Section 2.4 discusses code refactoring and test refactoring, emphasizing the requirement of preserving behavior while improving internal structure.
- Section 2.5 introduces LLMs and their application to software engineering tasks, particularly code generation, maintenance, and refactoring.
- Section 2.6 presents prompt engineering and the prompting strategies considered in this research.
- Section 2.7 reviews related studies and identifies the research gap addressed by this research.

2.1 Software Testing

Software systems currently play a central role in various aspects of contemporary society, being a fundamental component in sectors such as healthcare, finance, education, transportation, e-commerce, and governmental systems (Sommerville, 2011). This increasing dependence on computational systems imposes the need for such solutions to operate in a reliable, secure, and predictable manner, in accordance with predefined functional and non-functional requirements. Failure to meet these criteria can significantly compromise efficiency, information security, and, in critical scenarios, even the physical integrity of individuals or institutions (Spillner and Linz, 2021). Therefore, it is essential to employ quality control mechanisms, such as software testing, to ensure the final product meets specifications and operates adequately within its intended context.

Even the adoption of sound software engineering practices, such as modularity and code clarity, is not sufficient by themselves to guarantee the expected quality. In this context, software testing stands out as the primary mechanism used by the industry to detect faults, validate behavior, and increase system reliability before deployment to production environments (Ammann and Offutt, 2016). Software testing can be understood as a set of processes whose central objective is to ensure that a system's code behaves as expected, performing only the functions for which it was designed without producing side effects or undesired behaviors (Sommerville, 2011; Myers et al., 2011). This practice involves different approaches and levels of abstraction, ranging from unit tests, which validate small and isolated code units, to integration, system, and acceptance tests, which consider the application as a whole (Sommerville, 2011). Beyond verification, tests also serve as executable documentation and as a safeguard against regressions. Among their main features are the ability to reveal hidden defects, provide executable documentation, and act as a preventive mechanism for regressions throughout the development lifecycle (Sommerville, 2011).

The testing process can be conducted either manually or automatically, depending on the objectives, system complexity, and available resources. In manual testing, a professional directly interacts with the application, simulating end-user behavior to identify defects, inconsistencies, or unexpected behaviors. Automated testing, on the other hand, relies on *scripts* or specialized tools to execute predefined checks, offering as its main advantage the repeatability and efficiency of continuously running tests, particularly in agile and iterative development cycles (Amannejad et al., 2014). Despite its advantages, test automation faces challenges related to the readability, maintainability, and reliability of test code. Within this context, the so-called test smells emerge: these are antipatterns or poor practices that negatively affect the structural quality of tests, thereby hindering their ability to detect defects (Meszaros, 2007; Palomba et al., 2016).

2.2 Test smells

In software development, code is expected to be guided by sound design principles that promote quality and reduce rework (Suryanarayana et al., 2015). However, in practice, constraints such as tight deadlines and limited budgets often undermine this ideal. According to Pressman and Maxim (2021), managerial and technical factors frequently lead to the accumulation of ad hoc solutions that gradually degrade the system architecture. Even well-designed systems tend to experience structural erosion as they evolve, making the code increasingly difficult to understand, maintain, and scale (Pressman and Maxim, 2021; Fowler, 2018). To address this issue, the technique of refactoring proposes incremental improvements to the code without altering its external behavior. According to Fowler (2018), this practice aims to eliminate so-called *code smells*, which are symptoms of fragility or poor structure that may indicate design flaws. These *bad smells* were originally described for production code, but it was soon observed that test code also exhibited similar patterns. This suspicion was confirmed by van Deursen et al. (2001), who categorized what became known as test smells and proposed specific refactoring strategies to address them.

Test smells are characterized as poor design patterns or antipatterns present in test suites. These patterns often emerge when developers create test cases without adhering to best design practices, potentially indicating structural flaws and compromising the quality of the tests (Tufano et al., 2016). If left unaddressed, test smells can lead to several issues, such as low test maintainability due to redundancy, failure to detect real faults in production code due to missing assertions, and defects in the tests themselves. The last category includes nondeterministic tests, also known as flaky tests, which pass or fail randomly without any actual change to the tested software product (Vahabzadeh et al., 2015).

Although test smells represent a significant problem, studies (Tufano et al., 2016; Chen et al., 2023) have revealed that developers are unaware of their existence and their potential long-term impacts. These studies indicate that a considerable portion of developers neither recognize the presence of test smells nor perceive the need for test refactoring. This lack of awareness contributes to the persistence of these smells in the codebase, where they may remain uncorrected for extended periods. These findings underscore the need for automated tools capable of identifying and assisting developers in mitigating test smells.

An initial catalog defining 11 types of test smells was proposed by van Deursen et al. (2001), outlining the main issues that affect the quality and maintainability of automated tests. Later, this list was expanded with 12 additional test smells, addressing further problems that may undermine the effectiveness of testing practices (Aljedaani et al., 2021). The diversity of these test smells illustrates the complexity of the challenges involved in writing robust tests, requiring targeted approaches for their detection and correction.

Early detection and remediation of test smells are essential to maintain code quality and test reliability, fostering a healthy development environment and ensuring the delivery of high-quality software products (Bavota et al., 2015). By recognizing these flawed patterns within tests, development teams can perform targeted refactorings that improve both the structure and effectiveness of their test suites.

Research on test smell detection has primarily focused on statically typed programming languages, such as Java (Virgínio et al., 2021), Scala (De Bleser et al., 2019), Smalltalk (Reichhart et al., 2007), and C++ (Breugelmans and Van Rompaey, 2008). In contrast, JavaScript, which is currently one of the most popular programming languages (Cass, 2025), has received relatively little attention in this domain. This gap is particularly relevant given that JavaScript’s dynamic nature and flexible syntax may encourage the emergence of distinct test smell patterns.

Building upon this research landscape, this research focuses on a carefully selected set of test smells that are particularly relevant to JavaScript testing environments. These smells were chosen based on their prevalence in JavaScript projects and their potential impact on test quality. Table 2.1 presents the comprehensive set of test smells investigated in this research, including both classic smells originally identified in other programming languages and JavaScript-specific patterns that have emerged from recent research. The selection encompasses smells affecting various aspects of test quality, from assertion patterns to test structure and maintainability concerns.

2.3 Tools for *Test Smell* Detection

Detecting test smells is a complex yet essential activity to ensure software quality. The limitations and challenges of manual detection and analysis of such smells have been highlighted, demonstrating that manual methods are often ineffective due to the significant effort and time required, especially in cases involving large volumes of test code (Garousi and Küçük, 2018). In this context, automating this task can provide considerable gains in both efficiency and accuracy.

Automated tools for detecting test smells have gained increasing relevance, becoming essential to streamline their identification and mitigation, thereby supporting the maintainability and evolution of high-quality software systems. This automated approach not only accelerates the process of identifying test smells but also enables a broader and more systematic analysis of test suites, contributing to the continuous improvement of software quality (Pontillo et al., 2024). Moreover, these tools support development teams in adopting targeted refactoring practices, enhancing the effectiveness and reliability of automated tests (Peruma et al., 2020a).

Different environments and programming languages may exhibit distinct types of test smells (Aljedaani et al., 2021), making it essential to broaden research efforts to encompass a variety of programming languages. However, there are still notable gaps, as evidenced by the systematic mapping conducted by Aljedaani et al. (2021),

Table 2.1: Test Smells Adopted in This Research

<i>Smell</i>	<i>Characteristic</i>
<i>Anonymous Test</i> (Reichart, 2007)	Test method with a vague or meaningless name that fails to convey its purpose, making the test intent unclear.
<i>Conditional Test Logic</i> (Peruma et al., 2019)	Presence of conditionals (if-else, loops) inside the test method, compromising its effectiveness.
<i>Duplicate Assert</i> (Peruma et al., 2019)	Repeated verification of the same condition within the same method, harming readability.
<i>Exception Handling</i> (Peruma et al., 2019)	Absence of proper handling of expected exceptions, potentially concealing failures.
<i>Magic Number</i> (Peruma et al., 2019)	Use of literal numbers without explanation, reducing readability.
<i>Overcommented Test</i> (Reichart, 2007)	Test method excessively commented, hindering readability.
<i>Sleepy Test</i> (Peruma et al., 2019)	Presence of thread pause or fixed-duration delay calls inside the test method to simulate external events, compromising execution reliability.
<i>Suboptimal Assert</i> (Wang et al., 2021)	Use of inadequate assertions to verify conditions, reducing clarity.
<i>Unknown Test</i> (Peruma et al., 2019)	Complete absence of assertion statements in the test method, preventing explicit verification of expected conditions.
<i>Verbose Statement</i> (Breugelmans and Van Rompaey, 2008)	Excessively long or explicit code, reducing readability.

which emphasizes the need to expand tool support beyond Java to include popular languages such as JavaScript. The research shows that most detection tools focus on the Java ecosystem, which limits their applicability and leaves developers working with other languages without adequate resources to identify and address test smells.

One example of an automated tool for test smell detection is the JNose tool, which identifies 21 different types of test smells in Java projects (Virgínio et al., 2021). This tool allows developers to assess code quality and monitor improvements over time as the project evolves. Additionally, JNose calculates unit test coverage, offering a comprehensive view of test effectiveness within a project. In the Python context, PyNose was proposed as a tool designed to detect 17 types of test smells (Wang et al., 2021). By introducing a new category, *Suboptimal Assert*, PyNose highlighted the evolving nature of test smell research and the need for language-specific solutions.

To address the gap identified by Aljedaani et al. (2021), recent studies have focused

on developing automated test smell detection tools tailored for JavaScript. Among the proposed solutions are Steel (Jorge et al., 2021), TestSmellDetectorJs (Silva, 2022), and SNUTS.JS (Oliveira et al., 2024), each offering distinct features and supporting different sets of detectable test smells. A research conducted a qualitative study with developers of varying experience levels to understand their perceptions and strategies for handling seven JavaScript-specific test smells, such as *Anonymous Test*, *Comments Only Test*, and *Sensitive Equality* (Oliveira et al., 2025). The study employed focus groups and revealed that less experienced developers struggled to identify and mitigate these issues, while more senior developers proposed practical refactoring strategies. These findings reinforce the need for JavaScript-focused smell detection and refinement solutions, especially for smell categories that had not been adequately supported by previous static analysis tools.

The Steel tool (Jorge et al., 2021) was designed to identify 16 types of test smells, including: *Assertion Roulette*, *Conditional Test Logic*, *Duplicate Assert*, *Eager Test*, *Empty Test*, *Exception Handling*, *Global Variable*, *Ignored Test*, *Lazy Test*, *Magic Number Test*, *Mystery Guest*, *Redundant Assertion*, *Redundant Print*, *Resource Optimism*, *Sleepy Test*, and *Unknown Test*.

Building on this work, a complementary tool was developed to identify 8 additional types of test smells in JavaScript, expanding Steel’s coverage (Silva, 2022). This tool adapts test smells originally found in PyNose (Wang et al., 2021) to the JavaScript context, refining them to reflect the unique characteristics of the JavaScript language. Specifically, it detects smells such as *Non-Functional Statement*, *Suboptimal Assert*, *Unused Imports*, *Verbose Test*, and *Verifying in Setup Method*, along with three new types: *Complex Snapshot*, *Identical Test Description*, and *Only Test*. These contributions are significant because they not only adapt existing code smells to JavaScript but also identify novel anti-patterns specific to its ecosystem.

In addition to the aforementioned tools, SNUTS.JS provides broader support for analyzing and improving JavaScript unit test code (Oliveira et al., 2024). It identifies 15 types of test smells, six of which are unique to the tool: *Anonymous Test*, *Comments Only Test*, *General Fixture*, *Overcommented Test*, *Test without Description*, and *Transcripting Test*; and nine others adapted from existing literature: *Complex Snapshot*, *Conditional Test*, *Identical Test Description*, *Non Functional Statement*, *Only Test*, *Sensitive Equality*, *Suboptimal Assert*, *Verbose Statement*, and *Verify In Setup*. Beyond its detection scope, SNUTS.JS is open source and offers practical support for repository analysis through a documented Swagger API and an intuitive frontend interface, facilitating both integration and detailed inspection of JavaScript test suites.

2.4 Code Refactoring

Refactoring is defined as the systematic process of restructuring existing code without altering its external behavior, to improve its readability, simplicity, and main-

tainability (Fowler, 2018). This practice involves reorganizing and optimizing the internal structure of software systems to eliminate redundancies, reduce complexity, and facilitate future system evolution (Fowler, 2018). According to Opdyke (1992), who formally coined the term in his doctoral thesis, refactoring involves small and controlled transformations applied incrementally, ensuring that each change preserves the original functionality of the program. This approach is particularly important in continuously evolving systems, where frequent modifications can lead to code architecture degradation.

Despite its widely recognized benefits, refactoring faces several practical challenges. One of the main obstacles is the cost associated with performing such improvements, both in terms of time and technical effort (Mens and Tourwe, 2004). Another limiting factor is the tacit knowledge required from developers, who often need a deep understanding of the application’s domain, its dependencies, and its evolutionary history to make informed refactoring decisions.

Nevertheless, when applied systematically and supported by sound software engineering practices, refactoring proves to be a fundamental strategy for ensuring the maintainability and scalability of systems over time. It contributes to the reduction of cyclomatic complexity, facilitates component reuse, improves internal module cohesion, and reduces the incidence of bugs caused by excessive coupling or confusing structures (Fowler, 2018). Thus, refactoring is established as an essential practice not only for preserving the technical integrity of software systems, but also for sustaining their economic viability throughout the software life cycle.

Although refactoring is commonly applied to production code, its application to test code is equally relevant. Automated tests are also subject to structural degradation over time, especially in long-term projects, leading to fragile, redundant, or hard-to-maintain test scripts (van Deursen et al., 2001; Meszaros et al., 2003). The presence of test smells can compromise the reliability of the test suite and hinder its evolution. In this context, test code refactoring aims to restore clarity, modularity, and efficiency, ensuring that tests continue to fulfill their role effectively. According to van Deursen et al. (2001), the careful application of targeted refactorings can eliminate recurring anti-patterns, making test code more robust and aligned with software engineering best practices.

2.5 Large Language Models

LLMs are advanced artificial intelligence systems designed to understand and generate human-like text (Goodfellow et al., 2016). They are primarily based on *transformer architectures*, originally developed for Natural Language Processing (NLP), which enable efficient modeling of large-scale textual data (Gillioz et al., 2020). Trained through self-supervised learning on extensive corpora, LLMs learn general-purpose representations that support a wide range of tasks, including translation, summarization, and conversational interaction, establishing them as a central pil-

lar of modern AI research and applications (Naveed et al., 2024). Despite these capabilities, LLMs face challenges such as limited retention over long contexts and catastrophic forgetting during task-specific fine-tuning, which can lead to the loss of previously acquired knowledge (Naveed et al., 2024)

The scope of tools associated with LLMs has expanded rapidly, with applications emerging across domains such as medicine, education, finance, machine translation, question answering, text generation, and multilingual code synthesis, highlighting their versatility as general-purpose systems (Naveed et al., 2024). Recent advances further include tool-augmented LLMs, which can interact with external tools and APIs, thereby enhancing their capabilities in reasoning, planning, and execution.

As LLM capabilities matured, their application naturally extended into software engineering, where they are now widely employed for programming and code-related activities (Fan et al., 2023). Early evidence of this transition was provided by DeepMind’s AlphaCode, which demonstrated competitive performance in programming contests, ranking within the top 54% of simulated competitions involving more than 5,000 participants (Li et al., 2022). Shortly thereafter, OpenAI Codex, a GPT-3-based model that powered GitHub Copilot, marked a practical milestone by enabling large-scale AI-assisted programming in real-world development environments (Team, 2026). Subsequent generations, such as GPT-4, further improved code generation quality, achieving 67% zero-shot accuracy on the HumanEval benchmark and human-comparable performance on LeetCode-style tasks (Achiam et al., 2024). More recently, GPT-5 introduced architectural and system-level advances, including hierarchical routing, expanded context handling, and enhanced tool-use capabilities, supporting more reliable reasoning and complex code-oriented workflows (Leon, 2026; Singh et al., 2025).

Alongside the GPT lineage, Anthropic’s Claude family has shown strong and consistent performance in program synthesis and code generation. Large-scale evaluations on HumanEval report Pass@1 rates of 95.1% for Claude Sonnet 4 and 94.5% for Claude Opus 4, with statistically significant differences compared to GPT-family models (Bayram et al., 2025). Complementary performance-oriented benchmarks further indicate that Claude 4 Sonnet can outperform competing models and, in some cases, human-written solutions under strict runtime and memory constraints (Nia et al., 2025).

Google’s Gemini family expands LLM capabilities toward multimodal reasoning, supporting text, image, audio, video, and code processing within the same model family (Team et al., 2025). Benchmark results indicate that *Gemini Ultra* achieved state-of-the-art performance in several multimodal evaluations, while *Gemini Pro* reached 67.7% zero-shot accuracy on HumanEval, showing competitive coding capability (Team et al., 2025; Islam and Ahmed, 2024).

Beyond proprietary systems, open and semi-open LLMs have played a key role in advancing code intelligence research. The Qwen2.5-Coder family extends the Qwen

architecture with code-specific pretraining and demonstrates state-of-the-art performance across a wide range of code-related benchmarks, while maintaining permissive licensing suitable for empirical studies (Hui et al., 2024). Similarly, Code Llama provides open-weight models specialized for programming, supporting instruction following and code infilling, and enabling reproducible experimentation in both academic and industrial contexts (Roziere et al., 2023). Complementing these approaches, DeepSeek models emphasize reasoning-centric training strategies for LLMs, with DeepSeek-V3.2 (Liu et al., 2025a) combining sparse attention mechanisms for efficient long-context processing, scalable reinforcement learning-based post-training, and large-scale agentic task synthesis to improve reasoning and tool-use capabilities.

Taken together, the models discussed in this section illustrate the rapid evolution of LLMs from general-purpose language systems to powerful tools for software engineering and code intelligence. The diversity of proprietary and open approaches, combined with advances in reasoning, context handling, and controllability, provides a strong empirical and conceptual foundation for investigating LLM-driven refactoring and code quality improvement (Fan et al., 2023; Naveed et al., 2024).

2.6 Prompt Engineering

Prompt engineering (Liu and Chilton, 2022), also known as *prompt programming* (Ramchurn et al., 2021) or simply *prompting* (Bach et al., 2022), refers to the systematic construction of textual instructions designed to guide the behavior of generative artificial intelligence systems (Oppenlaender, 2024). This practice involves the process of designing and refining input instructions to effectively guide Language Models (LMs) in performing a wide range of tasks (Marvin et al., 2023). It has emerged as a critical technique for leveraging the full potential of pre-trained LLMs and Vision-Language Models (VLMs) (Sahoo et al., 2024).

Increasingly recognized as a valuable skill for software developers, prompt engineering facilitates more effective interaction with conversational LLMs such as Claude¹, Gemini², LLaMA³, and ChatGPT⁴ (White et al., 2023). Through carefully crafted instructions, developers can guide these models to produce accurate results, automate tasks, and conform to defined qualitative and quantitative criteria. Functioning as a form of natural language programming, prompt engineering enhances the alignment between input design and desired output, with applications spanning software development, document analysis, and cybersecurity (White et al., 2023).

In prompt engineering, a variety of prompting techniques are employed to optimize the performance of language models. These techniques differ in their levels of control,

¹<https://www.anthropic.com>

²<https://deepmind.google/technologies/gemini/>

³<https://ai.meta.com/llama/>

⁴<https://chat.openai.com>

specificity, and reasoning capabilities, making them particularly suitable for distinct software engineering tasks, including code refactoring processes (Sahoo et al., 2024). This section examines three fundamental prompting approaches that are particularly relevant for this research. Each technique will be analyzed in terms of its underlying mechanism, advantages, and limitations.

2.6.1 Zero-Shot Prompting

Zero-shot prompting involves providing a task description without supplementary examples, relying solely on the LLM’s pre-trained knowledge to generate appropriate responses (Radford et al., 2019; Wei et al., 2021). For instance, a prompt like *"Refactor this JavaScript test to remove Assertion Roulette smells"* directs the model to apply its latent understanding of test smells and refactoring patterns. This technique is computationally efficient and requires minimal setup, making it suitable for exploratory phases or broad-scope tasks (Kojima et al., 2022). However, its effectiveness depends heavily on the model’s prior exposure to similar contexts during training, which may lead to inconsistent results for niche or complex refactorings (Wei et al., 2022). In this research, *zero-shot* serves as a baseline for comparing more sophisticated techniques.

2.6.2 Few-Shot Prompting with Contextual Examples

Few-shot prompting builds upon *zero-shot* learning by incorporating task-specific demonstrations directly into the prompt. For instance, a refactoring prompt might include paired examples of smelly and cleaned test code before requesting a refactoring. These in-context examples serve as implicit guides, enabling the model to infer patterns and constraints more effectively (Brown et al., 2020).

While *few-shot* prompting enhances the performance of language models like GPT-3, especially for complex tasks, it comes with trade-offs. Increased token usage can reduce efficiency, particularly with lengthy inputs. Additionally, the selection and formatting of examples heavily influence model behavior, sometimes introducing biases (e.g., favoring common terms). Thus, while powerful, this approach requires careful prompt design to maximize accuracy and minimize unintended biases (Sahoo et al., 2024).

2.6.3 Chain-of-Thought Prompting

Chain-of-Thought (CoT) prompting explicitly instructs the LLM to articulate intermediate reasoning steps before delivering a final output. A refactoring prompt might state: *"First, identify all Assertion Roulette instances; second, explain why each violates best practices; third, propose fixes."* This technique mirrors human problem-solving processes, reducing hallucinations and improving logical coherence (Wei et al., 2022). CoT is particularly valuable for complex tasks, where multi-step analy-

sis is required. However, it increases prompt verbosity and computational overhead, necessitating trade-offs between explicitness and efficiency (Wei et al., 2022).

2.7 Related Work

Improving test code quality has long been a relevant topic in software engineering, particularly through the identification and correction of test smells. Early research concentrated mainly on the Java ecosystem, where automated support for test refactoring first became mature. Tools such as *DARTS* were designed to detect and refactor smells including *Eager Test* and *General Fixture* by applying predefined transformations integrated into development environments (Lambiasi et al., 2020). Similarly, *RTj* introduced automated support for *Rotten Green Tests*, combining detection rules with specific corrective refactorings (Martinez et al., 2020). Other solutions, such as *RAIDE*, demonstrated that automated IDE-based refactoring can successfully address smells like *Assertion Roulette* and *Duplicate Assert* through structured refactoring operations (Santana et al., 2024), while rule-based approaches have also been applied to reduce unnecessary cyclomatic complexity through automated simplification of redundant conditional structures (Magalhães et al., 2017). These approaches commonly rely on explicit mappings between smell types and predefined refactoring actions, which makes them effective in stable contexts but dependent on language-specific rules. In contrast, the present research investigates whether LLMs can perform smell-specific refactorings in JavaScript without relying on manually encoded refactoring rules, allowing greater flexibility within real-world test code contexts.

Beyond individual tools, the literature has also proposed systematic refactoring catalogs that associate each smell category with recommended corrective operations. These catalogs formalize strategies such as *Extract Method*, assertion decomposition, fixture reduction, and test renaming as recurring solutions for improving test maintainability (Martins et al., 2024). More recently, integrated frameworks such as *DANTES* extended this perspective by automatically combining detection and refactoring for eleven smell types in a unified workflow (Ramos et al., 2025). Although these contributions consolidate test refactoring knowledge, they remain fundamentally rule-based: each smell must be anticipated and connected to a predefined transformation pattern. This dependency limits their flexibility when equivalent smells appear under distinct structural contexts, even within the same testing ecosystem. For this reason, this research investigates LLMs as transformation agents capable of adapting refactoring decisions to the local semantics of each test case rather than executing only fixed refactoring templates.

Traditional approaches to test smell analysis have largely relied on static analysis rules, which require continuous adaptation as programming languages and testing practices evolve (Melo et al., 2025; Lucas et al., 2024). To mitigate this limitation, machine learning techniques have been explored as a means of learning smell patterns beyond handcrafted heuristics, demonstrating that data-driven models can

generalize detection across multiple smell categories (Pontillo et al., 2024). This line of research is reinforced by empirical evidence showing that test smells negatively affect maintainability, internal quality attributes, and even execution energy consumption (Damasceno et al., 2023; Misu et al., 2023). Despite these advances, most prior studies remain focused on identifying smell occurrences rather than producing corrected test code after detection. In this research, smell identification is performed beforehand through automated detection tools, allowing the experimental focus to shift toward the refactoring stage, where LLMs are evaluated as guided transformation agents responsible for generating executable corrections while preserving functional behavior.

The emergence of LLMs introduced a new paradigm for software maintenance by enabling generative transformations instead of deterministic refactoring scripts (Martinez et al., 2025). Initial studies demonstrated that models such as ChatGPT-4 can identify multiple test smells across programming languages (Lucas et al., 2024), while agent-oriented Small Language Models have shown promise in smell detection with lower computational cost (Melo et al., 2025). In parallel, LLM-based refactoring studies began exploring general code improvement tasks, including readability enhancement and structural simplification (Liu et al., 2024). Related evidence from software testing also indicates that LLMs can support adjacent test engineering activities beyond refactoring: Mendoza et al. (2024) compared *ChatGPT-3.5*, *ChatGPT-4*, *Gemini*, and *Copilot* for automated Behavior-driven development (BDD) test data generation and observed that stronger models consistently produced more semantically coherent and representative outputs under controlled prompting. A recent empirical study by Liu et al. (2025b) showed that general-purpose LLMs still face substantial limitations in autonomous refactoring: when asked to identify refactoring opportunities without explicit guidance, *GPT-4* detected only a limited portion of real opportunities, although performance improved significantly when refactoring categories and search scope were explicitly constrained. The same study also reported that part of the generated solutions introduced unsafe transformations, including syntax errors and behavioral changes. These findings reinforce the importance of controlled prompting and post-generation validation in LLM-based maintenance tasks. Tools such as *EM-Assist* used ChatGPT-3.5 to suggest *Extract Method* transformations interactively (Pomian et al., 2024), and iterative LLM workflows have demonstrated potential for progressively reducing complexity in production code (Choi et al., 2024). Nevertheless, these studies primarily address production code or generic refactoring scenarios, whereas this dissertation targets test-specific anti-patterns whose correction requires preserving assertion intent, execution semantics, and testing framework conventions.

More recent research has begun to examine LLM-driven test refactoring more directly. Empirical studies now compare state-of-the-art models for test smell identification and remediation in languages such as Java and Python (Santana Jr et al., 2025). Frameworks such as the Context-Enhanced LLM-Based Framework for Automatic Test Refactoring augment prompts with semantic context to improve trans-

formations in complex cases (Gao et al., 2025). Multi-agent architectures further distribute responsibilities across specialized LLM roles, improving coordination in large refactoring tasks (Pucho et al., 2025). At the same time, evaluation frameworks such as CTSES have been proposed to assess generated refactorings beyond superficial textual changes (Ouédraogo et al., 2025). These advances demonstrate that LLM-based refactoring is becoming methodologically mature; however, existing evaluations still concentrate on languages with more established tooling ecosystems. This dissertation differs by introducing a controlled empirical evaluation specifically centered on JavaScript test smells, where transformation quality is examined through smell removal, behavioral preservation, coverage impact, and unintended structural side effects.

Prompt engineering has also emerged as a central factor in improving LLM-based code transformation quality. Studies indicate that prompt structure, contextual augmentation, and training on smell-cleaned datasets can substantially affect generated code quality (Ouédraogo et al., 2024; Xue et al., 2026). Broader software engineering research likewise shows that prompt design influences automated testing, bug detection, and maintenance outcomes, although model non-determinism remains a persistent challenge (Siddiq et al., 2024; Dobslaw et al., 2025). While prior work typically evaluates one prompting strategy in isolation, this dissertation systematically compares *zero-shot*, *few-shot*, and CoT prompting under the same experimental conditions, allowing a direct analysis of how prompting strategy interacts with model capability during JavaScript smell refactoring.

Despite substantial progress in both automated refactoring and LLM-assisted software maintenance, JavaScript remains underexplored in the context of automated test smell correction. Existing JavaScript studies predominantly address smell detection rather than remediation (Lucas et al., 2024), and no prior work provides a large-scale empirical comparison of multiple LLMs performing smell-oriented refactoring in real JavaScript test suites. This dissertation addresses that gap by evaluating contemporary LLMs across multiple prompting strategies and multiple smell categories, with explicit assessment of smell removal, behavioral preservation, coverage effects, and unintended structural side effects. In doing so, it extends current literature from detection-oriented JavaScript research toward a broader understanding of LLM-based automated maintenance in dynamic-language test ecosystems.

2.8 Conclusion

This chapter presented the theoretical and empirical foundations that support the investigation conducted in this dissertation. It discussed the role of software testing in software quality assurance, the concept of test smells as recurring design problems that affect test maintainability and effectiveness, and the tools available for detecting such problems, particularly within JavaScript testing environments. The chapter also reviewed the principles of code and test refactoring, emphasizing the requirement of preserving behavior while improving internal structure.

In addition, the chapter introduced LLMs and their growing application to software engineering tasks, highlighting their potential for assisting software maintenance and automated code transformation. The discussion of prompt engineering provided the conceptual basis for the prompting strategies evaluated in this research, namely *zero-shot*, *few-shot*, and CoT prompting. Finally, the review of related work examined prior studies on test smell detection, automated refactoring, and LLM-assisted software maintenance, identifying a research gap regarding the systematic evaluation of LLM-based refactoring of JavaScript test smells.

Together, these foundations establish the conceptual framework required to understand the design decisions, evaluation criteria, and research questions adopted in this dissertation. Building upon this background, the next chapter presents the methodology used to evaluate the effectiveness and limitations of LLM-based refactoring of JavaScript test smells through a controlled empirical study.

Chapter 3

Methodology

This chapter presents the methodological procedures adopted to evaluate Large Language Models (LLMs) in JavaScript test code refactoring. The study combines empirical experimentation with structured tool-assisted procedures to assess refactoring outcomes across multiple complementary analytical dimensions, namely structural effectiveness, behavioral preservation, coverage impact, unintended structural side effects, and the influence of experimental factors on overall refactoring performance. The methodological design follows a multi-phase workflow that ensures analytical rigor through automated smell detection, controlled refactoring generation, and multidimensional post-refactoring validation. The following sections describe the complete experimental framework adopted in this dissertation.

- Section 3.1 presents the five research questions that guide this investigation, defining the analytical dimensions and evaluation objectives that structure the empirical study.
- Section 3.2 describes the pre-refactoring phase, including the overall research design, the selection of test smell detection tools, the definition of the target smell set, repository curation based on eight inclusion criteria, and the selection of LLMs.
- Section 3.3 details the refactoring process, including the execution pipeline, the three prompting strategies adopted in this research, *zero-shot*, *few-shot*, and CoT, together with prompt construction, generation controls, code extraction procedures, and intermediate result integration.
- Section 3.4 presents the post-refactoring analysis procedures, including functional behavior validation, reapplication of smell detection tools, and the consolidation of structural and execution outcomes into analyzable experimental records.

Each phase was designed to preserve methodological consistency while addressing the interaction between structural code improvement, behavioral reliability, and experimental comparability across models and prompting strategies.

3.1 Research Objective and Questions

This research aims to systematically evaluate the effectiveness, reliability, and structural consequences of LLMs in the automated refactoring of JavaScript test smells. The investigation adopts a layered evaluation perspective, progressing from direct refactoring outcomes to secondary structural effects and explanatory comparative analysis.

Specifically, the research assesses (i) whether LLMs successfully remove targeted test smells, (ii) whether such refactorings preserve the functional behavior of the test suite, (iii) how refactoring affects statement coverage after refactoring, (iv) whether unintended structural side effects emerge in the resulting test code, and (v) how experimental factors such as model choice and prompting strategy influence overall refactoring outcomes.

To guide this empirical investigation, we define the following research questions:

1. **RQ1: To what extent can LLMs successfully remove targeted JavaScript test smells?**

This question evaluates the primary structural effectiveness of LLM-based refactoring. For each detected smell instance, the test smell detection tools are re-applied after refactoring to verify whether the originally reported smell is no longer identified in the corresponding test method. Effectiveness is quantified through structural smell removal rates across all analyzed instances.

2. **RQ2: Do LLM-generated refactorings preserve the functional behavior of the test suite?**

This question assesses semantic preservation. For each refactored test, the complete test suite is executed before and after refactoring. A refactoring is considered behavior-preserving when the test suite executes successfully and no regressions are introduced. This dimension ensures that structural improvements do not compromise correctness.

3. **RQ3: How does LLM-based refactoring affect test coverage?**

Although a refactoring may preserve execution behavior, it can still alter the extent to which production code is exercised by the test suite. This question analyzes variations in statement, branch, function, and line coverage before and after refactoring in order to determine whether test effectiveness is preserved, degraded, or improved. Coverage variation is treated as an independent structural quality dimension that complements behavioral preservation.

4. **RQ4: What unintended structural side effects emerge from LLM-based refactoring?**

Beyond targeted smell removal, LLM-generated refactorings may introduce secondary structural degradations. After refactoring, the test smell detection

tools are re-executed to identify whether new smell categories emerge and whether the resulting test code presents additional structural deficiencies not present in the original version. This analysis focuses on the frequency, taxonomy, and distribution of newly introduced smells as unintended consequences of automated refactoring.

5. **RQ5: How do experimental factors influence LLM-based refactoring outcomes?** Analyzes how model choice and prompting strategy influence refactoring outcomes, with particular attention to the integrated success criterion defined from smell removal, behavioral preservation, and coverage stability.

Together, these research questions enable a multidimensional assessment of LLM-driven test refactoring, balancing effectiveness, correctness, structural quality, and explanatory analysis of influencing factors.

As illustrated in Figure 3.1, this research adopts a structured, multi-stage process organized into three main phases:

- **Pre-refactoring phase:** we selected the test smell detection tools, defined the specific smells to be investigated, curated the target repositories according to explicit inclusion criteria, and determined the LLMs to be evaluated.
- **Refactoring phase:** we executed an automated pipeline that applies *zero-shot*, *few-shot*, and CoT prompting strategies to each (smell, model) pair. For every validated smell instance, we systematically collected the original smelly test code together with the refactored versions generated by each LLM.
- **Post-refactoring phase:** we re-executed the test suites to validate behavioral preservation, reapplied the detection tools to assess smell elimination and potential smell introduction, measured changes in test coverage, and quantified modifications in the number of executed test cases. The integration of these evaluation dimensions enables a multidimensional assessment of LLM effectiveness, reliability, and structural impact.

These stages are described in greater detail in the following sections.

3.2 Pre-Refactoring Activities

In the initial phase of the research, we defined the core experimental elements that underpin the refactoring process and subsequent analysis. Specifically, the following subsections detail the components selected for the experiment:

- **Test smell detection tools** (Section 3.2.1): responsible for identifying smell instances in test code;
- **Target repositories** (Section 3.2.2): from which test methods were sampled;
- **Investigated test smells** (Section 3.2.3): defining the refactoring scope; and

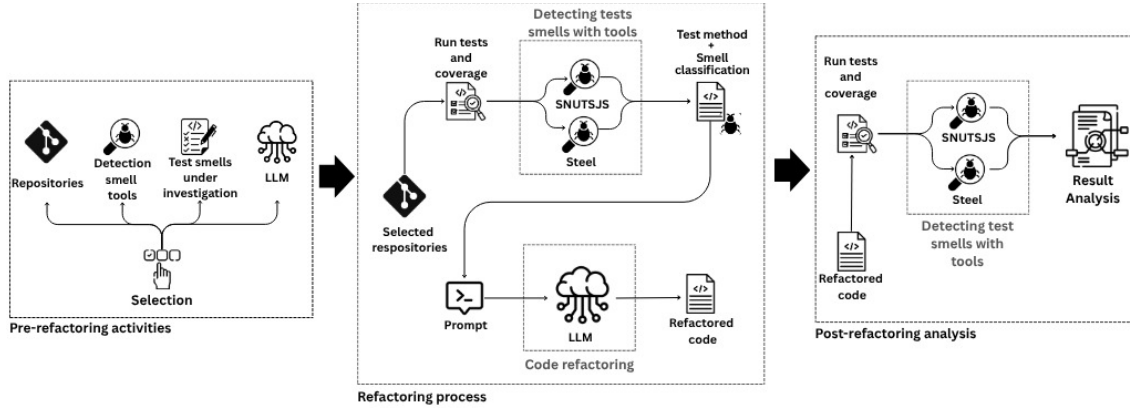


Figure 3.1: Research Design Flow

- **Large Language Models** (Section 3.2.4): used to perform automated refactoring.

3.2.1 Selection of Test Smell Detection Tools

We conducted an *ad hoc* review of the literature specialized in test smell detection techniques for the JavaScript language before selecting the tools. This analysis identified three tools: Steel (Jorge et al., 2021), TestSmellDetectorJs (Silva, 2022), and SNUTS.JS (Oliveira et al., 2024). To assess the suitability of these tools for the scope of this research, we conducted a practical evaluation of their functionalities. During this evaluation, we observed that *TestSmellDetectorJs* did not provide precise information regarding the location of identified smells, only reporting their presence in a given file. Since this research requires the exact identification of affected code fragments to support the refactoring process, this limitation rendered the tool unsuitable for use.

Therefore, we adopted the tools Steel and SNUTS.JS, as they proved more aligned with the research needs. Both tools not only support the detection of a variety of test smells but also provide detailed data about their location in the code, an essential requirement for the subsequent stages of analysis and refactoring. Since SNUTS.JS is implemented as an API, we adapted it to a command-line interface (CLI) to enable a more practical and faster detection process when analyzing local repositories.

3.2.2 Selection of Repositories

We selected the repositories used in this research through a judgment-based sampling approach to ensure compliance with pre-established criteria. This strategy enabled the intentional selection of open-source JavaScript projects hosted on GitHub, ensuring that we analyzed only repositories meeting the inclusion requirements. We detail the criteria used below:

1. **JavaScript as the primary language:** Selected projects must have JavaScript as their primary programming language, with at least 80% of the repository code written in JavaScript according to repository composition metrics. This criterion ensures technological homogeneity and analytical consistency with the research scope.
2. **Recent development activity:** To ensure that the selected projects are actively maintained, only those with a last commit dated January 1, 2025, or later were considered eligible. This criterion ensures that the projects remain up to date and aligned with current software engineering practices.
3. **Popularity and recognition:** To ensure that the selected projects are widely recognized by the developer community, a minimum of 5,000 GitHub stars was required. This threshold balances relevance with analytical feasibility.
4. **Open-source license:** All projects must have a recognized open-source license, allowing the software to be used, modified, and redistributed in accordance with free software principles.
5. **Presence of a test suite:** Eligible projects must contain an automated test suite. This ensures the feasibility of objective validation of modifications, particularly in the context of refactoring analysis.
6. **Use of the Jest testing framework:** Jest is chosen as the primary testing framework due to its widespread adoption and leading popularity within the JavaScript ecosystem, as evidenced by a survey conducted between September 24 and December 15, 2025 (Greif and Burel, 2025). Moreover, relying on a single framework enables standardization of test outputs, which simplifies and facilitates automated data analysis.
7. **Technical viability and tool compatibility:** The methodological integrity of this research requires that each selected repository be installable in a clean environment without manual intervention, that its test suite execute without pre-existing failures, and that both SNUTS.JS and Steel analyze the test code without runtime errors. Meeting these conditions ensures that any detected test smells genuinely reflect the project's testing practices rather than environmental artifacts or tool incompatibilities, thereby establishing a reliable foundation for subsequent refactoring analysis.

Initially, we used the *GitHub Search* (GHS) (Dabic et al., 2021) to assist in the sampling process. This tool, widely adopted in empirical studies on open-source repositories (Alfadel et al., 2023; Ciniselli et al., 2021; Mastropaolo et al., 2023), enables advanced queries on GitHub metadata, allowing results to be filtered according to specific criteria. In this research, we used it to select repositories satisfying criteria 1, 2, and 3, corresponding to JavaScript predominance, recent activity, and popularity. This initial screening yielded a preliminary sample of 681 repositories.

For the subsequent filtering stage, we implemented a Python script to apply the remaining automated criteria: 4, 5, and 6. The script performed automated checks on

each repository to ensure the presence of a recognized open-source license, existence of an automated test suite, and explicit use of the Jest framework. These checks relied on GitHub’s public API and static analysis of files such as `package.json`, which typically contains metadata on dependencies and test configurations. As a result, 83 projects met all automated inclusion criteria.

We applied Criterion 7 through a manual validation process. First, we cloned each of the 83 repositories and installed them in a clean execution environment. Next, we executed the automated test suite to verify successful installation and runtime stability. We then ran the test smell detection tools to confirm their compatibility with each project’s structure and test configuration. We excluded repositories that presented installation failures, unstable test execution, or incompatibility with the detection tools. After completing this validation stage, 28 repositories satisfied all criteria. Appendix B.1 presents the complete list of selected repositories.

To support replication, each filtering decision was recorded during the repository selection process, including the repository identifier, the criterion responsible for exclusion, and the execution status of installation, test execution, and detector compatibility checks. For repositories excluded during the technical viability stage, we distinguished between installation failures, pre-existing test failures, missing or incompatible Jest configurations, and failures caused by the smell detection tools. This logging procedure ensures that the final repository set can be traced back to the initial candidate pool and that exclusions are attributable to explicit methodological criteria rather than undocumented manual judgment.

3.2.3 Selection of Test Smells

The selection of test smells proceeded in two phases: first, defining which types of smells to investigate, and second, identifying concrete instances of those smells for subsequent analysis and refactoring.

For the first phase, we selected test smell categories based on the detection capabilities of SNUTS.JS and Steel. Given the limited overlap between the tools, with only five shared smell types, we prioritized distinct smells from each to maximize the diversity of analyzed test smells and enhance the representativeness of our research across different categories of test design issues.

We further refined the selection through scope-based inclusion criteria. To ensure actionability within our research context, we focused exclusively on smells occurring within test method bodies, the primary units targeted for automated refactoring. Consequently, we excluded smells that extended beyond this scope, such as *General Fixture* (overly generic test setups) and *Global Variable* (shared state across test suites), as their distributed nature impedes consistent and automated refactoring. We also discarded smells lacking executable code fragments and meaningful refactoring context.

Applying these criteria yielded ten smell categories (five per tool):

From **SNUTS.JS**:

1. *Conditional Test Logic*
2. *Overcommented Test*
3. *Suboptimal Assert*
4. *Anonymous Test*
5. *Verbose Statement*

From **Steel**:

1. *Duplicate Assert*
2. *Exception Handling*
3. *Magic Number*
4. *Sleepy Test*
5. *Unknown Test*

This selection encompasses both assertion-level and structural-level issues, providing a comprehensive foundation for evaluating LLM-based refactoring across diverse test smell categories.

For the second phase, we extracted smelly code instances from the selected repositories using the automated detection tools. This initial detection yielded 20,760 test smell instances across the analyzed test methods of the selected repositories. To ensure the robustness of the experimental dataset, we then performed a manual validation stage, confirming that the selected instances accurately represented the targeted smell categories before inclusion in the final corpus.

To support the validation process, we developed a comprehensive web-based tool with a front-end and back-end architecture, shown in Figure 3.2, that allows to interactively explore and management of candidate instances. The interface enables filtering by test smell category, repository, and detection tool, while providing intuitive navigation through the dataset. Users can inspect each test method alongside its detected smell type, facilitating efficient and accurate manual review of the extensive dataset.

Our sampling strategy aimed for balanced representation across all ten smell categories. We therefore selected 20 validated instances per category, establishing a target corpus of 200 confirmed smell instances. This normalization ensures that subsequent analyses are not biased by unequal representation of different smell types.

Three specialists independently validated each selected candidate instance through a binary classification, indicating whether the code exhibited a genuine test smell. We employed a majority voting mechanism: instances receiving at least two affirmative votes were retained in the corpus. Instances that failed to meet this threshold

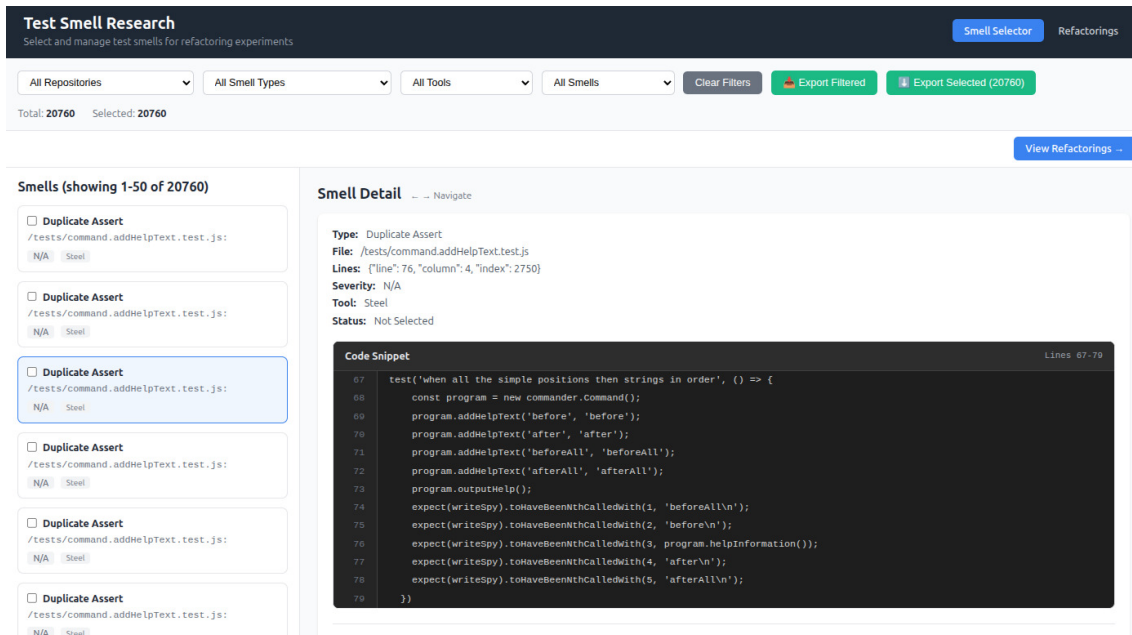


Figure 3.2: Interactive Web-based Tool for Exploring and Selecting Test Smells.

were discarded and replaced with new candidates, which underwent the same validation process. This iterative procedure continued until we achieved 200 validated instances.

This rigorous, multi-stage selection process ensures that our research corpus comprises authentic, representative test smells, thereby establishing a reliable and reusable foundation for evaluating LLM-based refactoring approaches and supporting future research endeavors.

3.2.4 Selected LLMs

The selection of LLMs for this research followed a set of explicit methodological criteria designed to ensure experimental rigor, comparability, and relevance to the task of automated JavaScript test smell refactoring. The adopted criteria are defined as follows:

1. **Model Capacity Requirement.** Only high-capacity models were considered. For open-weight systems, models were required to have approximately 30 billion parameters or more. This threshold was established to reduce performance variability associated with small-scale models and to ensure sufficient representational capacity for complex semantic transformations. Proprietary models were required to represent frontier-scale architectures.
2. **Instruction-Following Capability.** All selected models were required to support instruction-based interaction, enabling consistent application of *zero-shot*, *few-shot*, and CoT prompting strategies across experiments.

3. **Deployment Accessibility and Reproducibility.** Proprietary models were required to provide stable and documented API access, ensuring controlled and reproducible interaction. Open-weight models were required to be accessible through Hugging Face Inference Endpoints, enabling scalable and standardized execution without manual intervention.
4. **Architectural and Training Diversity.** The final selection intentionally includes both proprietary and open-weight systems, as well as code-specialized and general-purpose reasoning models. This diversity allows evaluation across complementary dimensions, including code specialization, reasoning depth, and deployment paradigm.

Based on the defined criteria, six LLMs were selected for empirical evaluation. The selected models enable a systematic assessment of automated JavaScript test smell refactoring under controlled experimental conditions, covering both proprietary and open-weight paradigms. This selection allows the study to analyze differences in semantic preservation, structural transformation quality, and generalization across heterogeneous code contexts. All models are evaluated under *zero-shot*, *few-shot*, and CoT prompting strategies to ensure consistent experimental treatment and a comprehensive comparative analysis.

- Proprietary Models for Commercial Benchmarking

Three proprietary models were selected to represent state-of-the-art commercial LLMs with demonstrated reasoning capabilities and relevance to structured code transformation tasks.

GPT-5.2 is OpenAI’s latest frontier model, characterized by architectural advances such as hierarchical routing, expanded context handling, and improved tool-use and reasoning capabilities, which support instruction adherence and semantic preservation in code refactorings (Leon, 2026; Singh et al., 2025).

Claude Sonnet 4.6, developed by Anthropic, emphasizes robust reasoning and consistency in multi-step program synthesis, achieving high Pass@1 accuracy on HumanEval and demonstrating strong performance and maintainability in performance-constrained code generation tasks (Bayram et al., 2025; Nia et al., 2025).

Gemini 2.5 Pro, from Google DeepMind’s Gemini family, provides a proprietary baseline with strong reasoning performance and long-context support, and has shown competitive capabilities in automated repair and transformation tasks in software engineering contexts (Sriwilailak et al., 2024; Comanici et al., 2025; Amorim et al., 2025).

- Open-Weight Models for Code Specialization

To complement proprietary systems, three open-weight models were selected for their focus on code intelligence, transparency, and suitability for local deployment.

Qwen 2.5 Coder 32B is a code-specialized open-weight model pretrained on large-scale code corpora, achieving state-of-the-art results across multiple benchmarks for code generation, reasoning, and repair, and offering permissive licensing for empirical research (Hui et al., 2024).

CodeLlama 34B Instruct, introduced by Meta, is an instruction-tuned open-weight model optimized for code understanding, infilling, and long-context processing, enabling the evaluation of complex code transformations in JavaScript test refactoring scenarios (Roziere et al., 2023).

DeepSeek V3.2 685B is an open-weight model that emphasizes efficient long-context processing and reasoning-centric post-training through sparse attention, scalable reinforcement learning, and agentic task synthesis, making it suitable for controlled and semantically precise code refactoring tasks under local execution constraints (Liu et al., 2025a).

All selected models are executed within a fully automated refactoring pipeline designed to ensure experimental consistency and reproducibility. Each model receives standardized inputs and prompting configurations, and the generated outputs are systematically collected for subsequent evaluation.

Proprietary models are accessed through their official APIs, enabling controlled interaction under consistent configuration parameters. Open-weight models are executed via Hugging Face Inference Endpoints, ensuring scalable deployment while maintaining uniform execution conditions across experiments. A detailed description of the refactoring pipeline, execution flow, and evaluation procedures is provided in the Section 3.3.

3.3 Refactoring Process

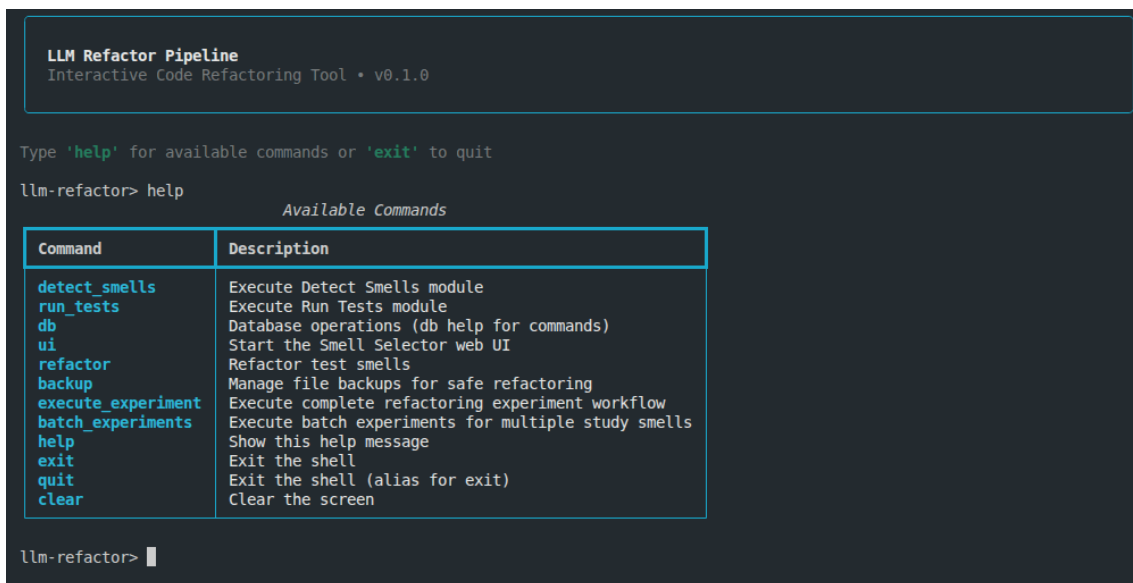
This section describes the methodology adopted to perform automated refactoring of JavaScript test smells using LLMs. The process was designed to ensure experimental rigor, reproducibility, and fairness across heterogeneous model providers, while enabling a systematic evaluation of different prompting strategies. All refactoring operations were executed through a unified and fully automated pipeline that abstracts provider-specific API differences and enforces consistent execution constraints.

3.3.1 Overview of the Refactoring Pipeline

The refactoring process is applied at the level of individual test methods identified as smelly in the dataset described in Section 3.2.3. Each refactoring instance is handled independently and follows a fixed execution sequence comprising prompt construction, model invocation, output extraction, and result validation. This design ensures experimental independence across refactoring tasks and prevents interference between different instances.

To enable execution across multiple LLM providers, we implemented a unified interaction layer that abstracts differences between proprietary and open-weight models. This abstraction guarantees that all models receive semantically equivalent inputs, consistent generation constraints, and identical post-processing logic, independent of their underlying APIs or deployment mechanisms.

Figure 3.3 depicts the CLI developed to operationalize this pipeline. It provides a unified entry point for configuring experiments, selecting models and prompting strategies, and executing refactoring tasks on the target repositories. By consolidating these operations, the interface supports controlled execution and ensures reproducibility across experimental runs.



```

LLM Refactor Pipeline
Interactive Code Refactoring Tool • v0.1.0

Type 'help' for available commands or 'exit' to quit
llm-refactor> help
Available Commands

```

Command	Description
<code>detect_smells</code>	Execute Detect Smells module
<code>run_tests</code>	Execute Run Tests module
<code>db</code>	Database operations (db help for commands)
<code>ui</code>	Start the Smell Selector web UI
<code>refactor</code>	Refactor test smells
<code>backup</code>	Manage file backups for safe refactoring
<code>execute_experiment</code>	Execute complete refactoring experiment workflow
<code>batch_experiments</code>	Execute batch experiments for multiple study smells
<code>help</code>	Show this help message
<code>exit</code>	Exit the shell
<code>quit</code>	Exit the shell (alias for exit)
<code>clear</code>	Clear the screen

```

llm-refactor>

```

Figure 3.3: CLI for Configuring and Executing the LLM-based Refactoring Pipeline

3.3.2 Prompting Strategies

Before defining the prompting strategies employed in this research, we first established a consistent and theoretically grounded contextual basis to support the refactoring tasks assigned to the LLMs. To this end, we constructed a dedicated *test smell catalog* derived from evidence reported in peer-reviewed scientific literature, including primary studies that formally define test smells, describe their structural manifestations, and discuss recommended refactoring practices. The catalog was specifically tailored to the set of JavaScript test smells selected for analysis and is presented in Appendix A. For each smell, it consolidates a formal definition, detection cues, representative manifestations, refactored JavaScript examples, and operational refactoring strategies, thereby providing a uniform and scientifically grounded characterization of each targeted design issue.

This catalog serves as the primary knowledge source for prompt construction and is

programmatically consumed to instantiate structured prompt templates across all strategies and models. By relying on curated and validated smell descriptions, the prompts minimize ambiguity and reduce dependence on implicit model knowledge, improving consistency and comparability across experimental conditions.

Using this catalog as a foundation, we evaluate three prompting strategies commonly adopted in LLM-based program refactoring research: *zero-shot*, *few-shot*, and CoT. Each strategy is instantiated through a structured prompt template derived from the catalog content and applied uniformly across all selected models. This design ensures that observed variations in refactoring outcomes can be attributed to differences in prompting strategy and model behavior, rather than inconsistencies in problem formulation or contextual information.

3.3.3 Zero-Shot Prompt Structure

The *zero-shot* strategy relies solely on explicit instructions and does not incorporate demonstration examples. Each prompt consists of a **system message** that defines the model’s role as a JavaScript test refactoring expert and enforces a strict output contract restricted to executable code, and a **user message** that specifies the target test smell and its formal definition, presents the original smelly test method, and enumerates explicit constraints and prohibitions to ensure output completeness and semantic preservation. The complete structure of the *zero-shot* prompt template is illustrated in Listing 1.

3.3.4 Few-Shot Prompt Structure

The *few-shot* strategy extends the *zero-shot* structure by adding up to three curated demonstration pairs for the target smell. Each pair contains a smelly test and its refactored counterpart drawn from the catalog, enabling the model to infer a transformation pattern before refactoring the target instance. The system message and output constraints remain identical to *zero-shot*, while the user message includes an additional instruction to study the examples and follow the demonstrated refactoring pattern. The complete structure of the *few-shot* prompt template is illustrated in Listing 2.

3.3.5 Chain-of-Thought Prompt Structure

The CoT strategy introduces explicit *internal* reasoning guidance while preserving output comparability by forbidding the disclosure of intermediate steps. In addition to the smell name and definition, the user message includes detection criteria and refactoring strategies sourced from the catalog, optionally followed by up to two reference examples. The prompt then prescribes a fixed, multi-phase internal reasoning routine (e.g., locate smell, infer intent, plan refactoring, validate), and explicitly instructs the model to keep this reasoning hidden and output only the final refactored JavaScript code under the same output contract used in the other

Listing 1: Zero-shot prompt template.

```

Zero-shot

System: You are a JavaScript test refactoring expert. Follow instructions strictly and output ONLY valid JavaScript code.
User:
### Your Task
Refactor the test code below to eliminate the test smell.
-----
### Context
Test Smell:
{smell_name}

Definition:
{smell_description}
Original Test Code:
{test_code}
-----

### Output Requirements
You MUST provide your response in this exact format:

```javascript
// Your COMPLETE refactored test code here
```

PRIMARY OBJECTIVE:
- Completely remove the test smell from the code

CONSTRAINTS (you MUST):
- Include the COMPLETE test function or block
- Preserve semantic behavior and all assertions
- Ensure code is syntactically correct and executable

PROHIBITIONS (you MUST NOT):
- Output partial code, fragments, or snippets
- Add explanations, descriptions, or commentary
- Include text outside the code
- Describe what you changed

```

strategies. The complete structure of the CoT prompt template is illustrated in Listing 3.

3.3.6 Parameter Configuration and Execution Constraints

The selection of generation parameters for LLMs is a critical methodological decision in code refactoring research, as no single configuration is universally optimal across tasks or models (Donato et al., 2025). Sampling parameters directly influence the non-deterministic behavior of LLMs and affect both the diversity and correctness of generated code. In refactoring scenarios, where refactorings must preserve program semantics and externally observable behavior, understanding and carefully controlling these parameters is essential to avoid unstable or hallucinated modifications (Martinez et al., 2025; Tapader et al., 2025).

Temperature controls the degree of stochasticity during token selection by shaping the sharpness of the probability distribution. Low temperature values produce more deterministic outputs by favoring high-probability tokens, whereas high temperatures flatten the distribution and increase the likelihood of sampling low-probability tokens. Although higher temperatures may be beneficial in creative code synthesis,

Listing 2: Few-shot prompt template with up to three demonstration pairs.

```

Few-shot

System:
You are a JavaScript test refactoring expert. Follow instructions strictly and output ONLY valid JavaScript code.

User:
### Your Task
Refactor the test code below to eliminate the test smell. Study the examples to understand the refactoring pattern.
-----

### Context
Test Smell:
{smell_name}

Definition:
{smell_description}

### Example 1
Original (with {smell_name}):
{example_1_smelly}
Refactored (smell removed):
{example_1_refactored}

### Example 2
Original (with {smell_name}):
{example_2_smelly}
Refactored (smell removed):
{example_2_refactored}

### Example 3
Original (with {smell_name}):
{example_3_smelly}
Refactored (smell removed):
{example_3_refactored}

Original Test Code:
{test_code}

Output Requirements
You MUST provide your response in this exact format:

```javascript
// Your COMPLETE refactored test code here
```

[ PRIMARY OBJECTIVE, CONSTRAINTS and PROHIBITIONS are the same as Listing 1]

Provide the complete refactored test code:

```

empirical studies show that they substantially increase output variance and hallucination rates in structured refactoring tasks, where the valid transformation space is narrow and semantically constrained (Donato et al., 2025). At the other extreme, setting temperature to zero does not fully eliminate non-determinism and may overly restrict the model, leading to underperformance in method-level code generation tasks (Donato et al., 2025). Based on this evidence, we fix the temperature to 0.3, which limits stochastic variation while avoiding strictly greedy decoding, thereby improving reproducibility without suppressing legitimate structural refactorings.

Exploration during generation is therefore primarily regulated through nucleus sampling (*top_p*). This parameter restricts token selection to the smallest set of tokens whose cumulative probability exceeds a given threshold, effectively preventing the

Listing 3: Chain-of-thought prompt template.

```

Chain of thought

System:
You are a JavaScript test refactoring expert. Follow instructions strictly and output ONLY valid JavaScript code.

User:
### Your Task
Refactor the test code below to eliminate the test smell.

-----

### Context
Test Smell:
{smell_name}

Definition:
{smell_description}

Detection Criteria:
{smell_detection}

Refactoring Strategies:
Apply these strategies in your solution:
1. {strategy_1}
2. {strategy_2}
...
n. {strategy_n}

### Reference Examples

#### Example 1
Original (with {smell_name}):
{example_1_smelly}
Refactored (smell removed):
{example_1_refactored}

#### Example 2
Original (with {smell_name}):
{example_2_smelly}
Refactored (smell removed):
{example_2_refactored}

Original Test Code:
{test_code}

Internal Reasoning (Hidden Chain-of-Thought)
You MUST internally reason step by step following this process:
1. Locate the Smell: Identify where and how the test smell manifests based on detection criteria
2. Understand Intent: Determine what behavior the test verifies and which assertions validate it
3. Evaluate Impact: Assess why the current structure violates best practices
4. Plan Refactoring: Design a solution using the strategies provided above
5. Validate: Ensure the refactored version meets all requirements

Output Requirements
You MUST provide your response in this exact format:

```javascript
// Your COMPLETE refactored test code here
```

[ PRIMARY OBJECTIVE, CONSTRAINTS and PROHIBITIONS are the same as Listing 1]

Provide your response:

```

inclusion of extremely low-probability tokens that are often associated with syntactic errors or irrelevant transformations (Tapader et al., 2025). Very low *top_p* values may overly constrain the model, resulting in conservative or repetitive out-

puts, while excessively high values reintroduce unstable behavior. We set *top_p* to 0.7, balancing controlled exploration with correctness by allowing limited flexibility for valid refactoring actions, such as statement reordering or assertion restructuring, without compromising semantic preservation (Martinez et al., 2025).

Overall, this configuration reflects a deliberate methodological choice aligned with the goals of this research. By jointly constraining stochasticity through temperature and regulating exploration through nucleus sampling, we ensure stable, reproducible, and semantically sound refactorings across heterogeneous LLMs, while minimizing sampling-induced bias in the comparative evaluation.

3.3.7 Extraction and Validation of LLM-Generated Code

To ensure the reliability and internal validity of the experimental results, we implemented a dedicated pipeline for extracting and validating JavaScript code generated by LLMs. In practice, LLM responses frequently mix executable code with natural language explanations, markdown formatting, and instructional comments. Directly consuming raw outputs would therefore introduce noise and ambiguity into the evaluation. The proposed pipeline addresses this challenge by systematically isolating code candidates and filtering out non-executable content while remaining robust to variations in output structure across models and prompting strategies.

The extraction process first identifies all fenced markdown code blocks in the LLM response, prioritizing blocks explicitly labeled as JavaScript but also considering generic blocks when necessary. If no fenced block is found, the raw response is treated as a fallback candidate, provided it satisfies minimal structural and content constraints. Each candidate is then evaluated using a lightweight, multi-criteria validation strategy. Structural soundness is assessed through delimiter balancing (braces, brackets, and parentheses), which intentionally tolerates JavaScript syntax errors such as typos or malformed keywords. This design choice reflects the goals of the study: to capture the actual refactoring behavior of LLMs, including imperfect or buggy outputs, rather than discarding them prematurely through strict parsing.

To further prevent explanatory prose from being misclassified as code, a natural language filtering heuristic is applied. After removing JavaScript comments, the proportion of words relative to characters is computed, with candidates exceeding a fixed threshold (25%) being rejected as predominantly natural language. When multiple candidates pass all checks, a deterministic selection mechanism chooses the highest-quality block based on a scoring function that rewards common JavaScript and test-related patterns (e.g., `it`, `describe`, `expect`, arrow functions, and method calls). By combining conservative structural validation, content-based filtering, and deterministic selection, this pipeline ensures that only meaningful LLM-generated JavaScript test code is retained for analysis, thereby supporting reproducibility and robustness in the empirical evaluation of refactoring effectiveness.

3.3.8 Results Integration

The integration of the three evaluation axes, functional correctness, smell elimination, and structural quality, provided a holistic perspective on the impact of LLMs on automated tests. This multidimensional analysis allowed us to identify successful refactorings that preserved behavior, removed smells, and maintained coverage stability; detect cases where the smell was eliminated, but other quality dimensions were negatively affected; and classify ineffective suggestions that resulted in no tangible improvements or introduced potential risks.

Ultimately, these results served as empirical evidence for answering the research questions proposed in this study, offering both quantitative and qualitative insights into the capabilities, limitations, and implications of using LLMs in the refactoring of automated tests.

3.4 Post-Refactoring Analysis

The post-refactoring phase operationalizes the *execution* and *analysis* steps of our experiment orchestration module. After the refactoring phase produces a candidate patch (Section 3.3), the pipeline applies the generated snippet to the target repository under a controlled workflow: *backup* the original file, *replace* only the affected snippet, *re-run* smell detection and the full test suite, *restore* the original version, and finally *persist* all outputs and metadata to both the dataset directory and the database. This design isolates the effects of each refactoring instance, prevents interference across experiments, and guarantees that repositories are left unchanged after each run, even when failures occur.

3.4.1 Functional Behavior Validation

To assess whether LLM-generated refactorings preserve the intended test behavior, we re-execute the repository test suite after applying each patch. Test execution is performed using a repository-specific command stored in a `.run_tests` configuration file, ensuring that each project is evaluated under its native tooling and scripts. The pipeline captures both a condensed report (`test_summary.txt`), which extracts coverage and pass/fail information when available, and the complete console log (`test_output.txt`). A refactoring is considered behavior-preserving when the post-refactoring execution completes successfully (exit code 0), and the boolean outcome (`tests_still_passing`) is recorded to support downstream quantitative analyses.

3.4.2 Reapplication of Smell Detection Tools

After functional validation, we reapply two complementary smell detection tools, SNUTS.JS and Steel, to the refactored repository state. For efficiency, both de-

tectors are executed concurrently, and their outputs are normalized and concatenated into a unified CSV artifact (`smells.csv`) stored within the experiment directory. Effectiveness is assessed by comparing the baseline detection results (generated on the original repository snapshot) against the post-refactoring results, with emphasis on (i) whether the *target* smell instance in the specific file was removed, and (ii) whether additional smells were introduced elsewhere as side effects. These outcomes are summarized via analysis flags (e.g., `target_smell_removed`, `introduced_new_smells`) and complemented by a structured report saved as `analysis/smell_analysis.json`.

Smell Removal Rate

To evaluate smell removal effectiveness, RQ1 adopts the *Smell Removal Rate* (SRR), which measures how often the targeted smell is no longer detected after refactoring.

Let N denote the total number of refactoring attempts and let r_i be a binary indicator associated with experiment i , where $r_i = 1$ when the targeted smell is removed and $r_i = 0$ otherwise. SRR is defined as:

$$\text{SRR} = \frac{\sum_{i=1}^N r_i}{N}$$

This metric is reported as a percentage and serves as the primary structural indicator for assessing RQ1.

Pass Preservation Rate

To assess behavioral preservation, RQ2 employs the *Pass Preservation Rate* (PPR), which captures whether refactored tests preserve the original execution outcome.

Only experiments whose baseline execution completed successfully are considered valid for comparison. Let N denote the number of valid experiments under this condition, and let p_i be a binary indicator such that $p_i = 1$ when the post-refactoring execution preserves the original pass condition and $p_i = 0$ otherwise. PPR is defined as:

$$\text{PPR} = \frac{\sum_{i=1}^N p_i}{N}$$

This metric provides an aggregate view of behavioral preservation and is reported as a percentage in the empirical analysis.

To complement this aggregate measure, non-preserved cases are further categorized to distinguish different types of regression. The classification follows a hierarchical procedure: experiments are first examined for increases in failing test cases (*intra-suite regression*); only when no such increase is observed are they evaluated for

increases in failing test suites (*suite-level regression*). Cases in which the generated code cannot be executed due to syntax errors or runtime interruptions are classified separately as execution-level failures.

This hierarchical scheme ensures that regression categories remain mutually exclusive and supports a more precise interpretation of behavioral degradation.

Coverage Variation Criterion

To assess structural variation after refactoring, RQ3 considers changes in test coverage between baseline and post-refactoring executions. For each experiment, coverage metrics (statement, branch, function, and line) are collected before and after refactoring, and deltas are computed accordingly.

Because coverage values are expressed as normalized proportions of exercised structural elements, very small differences may reflect instrumentation noise or marginal execution variation rather than meaningful structural change Meinke (2021). To address this, a tolerance threshold of **0.5 percentage points** is adopted to distinguish relevant variation from negligible fluctuation.

Let N denote the number of valid experiments, and let c_i be a categorical variable associated with experiment i , defined according to the following rule:

- c_i = improved if at least one coverage metric increases by ≥ 0.5 percentage points;
- c_i = degraded if at least one coverage metric decreases by ≥ 0.5 percentage points and no metric increases by ≥ 0.5 percentage points;
- c_i = preserved otherwise.

Based on this classification, coverage outcomes are analyzed in terms of their distribution across experiments, enabling the identification of structural preservation, improvement, and degradation patterns after refactoring.

This criterion functions as a conservative analytical filter, ensuring that only variations exceeding micro-oscillation effects are interpreted as meaningful structural changes.

Integrated Success Criterion

To support RQ5, the study defines an integrated success criterion that combines the principal quality dimensions evaluated in RQ1, RQ2, and RQ3. A refactoring attempt is considered successful only when three conditions are simultaneously satisfied:

- the targeted smell is removed;
- functional behavior is preserved;

- coverage does not decrease beyond the adopted threshold.

Let N denote the total number of included experiments and let s_i be a binary indicator such that $s_i = 1$ when experiment i satisfies all three conditions and $s_i = 0$ otherwise. The resulting *Refactoring Success Rate* (RSR) is defined as:

$$\text{RSR} = \frac{\sum_{i=1}^N s_i}{N}$$

This indicator synthesizes the principal dimensions of refactoring quality and is reported as a percentage when comparing experimental configurations.

3.5 Conclusion

The methodological design adopted in this research was structured to ensure that the evaluation of LLM-based refactoring extends beyond isolated code refactorings and captures complementary dimensions of refactoring quality, namely structural effectiveness, behavioral preservation, coverage impact, and secondary structural effects, under controlled experimental conditions. By combining explicit repository selection criteria, validated smell instances, architecturally distinct LLMs, and standardized prompting strategies, the study establishes an empirical setting capable of supporting robust comparisons across models and experimental configurations.

A central methodological concern throughout the design was to preserve comparability while maintaining ecological validity. The use of real-world open-source repositories, combined with automated execution pipelines and post-refactoring validation directly on original project environments, ensures that the observed outcomes reflect realistic software engineering conditions rather than artificial benchmark scenarios. At the same time, the strict control of generation parameters, prompt structure, and execution procedures reduces methodological noise and strengthens internal validity.

The integration of structural analysis with behavioral validation also broadens the evaluation perspective commonly adopted in automated refactoring studies. Rather than interpreting smell removal alone as evidence of successful refactoring, this methodology explicitly incorporates functional preservation and integrated success criteria, allowing the analysis to capture both improvements and trade-offs introduced by LLM-generated modifications.

Together, these methodological decisions provide a rigorous foundation for examining not only whether LLMs can refactor JavaScript test smells, but also how reliably such refactorings preserve software behavior and how experimental factors shape the resulting outcomes across models and prompting strategies. On this basis, the next chapter presents the empirical results obtained from the execution of the experimental pipeline and discusses the observed patterns across the five research questions.

Chapter 4

Results

This chapter presents the empirical results and findings from the evaluation of LLMs for the automated refactoring of JavaScript test smells. The study examines six state-of-the-art models across three prompting strategies, encompassing a total of 3,600 refactoring attempts derived from 200 distinct smell instances. The analyses are organized according to the five research questions defined in Chapter 3, each addressing a different dimension of refactoring quality:

- Section 4.1 addresses **RQ1**, evaluating the smell removal effectiveness of LLM-based refactorings by measuring how often the targeted test smells are successfully removed.
- Section 4.2 addresses **RQ2**, investigating whether refactored tests preserve their original functional behavior through execution-based validation.
- Section 4.3 addresses **RQ3**, examining the impact of refactoring on code coverage metrics as a proxy for test suite adequacy.
- Section 4.4 addresses **RQ4**, analyzing the unintended structural side effects introduced during refactoring, i.e., the emergence of new test smells.

Section 4.5 addresses **RQ5**, providing an explanatory synthesis of how two experimental factors namely model choice and prompting strategy influence the combined success of refactoring across the dimensions evaluated in RQ1 RQ3.

Each section reports results progressively, combining global indicators with disaggregated views by smell type, model, and prompting strategy. This structure allows both a high-level interpretation of overall performance and a detailed understanding of the conditions under which LLM-based refactoring is most effective or most prone to failure.

4.1 RQ1 – Effectiveness of Smell Removal

This section addresses the following research question:

RQ1: To what extent can LLMs successfully remove JavaScript test smells?

RQ1 assesses LLM-generated refactorings by determining whether the originally targeted test smell is no longer detected after the detection tools are re-applied to the refactored test code. Rather than limiting the analysis to a single aggregate indicator, smell removal effectiveness is examined across smell categories, prompting strategies, and LLMs. This disaggregated perspective makes it possible to identify where detector-level smell elimination is consistently achieved and where smell removal limitations remain more pronounced.

4.1.1 Experimental Scope and Global Smell Removal Effectiveness

The balanced experimental design adopted in this research enables systematic comparison of structural outcomes across heterogeneous experimental conditions while preserving equivalent representation across smell categories. Across all experimental configurations, LLMs successfully removed the targeted smell in 2,760 out of 3,600 refactoring attempts, yielding a global *Smell Removal Rate* (SRR) of **76.7%**.

At an aggregate level, this result indicates that LLM-generated refactorings frequently satisfy the detector-based criterion established for smell elimination. However, this global rate should not be interpreted as evidence of homogeneous refactoring effectiveness. Considerable variation emerges when the results are examined across smell categories and models, indicating that detector-level smell removal depends strongly on the nature of the smell and on the refactoring capability of the generating model. For this reason, the following analyses progressively disaggregate SRR across the main experimental dimensions in order to characterize where smell removal effectiveness is concentrated and where detector-level removal remains more difficult to achieve.

4.1.2 Effectiveness Across Smell Types

Figure 4.1 presents the SRR by smell category. Overall, the global SRR reaches 76.7%, corresponding to 2,760 successful smell removals out of 3,600 refactoring attempts. However, substantial variability is observed across smell types, with rates ranging from 92.2% to 33.6%.

A clear gradient emerges across smell categories. The highest removal rates are observed for *Unknown Test* (**92.2%**), *Exception Handling* (**91.9%**), and *Magic Number* (**91.1%**), forming a distinct high-performance group in which smell elimination

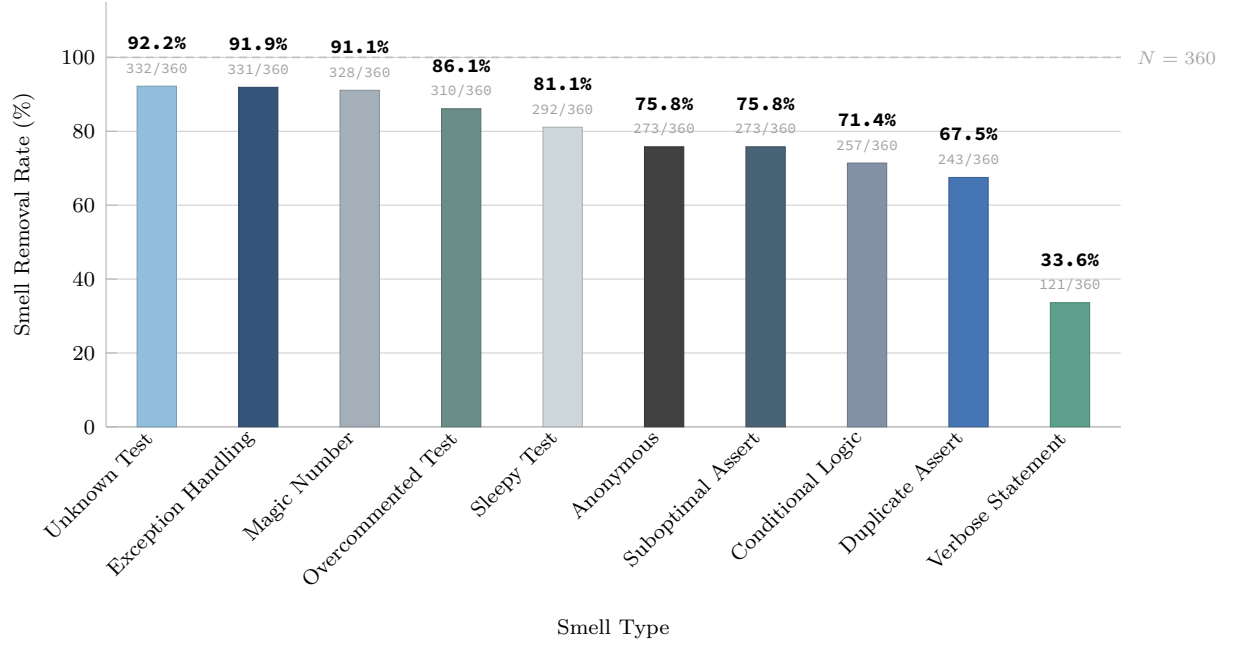


Figure 4.1: SRR by Smell Type

is achieved in almost all experimental configurations. Slightly below this range, *Overcommented Test* (**86.1%**) and *Sleepy Test* (**81.1%**) also maintain high removal rates. Taken together, these five categories suggest that LLMs achieve higher smell removal effectiveness when the targeted smell can be corrected through localized modifications, such as inserting assertions, adjusting exception checks, replacing literal values, removing comments, or eliminating explicit waiting instructions, without requiring broader reorganization of the surrounding test structure.

Intermediate effectiveness appears in *Suboptimal Assert* and *Anonymous Test*, both with **75.8%**. Although these values remain above the global midpoint, they already indicate a lower level of smell removal stability than the previous groups. In both categories, successful refactoring depends not only on removing the targeted test smell, but also on producing a replacement that remains semantically appropriate: in *Suboptimal Assert*, this involves selecting a more adequate assertion form, while in *Anonymous Test* it requires generating a clearer and meaningful test name. This suggests that smell removal becomes less consistent when the refactoring depends on qualitative decisions that go beyond direct code elimination.

The lower portion of the distribution begins with *Conditional Test Logic* (**71.4%**) and *Duplicate Assert* (**67.5%**). Although both values still indicate that smell removal is achieved in most cases, they reveal lower removal effectiveness than the previous groups. In both categories, successful refactoring depends on coordinated modification of multiple statements within the same test method: removing conditionals often requires reorganizing execution flow, while eliminating duplicated

assertions requires consolidating checks without altering test intent. This suggests that detector-level removal becomes less regular when the refactoring depends on coordinated adjustments across multiple related code elements rather than isolated edits.

The most resistant category is *Verbose Statement*, with an SRR of only **33.6%**. This sharp decrease indicates that verbosity remains difficult to eliminate even when refactoring is attempted explicitly. Unlike the other smell categories, reducing verbosity often requires broader rewriting of the test body, including condensation of statements, removal of unnecessary intermediate steps, or simplification of expression structure. The low SRR therefore suggests that current LLMs face difficulty when removing a test smell requires rewriting larger portions of code while still preserving the original test behavior.

Taken together, the distribution reveals that LLMs do not remove all smell types with the same effectiveness, but instead follow a clear smell-dependent gradient. Higher SRR values concentrate in categories whose correction can be achieved through localized and explicit code modifications, whereas lower SRR values emerge in smells that require broader reorganization of assertions, control flow, or statement composition. This indicates that the global SRR of 76.7% does not represent a uniform refactoring capability, but rather the coexistence of smell categories that are consistently removed and others in which detector-level elimination becomes less stable under the same experimental conditions. Since the present analysis is detector-based, this interpretation should be understood as a pattern derived from the observed removal distribution rather than as direct evidence of internal refactoring mechanisms.

4.1.3 Removal Effectiveness by Prompting Strategy

Figure 4.2 summarizes smell removal effectiveness across prompting strategies. *zero-shot*, *few-shot*, and CoT strategies achieve SRRs of 77.3%, 75.2%, and 77.5%, respectively.

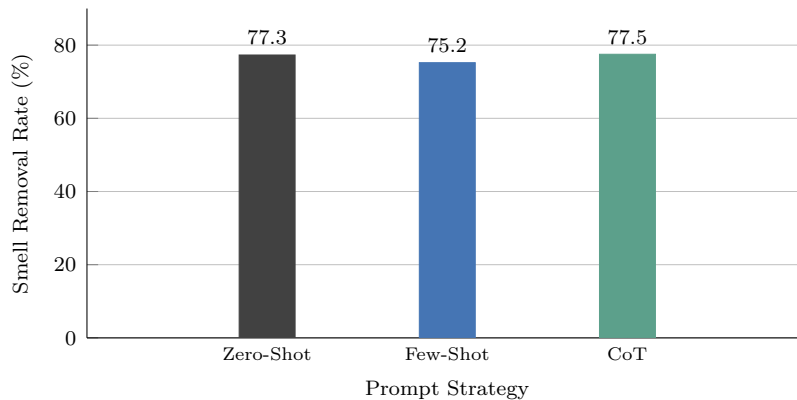


Figure 4.2: SRR by Prompt Strategy

The observed differences are small, with only a narrow variation across strategies. This suggests that, in the context of smell removal, prompt formulation exerts only a modest influence on overall effectiveness. Although CoT achieves the highest SRR, the advantage remains small and does not substantially alter the global performance. The results suggest that prompt elaboration contributes only modestly to detector-level smell removal when compared with the variation observed across smell categories and models.

4.1.4 Model-Level Performance and Interaction Effects

Figure 4.3 presents the Model $\times$ Prompt Smell Removal heatmap, allowing joint observation of model capability and prompt sensitivity. Three performance regimes emerge clearly. A high-performance group composed of *Gemini 2.5 Pro*, *GPT-5.2*, and *Claude Sonnet 4.6* achieves SRRs consistently above 87% across all prompting strategies. The stability of these models across prompts indicates robust internal representations for structural refactoring tasks.

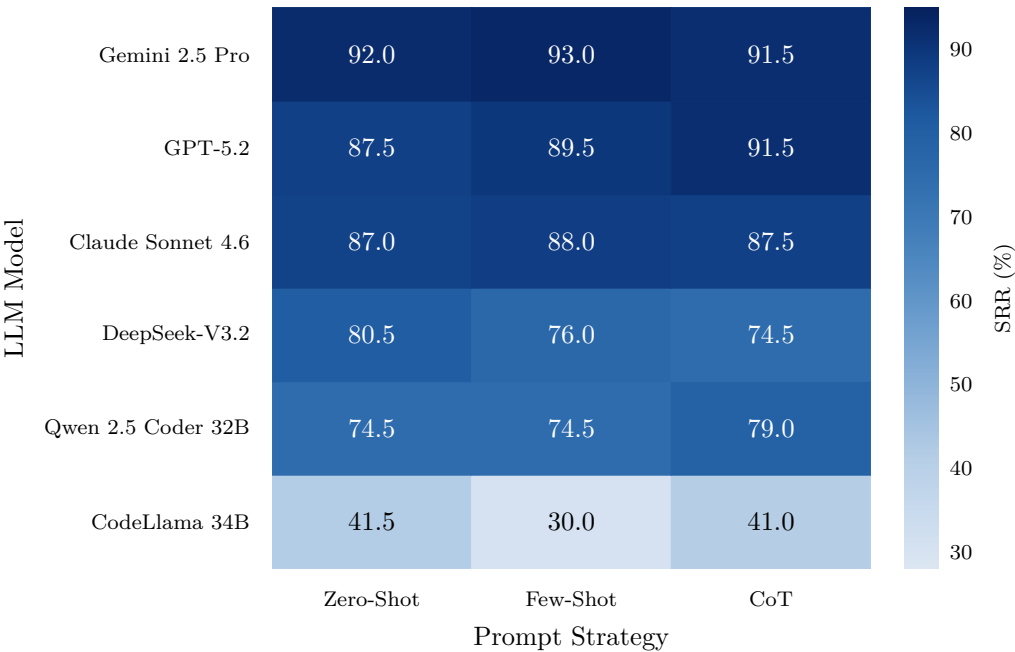


Figure 4.3: Heatmap — Model $\times$ Prompt Removal Matrix

A second group, composed of *DeepSeek-V3.2* and *Qwen 2.5 Coder 32B*, occupies an intermediate range near 76%–77%, with moderate sensitivity to prompting configuration. In these cases, prompt variation produces visible but limited fluctuations. *CodeLlama 34B* forms a clear low-performance outlier, with an overall SRR of 37.5%. Its weak performance across all prompts suggests that prompt refinement alone is insufficient to compensate for underlying model limitations. Taken together, these results indicate that stronger models remove smells consistently across prompt-

ing strategies, whereas weaker models do not substantially benefit from additional prompt engineering.

4.1.5 Variance Decomposition Insight

The observed variability across dimensions confirms that model-related effects dominate prompt-related effects. The gap between the best- and worst-performing models exceeds 55 percentage points, whereas the maximum variation attributable to prompting strategy remains below 3 percentage points. This large difference among models indicates that smell removal effectiveness is governed primarily by model capacity, training scale, and internal code representation quality. Compared with model selection, prompt engineering contributes only marginally to the successful removal of targeted smells.

4.1.6 Answer to RQ1

In response to RQ1, the results show that LLMs frequently achieve detector-level smell elimination of targeted JavaScript test smells, with a global SRR of **76.7%**. However, this effectiveness is strongly conditioned by smell type and by model choice. Some smell categories are removed consistently across most configurations, whereas others remain considerably more resistant to detector-level elimination. Model differences are pronounced, while prompting strategy exerts only limited aggregate influence. These findings indicate that smell removal is a realistic capability of current LLMs, but one whose effectiveness remains uneven across refactoring contexts.

4.2 RQ2 – Behavioral Preservation of Refactored Tests

This section addresses the following research question:

RQ2: Do LLM-generated refactorings preserve the functional behavior of the test suite?

Whereas RQ1 examined whether LLM-generated refactorings successfully remove the targeted test smell, RQ2 evaluates whether these transformations maintain execution behavior after refactoring. The analysis considers the complete experimental space in order to observe how behavioral preservation varies across smell categories, prompting strategies, and LLMs, while also identifying the main forms through which regressions emerge when semantic equivalence is not maintained.

4.2.1 Global Behavioral Preservation

Across all valid experimental configurations, 3,107 out of 3,600 refactoring attempts preserved the functional execution condition of the test suite, yielding a global *Pass Preservation Rate* (PPR) of **86.3%**. Conversely, 493 refactorings (13.7%) produced some form of behavioral non-preservation under the classification rule defined in Section 3.4.2.

At an aggregate level, this result indicates that most LLM-generated refactorings remain behaviorally stable after reintegration into the original repository environment. The global preservation rate is higher than the SRR reported for RQ1, suggesting that LLM-generated refactorings often preserve test behavior even when they fail to remove the targeted smell. However, aggregate preservation conceals important variation across smell categories and models. For this reason, the following analyses progressively examine how behavioral outcomes differ across these dimensions.

4.2.2 Behavioral Preservation Across Smell Types

Figure 4.4 reports PPR values by smell category. Behavioral preservation varies markedly across smells, ranging from 95.3% to 68.1%.

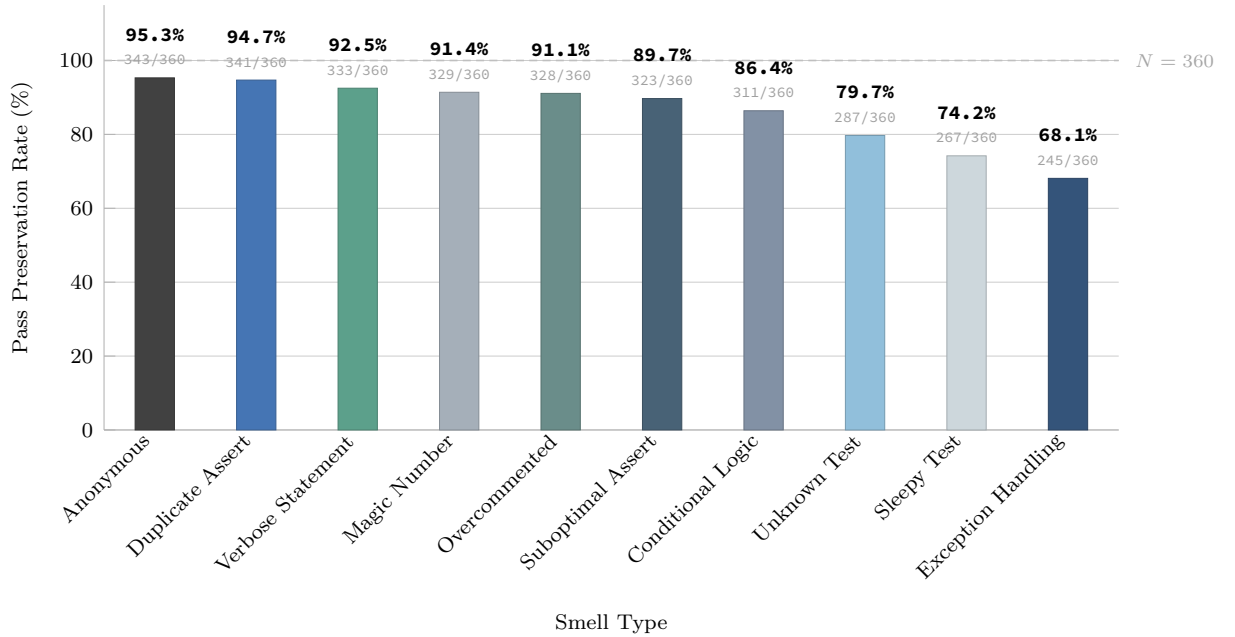


Figure 4.4: PPR by Smell Type

A consistent pattern emerges. Smells whose refactoring primarily affects naming, formatting, or localized assertion structure exhibit the highest preservation rates. *Anonymous Test* (95.3%), *Duplicate Assert* (94.7%), *Verbose Statement* (92.5%),

and *Magic Number* (91.4%) all maintain behavioral correctness in more than 90% of cases.

In contrast, smells that involve control flow, synchronization, or implicit behavioral assumptions are associated with higher regression risk. *Sleepy Test* (74.2%) and *Exception Handling* (68.1%) exhibit the lowest PPR values, reflecting the fragility of refactorings that alter timing behavior, exception propagation, or asynchronous execution patterns.

An important contrast with RQ1 also becomes visible here: some smells that were removed with high frequency show substantially lower behavioral preservation once execution is considered. This divergence reinforces that successfully removing the targeted smell does not necessarily imply semantic equivalence, especially when the refactoring affects execution-sensitive constructs.

4.2.3 Behavioral Preservation by Prompting Strategy

Figure 4.5 summarizes behavioral preservation across prompting strategies. *few-shot* prompting achieves the highest PPR (87.8%), followed by Chain-of-Thought (CoT) (86.8%) and *zero-shot* (84.4%).

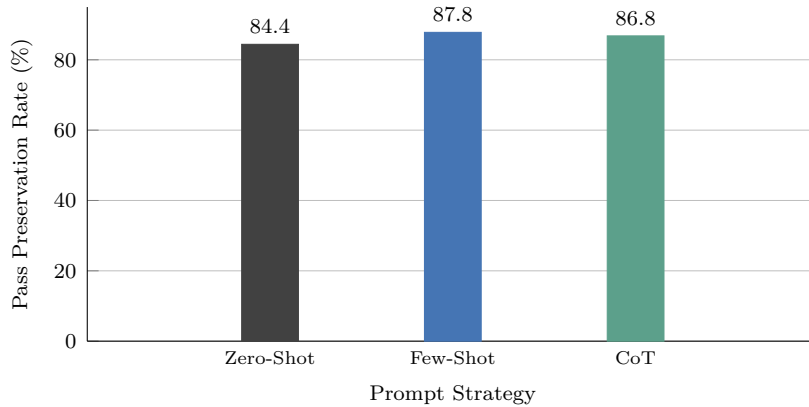


Figure 4.5: PPR by Prompt Strategy

The observed variation across strategies is limited to 3.4 percentage points, indicating that behavioral preservation is only weakly sensitive to prompt formulation. However, the relative ordering of strategies reveals a consistent pattern that merits closer examination.

Although CoT introduces additional structured guidance, its performance remains slightly below that of *few-shot*. This suggests that increasing prompt structure or reasoning depth does not necessarily translate into improved behavioral preservation. Instead, the results indicate that the presence of explicit refactoring patterns, as provided in *few-shot*, contributes more directly to maintaining semantic consistency.

This suggests that behavioral preservation depends on how strongly the generated refactoring is anchored to concrete output patterns. Even when CoT provides a more elaborate refactoring process, it leaves greater flexibility in how the final code is produced, which may introduce small variations that affect test outcomes. In contrast, *few-shot* appears to constrain the refactoring space more tightly, reducing variability in the generated outputs. This reduced variability may explain its consistently higher preservation rate, particularly in scenarios where multiple valid refactoring choices exist but only a subset preserves the original behavior.

The small gap between *few-shot* and CoT (1.0 percentage point) indicates that both strategies are effective in supporting behavior-preserving refactorings. However, their relative performance suggests that alignment to concrete refactoring patterns has a slightly stronger effect than additional reasoning structure.

Overall, these results reinforce the broader observation that prompting strategy plays a secondary role when compared to model capability. While different prompting configurations influence the stability of refactorings, their impact remains modest relative to the variability observed across models.

4.2.4 Model-Level Performance and Interaction Effects

Figure 4.6 presents the Model $\times$ Prompt PPR heatmap, enabling a joint analysis of semantic preservation across LLM architectures and prompting strategies. Two main patterns emerge from the matrix. First, semantic preservation is consistently high across most configurations. The majority of model-prompt combinations achieve PPR values above 85%, indicating that once refactorings are executed, behavioral regressions are relatively uncommon.

Second, differences appear between models. *Claude Sonnet 4.6* and *GPT-5.2* form the top performance tier, maintaining preservation rates close to or above 90% across all prompting strategies. This stability suggests that these models are particularly effective at performing structural test refactorings while preserving the intended test behavior.

DeepSeek-V3.2 and *Gemini 2.5 Pro* occupy an intermediate tier. Both models achieve high preservation rates overall, but their performance varies more across prompting configurations. *DeepSeek-V3.2* reaches its highest PPR under *few-shot* prompting (91.5%), while lower values are observed under *zero-shot* and CoT prompting. *Gemini 2.5 Pro* shows a similar pattern of moderate variation across prompts.

Qwen 2.5 Coder 32B and *CodeLlama 34B* present the lowest preservation rates among the evaluated models. *Qwen 2.5 Coder 32B* remains relatively stable across prompts, with PPR values between 85% and 86%. *CodeLlama 34B* exhibits the largest degradation, particularly under *zero-shot* prompting (73.5%), indicating a higher susceptibility to behavioral regressions during automated refactoring.

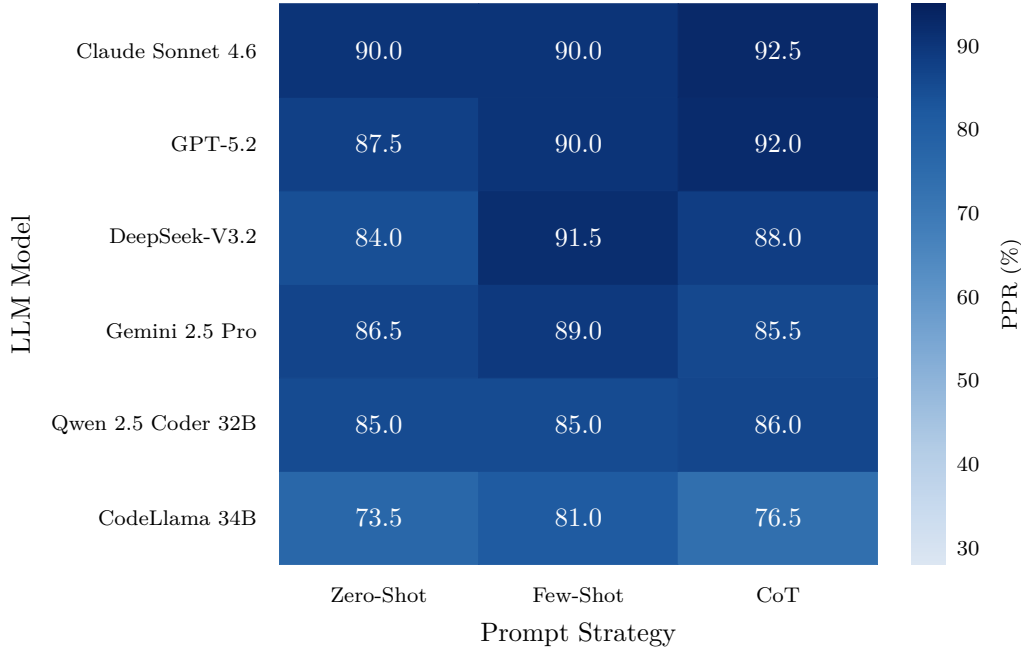


Figure 4.6: Heatmap — Model by Prompt PPR Matrix

Interestingly, the heatmap also suggests that prompting strategies exert a smaller influence on semantic preservation than model architecture. For most models, the variation between prompts remains within a narrow range of approximately 3–5 percentage points. In contrast, the difference between the best and worst performing models exceeds 15 percentage points, reinforcing the conclusion that model capability is the primary factor driving behavioral preservation performance in this experiment.

4.2.5 Failure Taxonomy and Regression Patterns

To better characterize the 493 non-preserved cases, Table 4.1 reports the global distribution of regression categories. The classification follows the hierarchical rule defined in Section 3.4.2. First, experiments are evaluated for increases in failing test cases (*intra-suite regression*); only when no such increase is observed are they evaluated for increases in failing suites (*suite-level regression*). Cases in which execution cannot proceed due to syntactic invalidity are classified separately as execution-level errors.

Under this classification scheme, intra-suite regressions constitute the largest category, with 309 instances (8.6% of all experiments), followed by suite-level regressions with 182 instances (5.1%). Execution-level errors are extremely rare, with only two cases (0.1%). Although execution-level errors are extremely rare, the two observed cases provide useful insight into the limits of automated refactoring. Both instances correspond to syntax errors produced by *CodeLlama 34B* when refactoring the *Un-*

Table 4.1: Failure Taxonomy

| Description | Failure Type | Count | (%) | Category |
|--------------------|-----------------|-------|------|------------|
| Preserved | None | 3107 | 86.3 | None |
| New test failures | Intra-suite | 309 | 8.6 | Regression |
| New Suite failures | Suite-level | 182 | 5.1 | Regression |
| Execution error | Execution-level | 2 | 0.1 | Error |

known Test smell, occurring under the *few-shot* and CoT prompting strategies. In these experiments, the refactored versions could not be executed because the generated code was syntactically invalid. As a result, the test runner was unable to execute the suite, leading to a forced execution failure classification.

While these cases represent only two out of 3,600 experiments, they highlight a distinct failure mode in LLM-based refactoring pipelines: the generation of non-executable code. Unlike behavioral regressions, which emerge during test execution, syntax errors prevent the validation process itself. Their extremely low frequency suggests that modern LLMs are generally capable of producing syntactically valid JavaScript, yet these isolated failures emphasize the importance of automatic code validation steps in empirical evaluation pipelines.

Intra-suite regressions correspond to scenarios in which the overall test suite continues to execute successfully after refactoring, but additional individual test failures emerge. Because suite-level regressions are only counted when no intra-suite increase is observed, these cases represent regressions where the number of failing suites increases without an accompanying rise in failing test cases. Consequently, suite-level regressions capture a distinct class of behavioral disruption that cannot be explained solely by localized test failures.

The following analyses report regression patterns at two complementary aggregation levels: smell category and model. Because these levels summarize different portions of the experimental design, the denominators in the *Count/Total* column differ accordingly. For smell-level analyses, each smell category comprises 20 validated instances evaluated across six LLMs and three prompting strategies, resulting in $20 \text{ instances} \times 6 \text{ LLMs} \times 3 \text{ prompting strategies} = 360$ refactoring attempts per smell. For model-level analyses, each LLM is evaluated across ten smell categories, 20 validated instances per category, and three prompting strategies, resulting in $10 \text{ smells} \times 20 \text{ instances} \times 3 \text{ prompting strategies} = 600$ refactoring attempts per model.

The distribution of intra-suite regressions across smell categories is reported in Table 4.2. Smells such as *Exception Handling* (93 cases) and *Sleepy Test* (90 cases) account for a large share of localized semantic degradations. Both smells involve control-flow or asynchronous constructs, suggesting that refactorings affecting temporal behavior or exception semantics are particularly prone to subtle behavioral shifts.

Table 4.2: Intra-suite Regressions by Smell Type

| Smell Type | Count/Total | (%) |
|----------------------|-------------|------|
| Exception Handling | 93/360 | 30.1 |
| Sleepy Test | 90/360 | 29.1 |
| SuboptimalAssert | 25/360 | 8.1 |
| OvercommentedTest | 25/360 | 8.1 |
| Unknown Test | 23/360 | 7.4 |
| ConditionalTestLogic | 16/360 | 5.2 |
| VerboseStatement | 13/360 | 4.2 |
| Magic Number | 10/360 | 3.2 |
| Duplicate Assert | 9/360 | 2.9 |
| AnonymousTest | 5/360 | 1.6 |

Model-level patterns further reinforce this interpretation. Table 4.3 shows that lower-performing models contribute more heavily to intra-suite regressions, with *CodeLlama 34B* (83 cases) and *Qwen 2.5 Coder 32B* (62 cases) accounting for a substantial portion of localized failures. In contrast, higher-capacity models such as *Claude Sonnet 4.6* and *GPT-5.2* exhibit fewer intra-suite degradations, indicating greater semantic stability under structural refactoring.

Table 4.3: Intra-suite Regressions by Model

| Model | Count/Total | (%) |
|--------------------|-------------|------|
| CodeLlama 34B | 83/600 | 26.9 |
| Qwen 2.5 Coder 32B | 62/600 | 20.1 |
| DeepSeek-V3.2 | 49/600 | 15.9 |
| Gemini 2.5 Pro | 44/600 | 14.2 |
| GPT-5.2 | 43/600 | 13.9 |
| Claude Sonnet 4.6 | 28/600 | 9.1 |

Suite-level regressions exhibit a different distribution pattern from intra-suite failures. Table 4.4 shows that *Unknown Test* accounts for the largest share of these regressions (48 cases, 26.4%), followed by *ConditionalTestLogic* (33 cases, 18.1%). Unlike intra-suite failures, which are strongly associated with asynchronous or exception-related behaviors, suite-level regressions appear more frequently in smells that involve structural ambiguity in test definitions or conditional execution paths.

This pattern suggests that certain refactorings may alter the execution structure of test suites rather than individual assertions. For example, modifications to conditional test logic or unclear test boundaries can affect how test runners interpret and organize test cases, potentially leading to a greater number of failing suites even when the number of failing test cases does not increase.

Model-level analysis further highlights this distinction. As shown in Table 4.5,

Table 4.4: Suite-level Regressions by Smell Type

| Smell Type | Count/Total | (%) |
|----------------------|-------------|------|
| Unknown Test | 48/360 | 26.4 |
| ConditionalTestLogic | 33/360 | 18.1 |
| Exception Handling | 22/360 | 12.1 |
| Magic Number | 21/360 | 11.5 |
| VerboseStatement | 14/360 | 7.7 |
| SubOptimalAssert | 12/360 | 6.6 |
| AnonymousTest | 12/360 | 6.6 |
| Duplicate Assert | 10/360 | 5.5 |
| OvercommentedTest | 7/360 | 3.8 |
| Sleepy Test | 3/360 | 1.6 |

CodeLlama 34B again contributes the largest number of regressions (53 cases), reinforcing the observation that lower-performing models are more prone to introducing structural disruptions in test suites. Interestingly, some higher-capacity models exhibit moderate suite-level regression counts despite relatively strong intra-suite stability. For instance, *Gemini 2.5 Pro* and *Claude Sonnet 4.6* present higher suite-level regression counts than expected from their intra-suite profiles, suggesting that certain refactoring transformations may alter suite organization even when individual test behaviors remain largely preserved.

Table 4.5: Suite-level Regressions by Model

| Model | Count/Total | (%) |
|--------------------|-------------|------|
| CodeLlama 34B | 53/600 | 29.1 |
| Gemini 2.5 Pro | 34/600 | 18.7 |
| Claude Sonnet 4.6 | 27/600 | 14.8 |
| Qwen 2.5 Coder 32B | 26/600 | 14.3 |
| DeepSeek-V3.2 | 24/600 | 13.2 |
| GPT-5.2 | 18/600 | 9.9 |

Taken together, these results indicate that behavioral regressions emerge through two distinct mechanisms. Intra-suite regressions primarily reflect localized semantic deviations affecting individual test assertions or execution paths. Suite-level regressions, in contrast, reflect broader structural changes in how tests are grouped, executed, or interpreted by the test runner. Distinguishing between these two categories therefore provides a more nuanced view of the risks associated with automated test refactoring.

Overall, the regression profile observed in RQ2 is dominated by localized behavioral deviations rather than catastrophic execution failures. Most violations manifest as increases in individual failing test cases within otherwise executable suites. The

hierarchical classification adopted in this study ensures that regression types remain analytically distinct and highlights that behavioral degradations are predominantly concentrated at the test-case level rather than at the suite execution level.

4.2.6 Answer to RQ2

In response to RQ2, the results show that LLM-generated refactorings preserve functional behavior in most cases, achieving a global PPR of 86.3%. This indicates that most LLM-generated refactorings preserve test behavior while addressing the targeted smell. Nevertheless, preservation is not uniform. Behavioral stability varies considerably across smell categories and model families. Refactorings involving localized structural adjustments tend to maintain correctness, whereas refactorings affecting control flow, asynchronous timing, or exception handling exhibit higher regression incidence. Model capacity emerges as the primary determinant of behavioral stability, while prompting strategy has only marginal impact.

Although prompting strategies show limited variation overall, their relative ordering provides additional insight. *few-shot* prompting achieves the highest preservation rate, followed closely by CoT, despite the latter providing more structured guidance. This suggests that increasing reasoning structure alone does not necessarily improve behavioral preservation. Instead, refactorings that are more strongly anchored to consistent output patterns tend to exhibit greater semantic stability. In this context, *few-shot* appears to reduce variability in the generated code, whereas CoT allows greater flexibility in how refactorings are realized, which may introduce subtle behavioral deviations.

The failure taxonomy analysis clarifies the nature of these regressions. Most non-preserved cases manifest as intra-suite regressions, where additional failing test cases appear within otherwise executable suites. Catastrophic execution failures are rare, and suite-level regressions occur less frequently than localized test-case increases. This distribution confirms that the dominant risk in LLM-based refactoring is not syntactic invalidity, but subtle semantic drift introduced during refactoring. Overall, LLM-based test refactoring is generally semantically reliable, but not behaviorally invariant. Successfully removing the targeted smell does not guarantee execution equivalence, reinforcing the necessity of dynamic validation in automated refactoring pipelines.

4.3 RQ3 – Coverage Impact of Refactoring

This section addresses the following research question:

RQ3: How does LLM-based refactoring affect test coverage?

RQ3 examines how refactoring influences the structural adequacy of the resulting test suite through coverage variation after refactoring. The analysis considers both aggregate coverage behavior and its distribution across experimental dimensions, in order to observe whether refactorings tend to preserve, improve, or reduce code coverage. This perspective extends the previous findings by examining whether refactorings that achieve smell removal and preserve test behavior also maintain the structural reach of the test suite.

Following the analytical criterion defined in the methodology, coverage results are interpreted as indicators of structural variation in exercised program elements after refactoring. Since small percentage-point oscillations in aggregated coverage proportions may reflect instrumentation noise or marginal execution variation rather than meaningful structural change, a tolerance threshold of ± 0.5 percentage points is adopted: each experiment is classified as *improved* if at least one of the four coverage metrics (statements, branches, functions, or lines) increases by 0.5pp or more; as *degraded* if at least one metric decreases by 0.5pp or more and no metric increases by 0.5pp or more; and as *preserved* otherwise. This criterion functions as a conservative analytical filter, ensuring that only variations exceeding micro-oscillation effects are interpreted as meaningful structural changes.

4.3.1 Overall Coverage Effects

Across the 3,600 experiments with valid coverage data, mean coverage deltas are positive for all measured dimensions:

- $\overline{\Delta}_{\text{statements}} = +0.840$ pp,
- $\overline{\Delta}_{\text{branches}} = +0.754$ pp,
- $\overline{\Delta}_{\text{functions}} = +0.878$ pp,
- $\overline{\Delta}_{\text{lines}} = +0.837$ pp.

These averages indicate a slight global tendency toward coverage improvement after refactoring. However, aggregate means alone do not fully capture structural impact, as positive averages can mask localized degradation concentrated under specific experimental conditions.

To complement this aggregate view, Figure 4.7 presents the overall distribution of experiments according to the adopted coverage classification criterion. Coverage remains structurally preserved in 97.0% of experiments, improves in 1.3%, and degrades meaningfully in 1.7%. Although the vast majority of transformations maintain structural adequacy, a small subset of refactorings results in a measurable reduction of code structure.

Taken together, these results indicate that coverage behavior is overwhelmingly stable under the adopted threshold. Structural preservation is dominant, and only a small fraction of refactorings alters the structure of the code sufficiently to produce a measurable decrease.

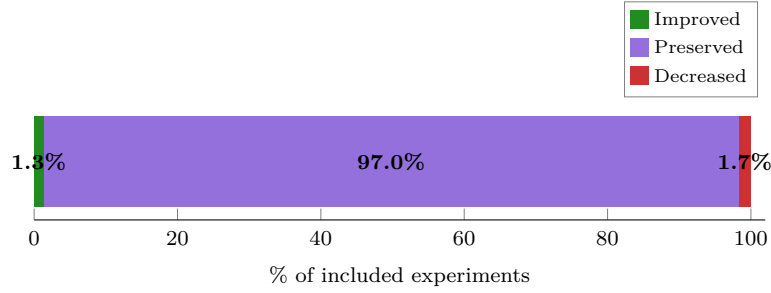


Figure 4.7: Overall Coverage Classification After Refactoring.

4.3.2 Model-Level Coverage Behavior

Figure 4.8 reports the mean change in statement coverage aggregated by model. For each experiment, coverage variation is computed as the difference between post-refactoring and baseline coverage:

$$\Delta coverage = coverage_{post} - coverage_{baseline}$$

where values are expressed in percentage points (pp). The values presented in Figure 4.8 correspond to the mean of these per-instance deltas grouped by model.

In this analysis, coverage variation is used to capture changes in the test suite’s structural reach after refactoring. Coverage measures the extent to which program statements are exercised by the test suite, whereas behavioral preservation is independently assessed in RQ2 through execution outcomes. Changes in coverage, therefore, reflect modifications in the set of executed code paths resulting from alterations in the test implementation after refactoring. Although such structural variation may indicate differences in control flow or execution structure, it does not necessarily imply functional regressions or semantic alterations. In particular, many refactorings modify how tests are written without affecting their observable outcomes. At the same time, substantial coverage variation may signal transformations that alter execution paths in ways that increase the likelihood of behavioral deviations. For this reason, coverage should be understood as a complementary structural indicator that may correlate with behavioral risk, but does not determine it.

All evaluated models present positive mean variation, indicating that, on average, refactoring does not reduce statement coverage in this dataset and instead leads to small net increases. For instance, the value +0.909 for *Qwen 2.5 Coder 32B* indicates that statement coverage increased by 0.909 percentage points on average relative to the baseline across its generated refactorings.

This result should be interpreted cautiously. A positive mean does not imply that coverage is consistently improved across all instances, but rather that increases and preserved cases outweigh reductions in the aggregated distribution. The $\pm 0.5pp$ classification threshold defined in Section 3.4.2 is applied at the level of individual

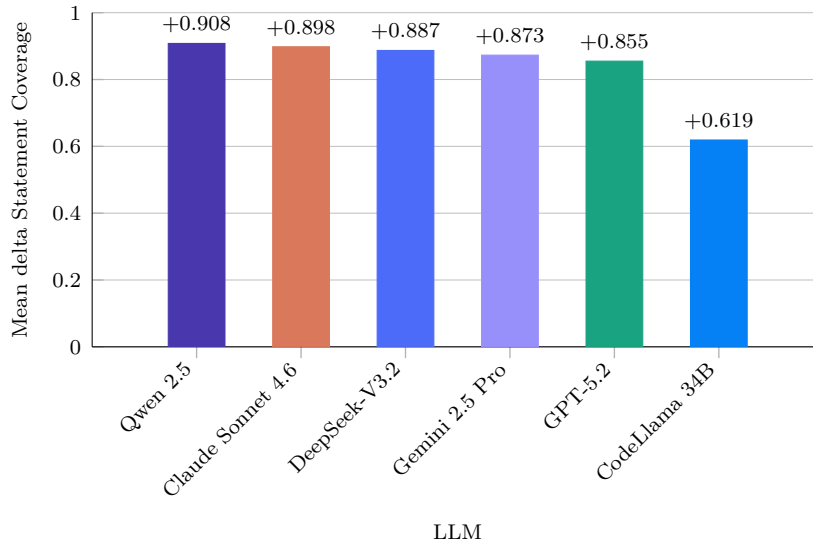


Figure 4.8: Mean Statement Coverage Change by Model.

experiments (as in Figure 4.7 and Tables 4.6–4.7); it is not meant to reclassify an already-aggregated mean, so the model-level averages reported here are presented purely as descriptive magnitudes.

The strongest average gains are observed for *Qwen 2.5 Coder 32B* (+0.909 pp) and *Claude Sonnet 4.6* (+0.898 pp), followed by *DeepSeek-V3.2* (+0.887 pp), *Gemini 2.5 Pro* (+0.873 pp), and *GPT-5.2* (+0.855 pp). *CodeLlama 34B* exhibits the smallest mean increase (+0.619 pp), suggesting a lower tendency toward coverage gains and a higher concentration of near-preservation outcomes.

These differences should not be attributed solely to model size. For example, *Qwen 2.5 Coder 32B*, which is not the largest model evaluated, achieves the highest mean gain. This suggests that coverage impact is influenced not only by model scale, but also by factors such as code-oriented training, instruction tuning, and the nature of the transformations produced during refactoring.

4.3.3 Coverage Risk by Smell Type

Tables 4.6 and 4.7 provide a detailed breakdown of coverage effects by targeted smell type and model, while Figure 4.9 extends the analysis by showing the interaction between model and prompting strategy in terms of mean statement coverage variation.

Two distinct structural profiles emerge. First, some smells consistently produce measurable coverage gains without any meaningful degradation. Refactoring *Duplicate Assert* yields strong positive deltas across all metrics (approximately +4.5 percentage points) and no cases of meaningful degradation (0.0%). These transformations frequently simplify redundant assertions while preserving the executable structure

Table 4.6: Coverage Delta by Smell Type

| Smell Type | n | $\overline{\Delta}_{\text{stmt}}$ | $\overline{\Delta}_{\text{branch}}$ | $\overline{\Delta}_{\text{func}}$ | $\overline{\Delta}_{\text{lines}}$ | Degraded % |
|----------------------|-----|-----------------------------------|-------------------------------------|-----------------------------------|------------------------------------|------------|
| Duplicate Assert | 360 | +4.581 | +4.150 | +4.781 | +4.576 | 0.0% |
| Unknown Test | 360 | +4.200 | +3.725 | +4.391 | +4.193 | 1.7% |
| OvercommentedTest | 360 | -0.001 | +0.023 | -0.022 | -0.002 | 3.1% |
| AnonymousTest | 360 | -0.005 | -0.003 | -0.020 | -0.004 | 2.2% |
| VerboseStatement | 360 | -0.010 | -0.008 | -0.012 | -0.011 | 0.8% |
| ConditionalTestLogic | 360 | -0.010 | +0.003 | -0.005 | -0.012 | 1.4% |
| Sleepy Test | 360 | -0.012 | -0.010 | 0.000 | -0.017 | 0.6% |
| Exception Handling | 360 | -0.066 | -0.043 | -0.067 | -0.069 | 4.7% |
| Magic Number | 360 | -0.103 | -0.108 | -0.096 | -0.104 | 0.8% |
| SuboptimalAssert | 360 | -0.174 | -0.191 | -0.167 | -0.179 | 1.9% |

Table 4.7: Coverage Delta by Model

| Model | n | $\overline{\Delta}_{\text{stmt}}$ | Degraded Exps | Degraded % |
|--------------------|-----|-----------------------------------|---------------|------------|
| Qwen 2.5 Coder 32B | 600 | +0.909 | 6 | 1.0% |
| Claude Sonnet 4.6 | 600 | +0.898 | 9 | 1.5% |
| DeepSeek-V3.2 | 600 | +0.887 | 7 | 1.2% |
| Gemini 2.5 Pro | 600 | +0.873 | 10 | 1.7% |
| GPT-5.2 | 600 | +0.855 | 14 | 2.3% |
| CodeLlama 34B | 600 | +0.619 | 16 | 2.7% |

of the test, often making assertion logic more direct without reducing exercised statements.

Second, certain smells exhibit comparatively greater coverage sensitivity, even though absolute degradation rates remain low under the adopted threshold. *Exception Handling* presents the highest degradation rate (4.7%), followed by *Overcommented Test* (3.1%) and *Anonymous Test* (2.2%). These smells typically require modifications that affect exception propagation, comment or statement organization, or test structure, which can influence the set of executed paths after refactoring.

Figure 4.9 reveals that coverage behavior is also shaped by the interaction between model and prompting strategy. Most configurations remain concentrated in a narrow positive range, indicating that statement coverage is generally preserved or slightly improved regardless of the prompt used. However, the heatmap also shows that prompting effects are not uniform across models.

For most models, *few-shot* and CoT tend to produce slightly higher mean gains than *zero-shot*. For example, *Claude Sonnet 4.6* increases from 0.886 under *zero-shot* to 0.908 under CoT, while *DeepSeek-V3.2* reaches 0.908 under both *few-shot* and CoT. A similar pattern appears for *Qwen 2.5 Coder 32B*, which achieves the highest values in the matrix, peaking at 0.912 under *few-shot*. These results suggest

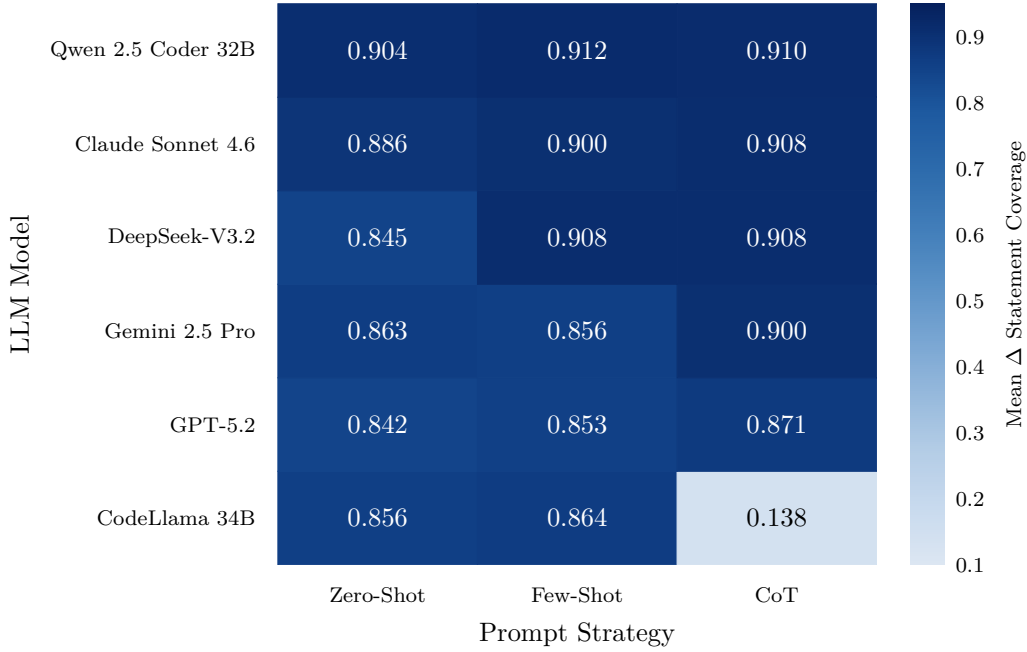


Figure 4.9: Heatmap – Model by Mean Statement Coverage Matrix

that additional prompt structure may help some models produce refactorings that slightly expand or preserve executable statement coverage.

At the same time, prompt benefits are clearly model-dependent. *Gemini 2.5 Pro* shows its highest value under CoT(0.900), whereas *GPT-5.2* improves more moderately across strategies, from 0.842 under *zero-shot* to 0.871 under CoT. This indicates that prompt elaboration contributes incremental gains rather than systematic improvements.

A notable exception appears in *CodeLlama 34B*. While *zero-shot* (0.856) and *few-shot* (0.864) remain aligned with the general pattern, CoT drops sharply to 0.138. This isolated reduction suggests that, for weaker models, longer reasoning-oriented prompts may introduce unstable transformations that substantially alter executable structure instead of preserving it. Consistent with this pattern, *CodeLlama 34B* also records the highest degradation rate among all models under the adopted threshold (2.7%), followed by *GPT-5.2* (2.3%) and *Gemini 2.5 Pro* (1.7%), while *Qwen 2.5 Coder 32B* records the lowest (1.0%).

Refactorings involving exception propagation and comment or assertion reorganization naturally affect the set of statements and branches exercised at runtime. Because coverage models are grounded in control-flow traversal (Meinke, 2021), such transformations may reduce path diversity even when test execution remains behaviorally correct. This result explains why *Exception Handling* remains the most coverage-sensitive smell even when the global averages remain positive and the overall degradation rate stays low.

Interestingly, *Magic Number* exhibits low degradation (0.8%) despite small negative mean deltas, suggesting that value abstraction tends to preserve execution structure and usually affects only literal expressions rather than executable paths.

4.3.4 Interpretation in Light of Coverage Theory

Coverage quantifies exercised structural elements rather than semantic correctness. Under this interpretation:

- Coverage preservation suggests structural equivalence of execution;
- Coverage improvement may indicate exposure of previously unexecuted structural units;
- Coverage reduction reflects reduced structural exploration even when tests still pass.

The global results indicate that meaningful reduction remains limited to **1.7%** of experiments, while the vast majority of refactorings preserve coverage exactly or produce slight positive shifts. This suggests that LLM-based refactoring is predominantly coverage-stable under the adopted threshold.

The model–prompt interaction analysis further reinforces this interpretation. Across most configurations, *zero-shot*, *few-shot*, and CoT produce only small differences in mean statement coverage, indicating that prompt formulation does not systematically alter structural adequacy. When differences appear, they are model-dependent rather than universal, with stronger models typically maintaining stable gains across strategies and weaker models showing greater sensitivity.

Because coverage is widely used as a practical adequacy proxy in software testing (Meinke, 2021), even the localized reductions that remain after filtering out micro-oscillations are structurally relevant. This is because a single refactoring may preserve the observable behavior of the tests while reducing the set of program elements exercised during execution. This distinction motivates the use of coverage variation as a complementary dimension of refactoring quality.

4.3.5 Answer to RQ3

In response to RQ3, the results indicate that LLM-based refactoring is predominantly structurally stable: the vast majority of transformations either preserve coverage or produce small positive gains, while only a small minority introduces meaningful degradation. Measurable degradation remains present in **1.7%** of experiments, concentrated mainly in smells whose refactoring affects exception propagation or broader test reorganization, such as *Exception Handling*. The results also show that coverage behavior varies across models and, to a lesser extent, across prompting strategies, although prompt effects remain secondary and highly dependent on the model used.

Taken together, these findings demonstrate that preserving execution behavior alone does not guarantee preservation of structural test strength. Coverage monitoring remains an important complementary criterion in automated refactoring evaluation, especially when refactorings affect exception handling, statement organization, or executable paths.

4.4 RQ4 – Unintended Structural Side Effects

This section addresses the following research question:

RQ4: What unintended structural side effects emerge from LLM-based refactoring?

Whereas RQ1 evaluated smell removal, RQ2 examined behavioral preservation, and RQ3 analyzed coverage variation, RQ4 investigates whether LLM-generated refactorings introduce additional test smells in the resulting test code. This analysis focuses on side effects that may appear even when the original smell is removed, showing that refactoring can improve one aspect of the test while creating new maintenance problems. Specifically, we examine whether new smell categories emerge after refactoring and analyze how these occurrences vary across smell types and models. This perspective helps determine whether LLM-generated refactorings preserve the broader quality of the test suite rather than merely addressing the targeted smell.

4.4.1 Global New-Smell Introduction

The analysis of newly introduced smells considers only the smell categories selected for this study, as described in Section 3.2.3. One experiment produced a syntax error after refactoring, preventing the smell detection tools from analyzing the resulting test code. Consequently, this case was excluded from the structural side-effect analysis, resulting in a total of **3,599 analyzed experiments**.

Across these experiments, the target smell was successfully removed in **2,760 cases (76.7%)**, consistent with the removal performance reported in RQ1. However, **484 experiments (13.4%)** introduced at least one new smell that was not present prior to refactoring. In total, **1,532 new smell instances** were introduced across the dataset, corresponding to an average of **0.426 added smells per refactoring attempt**. These results indicate that although LLM-based refactoring frequently succeeds in removing the targeted smell, a meaningful proportion of refactoring attempts introduce new test smells elsewhere in the test code.

4.4.2 Removal vs. New-Smell Introduction by Smell Type

Table 4.8 and Figure 4.10 jointly report removal effectiveness and new-smell introduction rates across smell categories.

Table 4.8: Smell Removal and New-Smell Introduction by Smell Type

| Smell Type | Total | Removal | | New Introduction | | Net (R-I) |
|----------------------|-------|---------|------|------------------|------|-----------|
| | | Count | (%) | Count | (%) | |
| Unknown Test | 360 | 332 | 92.2 | 110 | 30.6 | 222 |
| Sleepy Test | 359 | 292 | 81.3 | 103 | 28.7 | 189 |
| Magic Number | 360 | 328 | 91.1 | 69 | 19.2 | 259 |
| ConditionalTestLogic | 360 | 257 | 71.4 | 65 | 18.1 | 192 |
| VerboseStatement | 360 | 121 | 33.6 | 57 | 15.8 | 64 |
| OvercommentedTest | 360 | 310 | 86.1 | 29 | 8.1 | 281 |
| Duplicate Assert | 360 | 243 | 67.5 | 28 | 7.8 | 215 |
| SuboptimalAssert | 360 | 273 | 75.8 | 14 | 3.9 | 259 |
| AnonymousTest | 360 | 273 | 75.8 | 7 | 1.9 | 266 |
| Exception Handling | 360 | 331 | 91.9 | 2 | 0.6 | 329 |

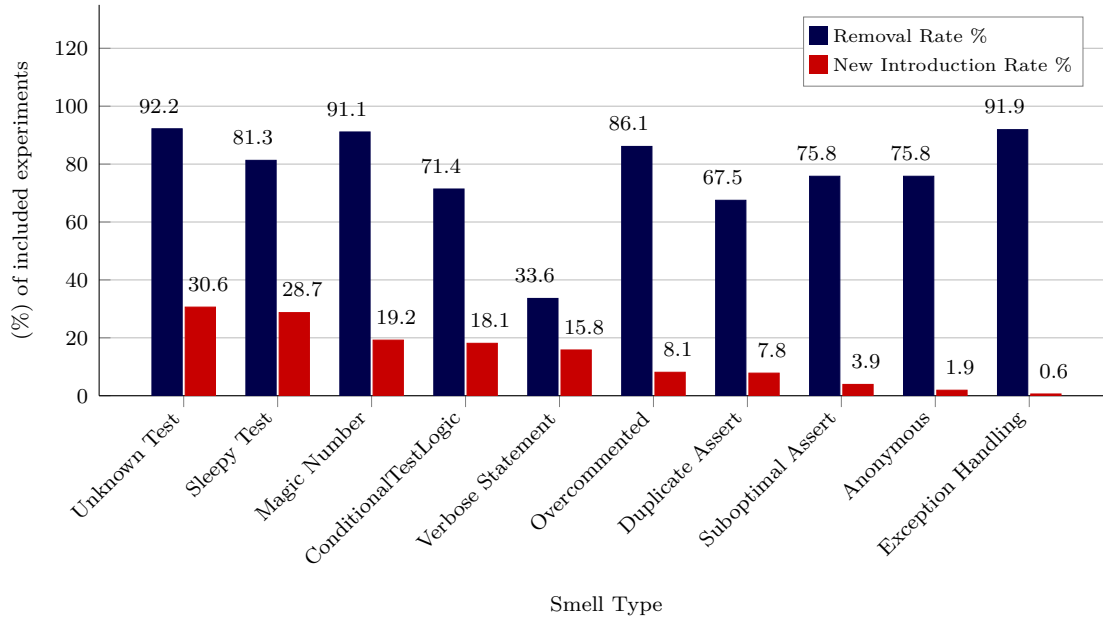


Figure 4.10: Smell Removal and New Smell Introduction Rates by Smell Type

Substantial differences emerge across smell categories. Several smells exhibit both high removal effectiveness and low side effects. For instance, *Exception Handling* achieves a removal rate of **91.9%** while introducing new smells in only **0.6%** of experiments, indicating that these transformations are typically localized and less prone to introducing new test smells. Similarly, *Anonymous Test* and *Suboptimal Assert* exhibit relatively low side-effect rates despite moderate removal effectiveness.

In contrast, other smells reveal a clear trade-off between removal success and new-smell introduction. *Unknown Test* achieves one of the highest removal rates (**92.2%**) yet introduces new smells in **30.6%** of experiments. A similar pattern appears for *Sleepy Test*, which combines strong removal effectiveness (**81.3%**) with a high side-effect rate (**28.7%**). These smells typically require more intrusive transformations, such as restructuring test logic or decomposing complex test bodies, which may increase the likelihood of introducing additional test smells.

At the opposite extreme, *Verbose Statement* represents the most problematic smell in the dataset, combining low removal effectiveness with a non-negligible rate of new-smell introduction. It exhibits the lowest removal rate (**33.6%**) while still introducing new smells in **15.8%** of experiments. This suggests that simplifying excessively verbose tests often requires substantial rewriting that may fail to eliminate the original smell while simultaneously creating new ones. Overall, these results indicate that structural side effects are strongly smell-dependent. Refactorings involving substantial restructuring of test logic or synchronization mechanisms tend to be more prone to introducing additional test smells than localized code transformations.

4.4.3 Taxonomy of Newly Introduced Smells

Table 4.9 presents the distribution of smell categories introduced after refactoring. It reports two complementary percentage-based measures. The first, *% of Added*, indicates the share of each smell type among all newly introduced smell instances, capturing how the overall volume of newly introduced smells is distributed across categories. The second, *% of Experiments*, indicates the proportion of refactoring attempts in which each smell type appeared at least once, capturing how prevalent each smell category is across the experimental set.

Table 4.9: Taxonomy of Newly Introduced Smells

| Introduced Smell | Count | % of Added | % of Experiments |
|-------------------------------|-------|------------|------------------|
| <i>Duplicate Assert</i> | 605 | 39.5 | 16.8 |
| <i>Conditional Test Logic</i> | 452 | 29.5 | 12.6 |
| <i>Magic Number</i> | 164 | 10.7 | 4.6 |
| <i>Verbose Statement</i> | 90 | 5.9 | 2.5 |
| <i>Exception Handling</i> | 68 | 4.4 | 1.9 |
| <i>Unknown Test</i> | 53 | 3.5 | 1.5 |
| <i>Suboptimal Assert</i> | 42 | 2.7 | 1.2 |
| <i>Overcommented Test</i> | 30 | 2.0 | 0.8 |
| <i>Sleepy Test</i> | 26 | 1.7 | 0.7 |
| <i>Anonymous Test</i> | 2 | 0.1 | 0.1 |

The distribution of newly introduced smells is highly concentrated. *Duplicate Assert* alone accounts for 39.5% of all newly introduced smell instances, followed by *Condi-*

tional Test Logic (29.5%). Together, these two categories represent nearly 70% of all newly introduced smells observed in the dataset, indicating that most added smells are concentrated in a small subset of categories. Both smells also appear in a non-negligible portion of experiments (16.8% and 12.6%, respectively), suggesting that these cases are not isolated events but recur across different refactoring attempts. At the same time, their higher contribution to the total number of added instances indicates that they often appear multiple times within individual refactored tests, particularly in the case of *Duplicate Assert*.

Beyond these dominant categories, the remaining smells exhibit substantially lower frequencies. *Magic Number* accounts for 10.7% of added instances, while all other categories individually remain below 6%. This long-tail distribution indicates that, although different types of smells can be introduced after refactoring, their occurrence is comparatively limited and does not significantly alter the overall concentration pattern.

Taken together, the results show that newly introduced smells are unevenly distributed across smell categories, with a clear dominance of assertion- and logic-related smells. This suggests that the impact of refactoring is not uniform, but instead concentrated in specific types of test-code refactorings observed in the dataset.

4.4.4 Smell Interaction Matrix

Figure 4.11 presents the smell interaction matrix, illustrating how refactoring a given smell may introduce other smell categories.

The matrix reveals **55 non-zero interaction pairs** among the ten targeted smells. The strongest interaction occurs when refactoring *Conditional Test Logic*, which introduces *Duplicate Assert* in **487 instances**. This pattern suggests that when conditional logic is simplified or decomposed, LLMs often duplicate assertion statements across newly generated test paths.

Another notable interaction occurs when refactoring *Sleepy Test*, which frequently introduces *Conditional Test Logic*. This indicates that replacing time-based synchronization mechanisms may increase branching logic within the test body. Overall, the interaction matrix demonstrates that newly introduced smells are associated with systematic refactoring patterns rather than random noise. Certain refactoring operations consistently lead to specific types of new test smells.

4.4.5 Smell Introduction by Model and Prompt Strategy

Figure 4.12 reports the percentage of experiments per model that introduced at least one smell, while Figure 4.13 extends the analysis by showing how this rate varies across prompting strategies.

Model-level differences remain substantial. *CodeLlama 34B* exhibits the lowest side-effect rate (**6.3%**), followed by *Claude Sonnet 4.6* (**9.5%**). Intermediate models such

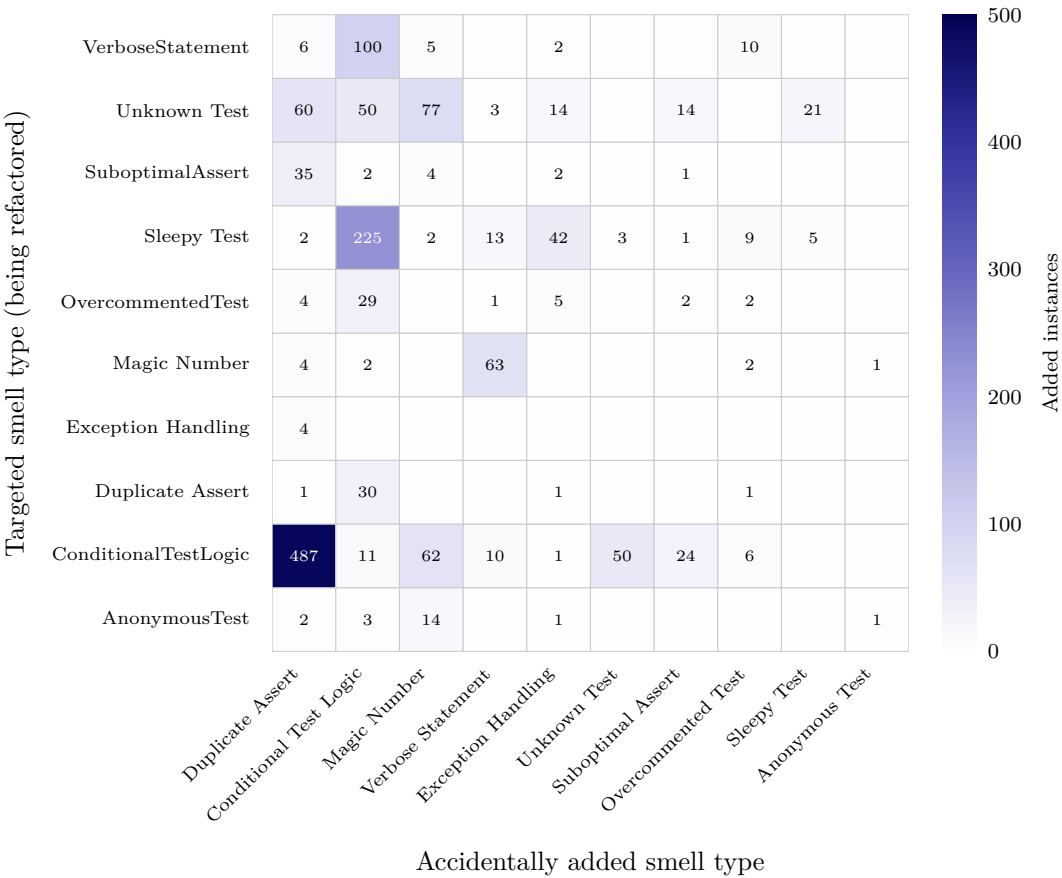


Figure 4.11: Smell Interaction Matrix Showing How Refactoring One Smell Introduces Others

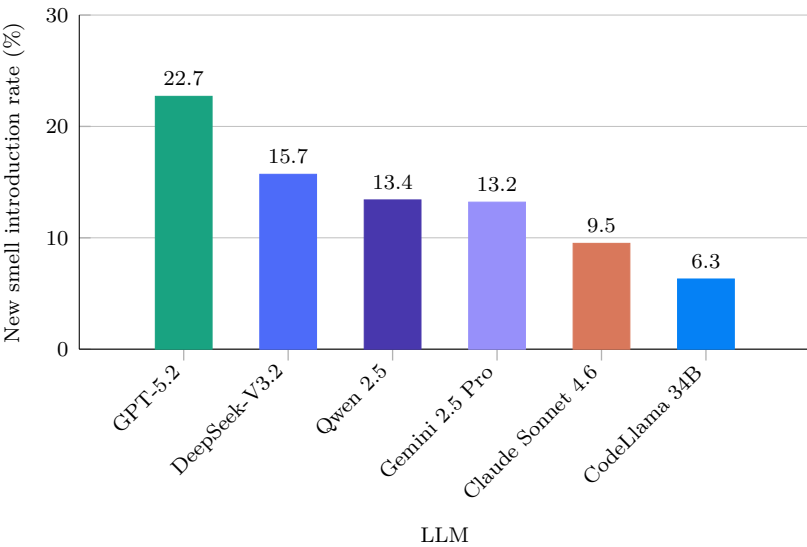


Figure 4.12: New Smell Introduction Rate by Model

as *Gemini 2.5 Pro* and *Qwen 2.5 Coder 32B* introduce new smells in approximately **13%** of experiments, while *DeepSeek-V3.2* reaches **15.7%**. The highest side-effect rate is observed for *GPT-5.2*, which introduces new smells in **22.7%** of experiments.

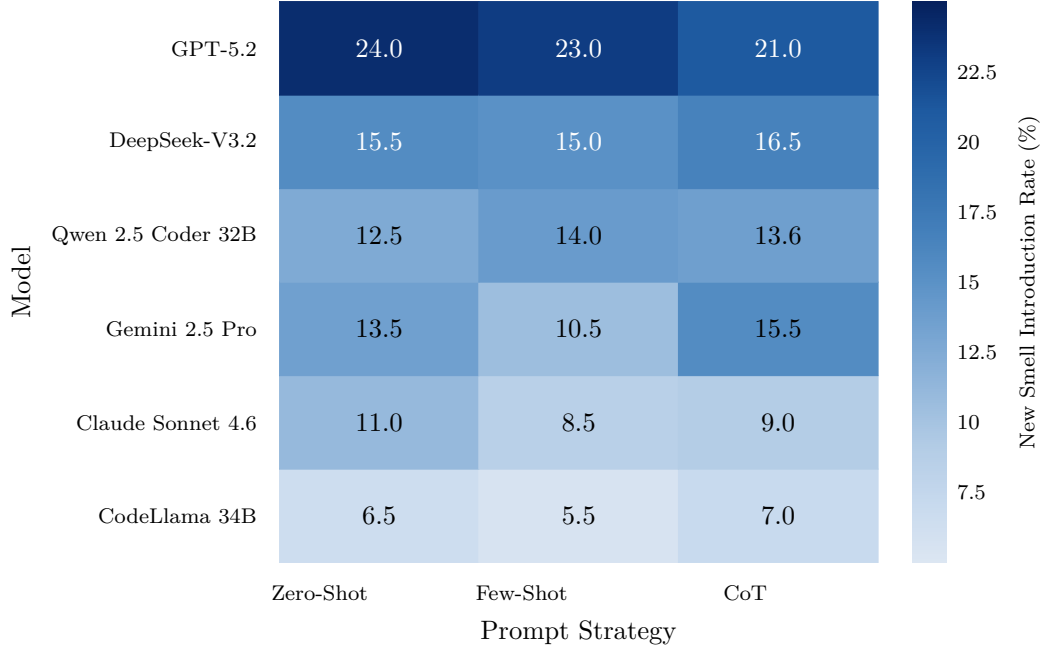


Figure 4.13: Heatmap – New Smell Introduction Rate by Model and Prompt Strategy

The heatmap reveals that prompting strategy produces only moderate variation, but its effect is not uniform across models. For most models, the absolute differences between *zero-shot*, *few-shot*, and CoT remain limited, suggesting that the introduction of new smells depends primarily on model behavior rather than prompt formulation alone.

Some localized patterns nevertheless emerge. For *Claude Sonnet 4.6*, *few-shot* yields the lowest side-effect rate (**8.5%**), slightly below both *zero-shot* and CoT. A similar reduction appears for *Gemini 2.5 Pro*, where *few-shot* decreases new smell introduction to **10.5%**, compared with **13.5%** under *zero-shot* and **15.5%** under CoT. This suggests that example-guided prompting may help some stronger models perform more controlled structural rewrites.

By contrast, *GPT-5.2* remains the most side-effect-prone model under all strategies, although its rate decreases slightly from **24.0%** under *zero-shot* to **21.0%** under CoT. This indicates that even highly capable models may generate broader structural transformations that increase the probability of collateral smell introduction.

Open-weight models also show heterogeneous prompt sensitivity. *DeepSeek-V3.2* remains relatively stable across strategies, varying only between **15.0%** and **16.5%**,

whereas *Qwen 2.5 Coder 32B* shows a small increase under *few-shot*. *CodeLlama 34B* remains consistently low across all prompts, which is compatible with its more conservative transformation profile observed in previous analyses.

Taken together, these results suggest that models capable of performing more aggressive structural transformations may also lead to the introduction of additional test smells. However, the heatmap shows that prompting can slightly modulate this tendency, particularly for models that already operate near the upper performance range.

4.4.6 Answer to RQ4

In response to RQ4, the results show that LLM-based refactorings introduce new test smells in a meaningful portion of the analyzed cases. Although the target smell is removed in **76.7%** of cases, approximately **13.4%** of refactorings introduce at least one new smell, producing a total of **1,532 added smell instances**.

These newly introduced smells are highly concentrated in a small number of categories, particularly *Duplicate Assert* and *Conditional Test Logic*. Interaction analysis further reveals systematic refactoring patterns, indicating that the introduction of new test smells follows recurring model behaviors rather than random errors.

Model-level analysis demonstrates that strong smell removal performance does not necessarily imply lower new-smell introduction. Some high-performing models remove smells more effectively but also introduce new smells more frequently, while the prompting strategy exerts only a secondary influence, producing small but model-dependent variations in new-smell introduction rates. Overall, the results reinforce that smell removal and the introduction of new test smells must be evaluated jointly when assessing automated refactoring systems.

4.5 Influence of Experimental Factors on Refactoring Outcomes

This section addresses the following research question:

RQ5: How do experimental factors influence LLM-based refactoring outcomes?

RQ5 adopts an explanatory perspective by examining how the main experimental factors shape the outcome patterns established in the previous research questions. Rather than introducing a new primary quality dimension, the analysis reuses the results on smell removal, behavioral preservation, and coverage variation to explain why some configurations consistently outperform others and where refactoring quality becomes more sensitive to model choice or prompt formulation.

The analysis focuses primarily on two experimental factors: the selected LLM and the prompting strategy used to generate the transformation. To support a unified interpretation across dimensions, an integrated success indicator is used as a complementary synthesis, allowing observation of how often the main quality requirements are simultaneously satisfied within the same refactoring attempt.

4.5.1 Global Integrated Success

Following the integrated success criterion defined in Section 3.4.2, a refactoring attempt is considered successful only when it removes the targeted smell while preserving behavior and coverage stability. Across all 3,600 experiments, **2,337 refactorings** satisfy this criterion, yielding an overall **RSR of 64.9%**. The corresponding 95% Wilson confidence interval is **[63.3%, 66.5%]**, indicating that approximately two out of every three attempts meet the combined quality requirements.

As expected, this rate is lower than the individual metrics reported in previous sections because it requires all quality dimensions to hold simultaneously. Thus, the result captures the cumulative difficulty of achieving complete refactoring success, rather than success in any single dimension alone. Rather than replacing the individual analyses, the integrated rate provides a reference point for interpreting complete refactoring quality. The following subsections examine how this outcome varies across prompting strategies, models, and their interaction.

4.5.2 Influence of Prompting Strategy

Table 4.10 summarize integrated success across prompting strategies. The three strategies produce closely aligned outcomes: *zero-shot* achieves **64.3%**, *few-shot* **64.8%**, and CoT **65.8%**. The maximum observed difference is only **1.5 percentage points**, and the confidence intervals overlap substantially.

Table 4.10: Smell Removal Results by Prompt Strategy

| Prompt | Attempts | Successes | RSR % | 95 % CI |
|------------------|----------|-----------|-------|----------------|
| Zero-Shot | 1200 | 771 | 64.2% | [61.5%, 66.9%] |
| Few-Shot | 1200 | 777 | 64.8% | [62.0%, 67.4%] |
| Chain-of-Thought | 1200 | 789 | 65.8% | [63.0%, 68.4%] |

This narrow range indicates that the prompting strategy exerts only limited influence on complete refactoring success when all quality constraints are considered jointly. Although CoT reaches the highest aggregate rate, the practical advantage over simpler prompting forms remains modest.

This pattern reinforces what was already observed across earlier sections: prompt design can slightly modulate outcomes, but does not substantially alter the overall quality profile of LLM-based refactoring.

4.5.3 Influence of Model Choice

Table 4.11 report integrated success by model.

Table 4.11: Smell Removal Results by Model

| Model | Attempts | Successes | RSR % | 95 % CI |
|--------------------|----------|-----------|-------|----------------|
| GPT-5.2 | 600 | 475 | 79.2% | [75.7%, 82.2%] |
| Gemini 2.5 Pro | 600 | 475 | 79.2% | [75.7%, 82.2%] |
| Claude Sonnet 4.6 | 600 | 470 | 78.3% | [74.9%, 81.4%] |
| DeepSeek-V3.2 | 600 | 399 | 66.5% | [62.6%, 70.2%] |
| Qwen 2.5 Coder 32B | 600 | 381 | 63.5% | [59.6%, 67.3%] |
| CodeLlama 34B | 600 | 137 | 22.8% | [19.7%, 26.4%] |

Model-level differences are pronounced. *GPT-5.2* and *Gemini 2.5 Pro* achieve the highest overall success rate (**79.2%** each), followed closely by *Claude Sonnet 4.6* (**78.3%**). A second performance tier is formed by *DeepSeek-V3.2* (**66.5%**) and *Qwen 2.5 Coder 32B* (**63.5%**), both clearly below the leading group. At the lower extreme, *CodeLlama 34B* records only **22.8%**, standing as a clear outlier.

The gap between the best and worst models is **56.3 percentage points**, which is far larger than the variation observed across prompting strategies. This strongly indicates that model choice is the dominant determinant of refactoring outcomes. A broader pattern also emerges when considering model families. The three proprietary models (*GPT-5.2*, *Gemini 2.5 Pro*, and *Claude Sonnet 4.6*) all cluster around **78%–79%** RSR, whereas the open-weight models show markedly lower and more heterogeneous performance. Although family-level grouping is secondary to the model-specific analysis, the contrast suggests that architectural maturity, training quality, and alignment for code transformation tasks play a substantial role in determining overall effectiveness.

4.5.4 Model x Prompt Interaction Effects

Figure 4.14 presents the model $\times$ prompt interaction matrix.

The interaction matrix reveals two main patterns. First, high-performing models remain strong across all prompting strategies. For example, *Claude Sonnet 4.6* varies only from **77.0%** to **79.5%**, and *Gemini 2.5 Pro* varies from **78.0%** to **81.0%**. This stability suggests that once a model has sufficient capability, the exact prompt formulation has only limited influence on its integrated success profile.

Second, some models show clearer prompt sensitivity. *GPT-5.2* improves from **74.5%** under *zero-shot* to **84.0%** under CoT, representing the largest gain among all models. This indicates that stronger models may still benefit from structured prompting when performing more complex, multi-constraint refactorings. By contrast, *DeepSeek-V3.2* performs best under *few-shot* (**69.0%**) and slightly worse un-

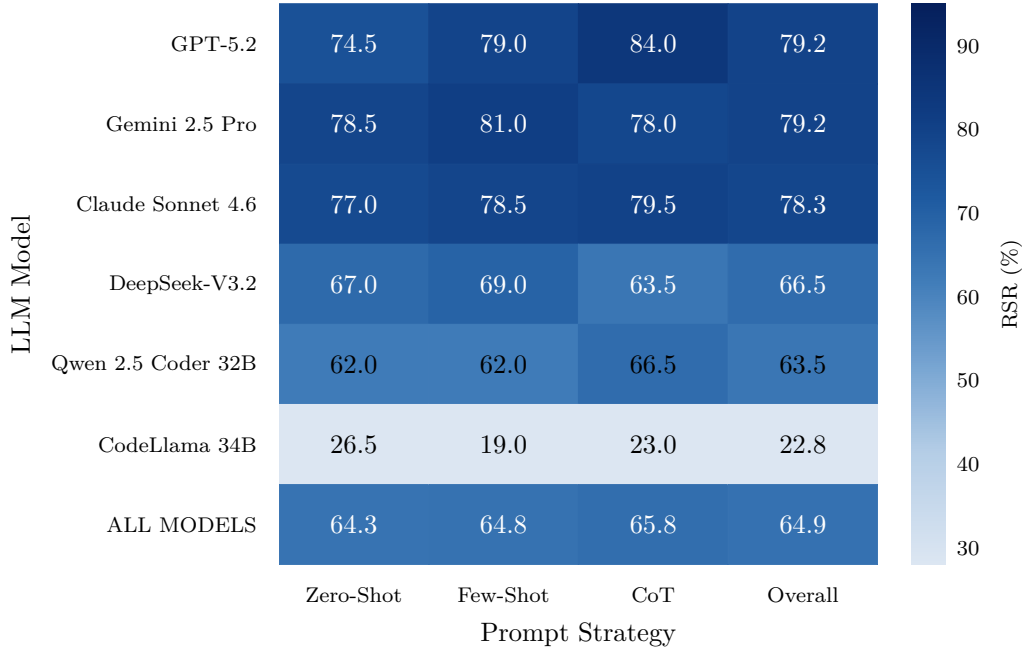


Figure 4.14: Heatmap of RSR by Model and Prompting Strategy

der CoT (**63.5%**), showing that prompt benefits are not uniform across architectures.

The weakest model, *CodeLlama 34B*, remains low under all strategies, ranging from **19.0%** to **26.5%**. This confirms that prompt design cannot compensate for substantial model limitations. Even the best-performing strategy leaves CodeLlama far below the remaining models.

Overall, the heatmap reinforces the explanatory conclusion that prompt effects are *conditional rather than universal*: they matter most when the underlying model is already capable, and least when the model lacks sufficient transformation quality to satisfy the combined quality requirements.

4.5.5 Failure Cause Diagnosis

Table 4.12 summarizes the main causes of failure among the 1,263 unsuccessful refactorings.

Table 4.12: Distribution of Refactoring Failure Causes

| Cause | Description | Count | % of Failures |
|--------------------|-------------------------------------|-------|---------------|
| removal_failure | Smell not removed | 840 | 66.5% |
| behavior_violation | Behavioral regression | 493 | 39.0% |
| coverage_violation | Coverage degradation (< -0.5 pp) | 63 | 5.0% |

The most frequent cause is *removal failure*, observed in **840 cases (66.5%** of failures), showing that complete success is limited mainly by unsuccessful smell removal rather than by downstream validation. The second most frequent cause is *behavioral violation*, affecting **493 cases (39.0%)**. This reinforces the importance of semantic validation, since refactoring may remove or appear to address the smell while still introducing regressions that invalidate the result.

By contrast, *coverage violation* is relatively uncommon, appearing in only **63 cases (5.0%)**. This does not mean that coverage is irrelevant, but rather that meaningful coverage degradation is a secondary failure mode compared with smell persistence or behavioral regression. Taken together, this diagnostic profile clarifies where integrated refactoring success breaks down. Most failures occur because the targeted smell remains unresolved, while a smaller but still substantial share results from behavioral instability. Coverage loss, under the adopted threshold, contributes relatively little to the overall failure profile.

4.5.6 Explanatory Synthesis

RQ5 reveals a clear hierarchy of experimental influence. Among the factors considered, **model choice** is by far the strongest explanatory variable. Prompting strategy contributes only limited aggregate variation, although it can produce noticeable gains for specific models, particularly stronger ones such as *GPT-5.2*. This means that prompt engineering is best interpreted as a secondary optimization layer rather than as a substitute for model capability.

The interaction analysis also shows that integrated success is more difficult to achieve than any isolated outcome from RQ1–RQ4. A model must not only remove the smell, but do so without breaking test behavior and without causing meaningful coverage degradation. As a result, RQ5 provides a more realistic view of practical refactoring utility: performance differences across configurations emerge from the joint ability to satisfy multiple quality constraints simultaneously.

More broadly, the results suggest that performance variation in LLM-based refactoring reflects differences in how models balance structural transformation strength against semantic and structural stability. The best-performing models are not simply those that transform code aggressively, but those that do so while remaining behaviorally and structurally controlled.

4.5.7 Answer to RQ5

In response to RQ5, the results show that experimental factors influence LLM-based refactoring outcomes primarily through **model choice**, while prompting strategy plays a secondary role. The overall RSR is **64.9%**, but this value masks strong differences across models: the best models achieve success rates near **80%**, whereas the weakest model remains below **25%**.

Prompting strategy has only a limited aggregate impact, although structured prompting can improve outcomes for some already strong models. The model $\times$ prompt matrix further shows that prompting benefits are not uniform, but depend on the underlying model’s ability to translate prompt guidance into high-quality code transformations.

Failure diagnosis reveals that most unsuccessful attempts are explained by *smell persistence*, followed by *behavioral regression*, while coverage degradation is comparatively rare. Thus, the main explanatory conclusion of RQ5 is that successful LLM-based refactoring depends chiefly on the intrinsic transformation capability of the model, with prompt design acting as a complementary but not decisive factor.

4.6 Conclusion of Results

This chapter presented an empirical evaluation of LLM-based refactoring for JavaScript test smells across five complementary quality dimensions, based on 3600 refactoring attempts derived from 200 validated smell instances and six state-of-the-art models under three prompting strategies. The structural analysis showed that LLMs achieve a global SRR of 76.7%, indicating that detector level smell elimination is frequently attainable. However, effectiveness varied substantially across smell types, ranging from 92.2% for *Unknown Test* to 33.6% for *Verbose Statement*. This gradient reveals a fundamental pattern: localized transformations succeed consistently, whereas refactorings requiring broader semantic restructuring remain persistently challenging. The behavioral analysis revealed that most refactorings preserve execution outcomes, with a global PPR of 86.3%. Critically, the divergence between SRR and PPR for certain smells demonstrates that satisfying a detection tool does not guarantee semantic equivalence. *Exception Handling* exemplifies this tension, achieving 91.9% SRR while preserving behavior in only 68.1% of cases. A failure taxonomy further showed that regressions manifest predominantly as intra suite failures rather than catastrophic execution errors, indicating that the primary risk lies in subtle semantic drift. The coverage analysis introduced a third quality dimension, showing that refactorings overwhelmingly preserve structural adequacy under the adopted threshold, with meaningful coverage degradation confined to a small minority of experiments (1.7%) and concentrated mainly in smells such as *Exception Handling*, whose refactoring affects exception propagation and can alter the set of executed paths. This finding confirms that behavioral preservation alone does not capture the full structural impact of refactoring, as a test may continue to pass while exercising fewer execution paths. The analysis of newly introduced smells revealed that refactoring is not purely corrective. New smells were introduced in a meaningful portion of experiments, with *Duplicate Assert* and *Conditional Test Logic* accounting for nearly 70% of all new introductions. The interaction matrix further showed that certain refactoring operations systematically produce specific new smells, indicating that new-smell introduction follows predictable refactoring patterns rather than random noise. The integrated analysis synthesized these di-

mensions, yielding a global RSR of 64.9%. This more conservative estimate reflects the cumulative difficulty of simultaneously removing the targeted smell, preserving behavior, and maintaining coverage stability within a single refactoring. Across all dimensions, a consistent hierarchy emerged: model choice exerts a dominant influence on refactoring outcomes, while prompting strategy plays a secondary role. Proprietary frontier models, including *GPT-5.2*, *Claude Sonnet 4.6*, and *Gemini 2.5 Pro*, consistently outperformed open weight alternatives such as *DeepSeek-V3.2*, *Qwen 2.5 Coder 32B*, and *CodeLlama 34B*, though higher transformation capability was sometimes associated with increased side effect introduction. Together, these findings establish an empirical baseline for understanding what LLM-based test refactoring can and cannot yet accomplish. The results show that current models possess meaningful capabilities for removing JavaScript test smells, but also reveal persistent constraints that define the boundaries of reliable deployment. Effectiveness is strongly conditioned by the nature of the targeted smell, successful smell removal does not guarantee behavioral preservation, improvements in one dimension may introduce regressions in others, and the choice of the underlying model fundamentally shapes outcomes. The following chapter interprets these patterns, examines their implications for research and practice, and discusses how these constraints inform the design of realistic AI-assisted test maintenance workflows.

Chapter 5

Discussion

The results presented in the previous chapter show that LLM-based test smell refactoring achieves substantial effectiveness when evaluated along isolated dimensions, but reveals important limitations when these dimensions are considered jointly. In particular, while smell removal reaches 76.7% and behavioral preservation reaches 86.3%, the integrated success rate decreases to 64.9%, indicating that satisfying multiple quality requirements simultaneously remains a non-trivial challenge. This gap shows that refactoring quality cannot be fully characterized by any single metric, but instead emerges from the interaction between smell removal, behavioral preservation, and coverage stability.

These findings suggest that LLM-based refactoring should be understood as a conditional capability rather than a uniformly reliable process. In practice, successful refactoring requires simultaneously removing the targeted smell, preserving the original behavior of the test, and maintaining its coverage adequacy. However, the results show that these objectives are not consistently achieved together. In many cases, refactorings that successfully eliminate a smell introduce behavioral deviations or reduce coverage, while behavior-preserving changes may fail to fully resolve the targeted smell. This indicates that refactoring outcomes are shaped by trade-offs between competing requirements, rather than by the optimization of a single objective.

Building on this perspective, this chapter provides a transversal interpretation of the results obtained across the research questions, with the objective of explaining the conditions under which LLM-based refactoring succeeds or fails. Rather than reiterating empirical findings, the discussion focuses on the mechanisms underlying the observed behavior and on the interactions between the different quality dimensions.

The discussion is organized as follows:

- Section 5.1 integrates the results across all dimensions and explains the gap between isolated and combined performance.

- Section 5.2 investigates how smell removal effectiveness varies as a function of refactoring complexity.
- Section 5.3 examines the relationship between smell removal and behavioral preservation, highlighting the structural and semantic gap.
- Section 5.4 discusses coverage variation as an independent indicator of test adequacy.
- Section 5.5 analyzes the role of model capability and prompting strategies.
- Section 5.6 evaluates the occurrence of newly introduced smells after refactoring.
- Section 5.7 discusses threats to validity.
- Section 5.8 presents implications for research and practice.

5.1 A Multi-Dimensional Interpretation of Refactoring Success

The interpretation of refactoring success emerges progressively from the combined evidence obtained across the different analytical dimensions considered in this study. The results first establish, as discussed in RQ1, that LLMs are capable of performing test smell removal at a substantial scale, achieving an overall Smell Removal Rate (SRR) of approximately 76.7%, which confirms that automated refactoring can eliminate a large portion of detected test smells. However, this effectiveness is not uniformly distributed, as removal rates vary significantly across smell types, ranging from over 90% in categories such as *Unknown Test* and *Exception Handling* to approximately 33.6% in *Verbose Statement*. This indicates that the ability to remove smells depends strongly on the complexity of the required refactoring. Despite this relatively high removal capability, smell removal alone does not fully characterize refactoring quality. As highlighted in RQ2, preserving observable behavior constitutes a distinct and non-trivial requirement, with a global Pass Preservation Rate (PPR) of 86.3%, showing that a non-negligible portion of refactorings that successfully remove smells still introduce regressions at the test execution level. This discrepancy demonstrates that removing a targeted smell and preserving test behavior are not inherently aligned, and that refactorings capable of eliminating test quality problems may still violate execution semantics.

This distinction becomes even more evident when considering test adequacy, as examined in RQ3, where coverage variations reveal that behavior-preserving refactorings do not necessarily maintain the same level of execution reach. Although many refactorings preserve coverage within the defined threshold, a relevant subset introduces reductions that indicate a contraction in the diversity of exercised program elements, even when all tests continue to pass. Given that coverage is widely

used as a proxy for test adequacy (Meinke, 2021), this result shows that correctness alone is insufficient to guarantee test effectiveness. In parallel, the analysis of newly introduced smells in RQ4 demonstrates that refactoring is not an isolated improvement process. A considerable number of refactorings introduce new test smells, with categories such as *Duplicate Assert* and *Conditional Test Logic* accounting for a substantial portion of the newly introduced issues, indicating that removing one smell may generate additional test quality problems that partially offset the benefits of the refactoring. This behavior reinforces the presence of trade-offs, where eliminating one type of deficiency may lead to the emergence of another.

Taken together, these observations indicate that each dimension captures a relevant but incomplete aspect of refactoring quality, and that a comprehensive interpretation requires considering their interaction rather than their isolated effects. Test smell removal, behavioral preservation, coverage stability, and the absence of newly introduced smells constitute complementary requirements that must be satisfied jointly, which is precisely the condition formalized in RQ5 through the Refactoring Success Rate (RSR). When these dimensions are combined, the overall success rate decreases to approximately 64.9%, revealing a substantial gap between isolated and integrated performance. This reduction is not merely a consequence of lower effectiveness, but rather the result of cumulative constraints, where each additional requirement excludes refactorings that would otherwise be considered successful when evaluated independently.

The analysis of failure causes further clarifies where this reduction concentrates. Most unsuccessful refactorings are associated with incomplete smell removal, confirming that removing the targeted smell remains the primary bottleneck even in the presence of relatively high SRR values. Behavioral regressions constitute the second most frequent source of failure, particularly in smells involving control flow, asynchronous behavior, or exception handling, where refactorings tend to be more intrusive. Coverage-related failures, although less frequent, still represent a distinct class of inadequacy, indicating that some refactorings reduce the execution reach of the test suite without necessarily affecting its observable outcomes. This distribution explains the gap between isolated and integrated performance by showing that refactoring success depends on satisfying multiple conditions simultaneously rather than independently, effectively reducing the feasible space of valid refactorings.

Overall, the results indicate that LLM-based refactoring operates under a constrained solution space, in which refactorings must simultaneously achieve smell removal, semantic preservation, and adequacy maintenance. While models demonstrate strong capability in addressing individual aspects of the problem, their performance decreases when these requirements are combined, suggesting that current approaches are better suited for localized refactorings than for globally consistent refactoring. This limitation highlights the need for evaluation frameworks that capture the interaction between quality dimensions, as single-metric assessments tend to overestimate effectiveness by ignoring the accumulation of partial failures. The use of integrated criteria, as represented by RSR, therefore provides a more realistic

assessment of refactoring reliability in practical software engineering scenarios.

More importantly, this distribution explains the gap between isolated and integrated performance by showing that refactoring success depends on satisfying multiple conditions simultaneously rather than independently, which effectively reduces the feasible space of valid refactorings. A refactoring attempt must not only remove the targeted smell, but also preserve observable behavior and maintain coverage within the defined threshold, and each of these requirements acts as a filtering condition over the set of possible outputs generated by the model. As a result, refactorings that remove the targeted smell may still fail due to behavioral regression, while behavior-preserving refactorings may be considered unsuccessful due to incomplete smell removal or coverage reduction, indicating that success is determined by the intersection of conditions rather than their individual satisfaction.

This behavior also reveals a characteristic limitation of current LLM-based refactoring approaches. The results suggest that models tend to optimize for local compliance with the requested refactoring, focusing on eliminating the visible manifestation of a test smell, but do not consistently account for the broader implications of these changes across other quality dimensions. In this sense, the refactoring process can be interpreted as a form of local optimization under incomplete global constraints, in which the primary objective is achieved without fully satisfying secondary requirements related to semantic preservation and adequacy, reinforcing the need to evaluate automated refactoring beyond smell removal alone (Pressman and Maxim, 2021).

From a software engineering perspective, these findings reinforce the interpretation of refactoring as an inherently multi-dimensional activity, in which smell removal, semantic correctness, and test adequacy must be considered jointly in order to achieve meaningful quality gains (Pressman and Maxim, 2021; Myers et al., 2011). The results demonstrate that improvements along a single dimension do not necessarily translate into overall effectiveness, and that trade-offs are intrinsic to the refactoring process. From an evaluation standpoint, this implies that single-metric assessments tend to overestimate the effectiveness of automated refactoring approaches, as they fail to capture the interactions between dimensions and the accumulation of partial failures that emerge under integrated evaluation. An approach based on combined criteria, as represented by RSR, is therefore necessary to provide a realistic assessment of refactoring reliability in practical scenarios.

5.2 Smell Removal Effectiveness as a Function of Refactoring Complexity

As indicated by the multi-dimensional analysis discussed in the previous section, the effectiveness of LLM-based refactoring cannot be understood as a uniform capability, but rather as a phenomenon that varies across different refactoring conditions. In

this context, the variability observed in SRR across smell types suggests that smell removal is conditioned by the complexity of the refactoring required, revealing the existence of a refactoring complexity spectrum.

As presented in Section 4.1, at the upper end of this spectrum, smells such as *Unknown Test*, *Exception Handling*, and *Magic Number* exhibit consistently high removal rates. These results indicate that such smells are more consistently removed by the models, suggesting that their correction can be achieved through refactorings that do not require extensive restructuring of the test. For example, in the case of *Exception Handling*, prior work characterizes the refactoring as the replacement of imperative exception-handling constructs with framework-level mechanisms for exception verification (Meszaros, 2007; Peruma et al., 2019; Soares, 2023). This type of refactoring is well-defined and limited in scope, which is consistent with the higher effectiveness observed in the results.

For smells presenting intermediate removal rates, such as *Conditional Test Logic*, *Duplicate Assert*, and *Suboptimal Assert*, the results show a clear decrease in smell removal effectiveness when compared to the previous group. This difference suggests that the correction of these smells is less consistently achieved by the models. Based on the observed outcomes, these cases can be interpreted as involving refactorings that extend beyond strictly localized edits, potentially affecting multiple elements of the test structure. As a result, the refactoring process appears to require a higher degree of coordination across different parts of the test, which increases the likelihood of partial or inconsistent refactorings.

At the lower end of the spectrum, *Verbose Statement* exhibits the lowest removal rates, indicating a qualitatively different type of refactoring challenge. According to the test smell literature, this smell is associated with excessively large and conceptually overloaded test methods, whose refactoring typically involves decomposing a single test into multiple smaller and more focused units (Meneses, 2025; Soares et al., 2024). Unlike the more localized corrections observed in other smell categories, this refactoring requires reorganization at the level of the test as a whole. This broader scope of modification is consistent with the reduced effectiveness observed in the results, as it demands more extensive changes than those required by simpler, localized refactorings.

Taken together, these observations indicate that smell removal is closely tied to the nature of the refactoring required, particularly with respect to how localized or extensive the necessary changes are. Rather than suggesting a binary distinction between simple and complex cases, the results point to a continuum in which increasing refactoring complexity is associated with progressively lower reliability in automated refactoring. This interpretation follows directly from the empirical distribution of removal rates across smell types and should therefore be understood as a characterization of externally observable behavior, rather than as evidence of the internal reasoning processes of the models. In this sense, the observed pattern aligns with established perspectives in refactoring research, which emphasize that

refactorings involving broader changes require a deeper understanding of program structure and intent (Fowler, 2018; Mens and Tourwe, 2004).

5.3 The Structural and Semantic Gap in Behavioral Preservation

The relationship between smell removal and behavioral preservation reveals a consistent gap between the ability of LLMs to perform test smell refactoring and their ability to maintain the original execution behavior of the test suite. As discussed in Section 4, the overall Smell Removal Rate (SRR) reaches approximately 76.7%, while the global Pass Preservation Rate (PPR) is 86.3%, indicating that although most refactoring attempts preserve behavior, a non-negligible portion of cases that remove the targeted smell still lead to behavioral deviations. This discrepancy shows that smell removal alone does not guarantee semantic correctness, since a subset of refactorings that successfully eliminate the targeted smell still alters the observable behavior of the test suite. Therefore, smell removal and behavioral preservation must be understood as distinct, albeit related, dimensions of refactoring quality, rather than interchangeable indicators of success.

This gap becomes more evident when analyzing smell categories whose refactoring directly interacts with execution semantics. In the case of *Sleepy Test*, the results show a PPR of 74.2%, which is substantially below the global PPR of 86.3%, indicating a higher incidence of behavioral inconsistencies after refactoring. An even more pronounced effect is observed for *Exception Handling*, which presents the lowest preservation rate among all categories at 68.1%, despite achieving one of the highest SRR in RQ1. This divergence of more than 18 percentage points between smell removal and behavioral preservation highlights a clear tension between eliminating the smell and maintaining execution equivalence. In both cases, the literature characterizes these smells as involving timing dependencies or exception management mechanisms (Peruma et al., 2019), which are intrinsically tied to runtime behavior. These findings indicate that refactorings affecting these aspects alter execution-level properties such as synchronization, control flow, or error propagation, increasing the likelihood of deviations in test outcomes even when the targeted smell is removed.

In contrast, other smell categories exhibit a much closer alignment between smell removal and behavioral preservation. For instance, *Anonymous Test* achieves a PPR of 95.3% and *Duplicate Assert* reaches 94.7%, both substantially above the global average of 86.3%. These values indicate that refactorings in these categories preserve behavior in the vast majority of cases. According to the literature, these smells are primarily associated with naming clarity and redundant assertions (Peruma et al., 2019), which do not directly affect control flow, execution timing, or state transitions. The results therefore suggest that refactorings that operate at the syntactic or organizational level, without modifying execution semantics, are significantly more stable from a behavioral perspective. This contrast reinforces that the risk of behav-

ioral deviation is not uniformly distributed across smell types, but instead depends on how closely the refactoring interacts with runtime properties of the test.

The distribution of failure types provides additional empirical support for this interpretation by detailing how behavioral deviations manifest across the dataset. As reported in Section 4.2, intra-suite failures account for 309 cases, corresponding to 8.6% of all experiments, while suite-level failures represent 182 cases, or 5.1%, and execution-level errors are extremely rare, with only 2 occurrences, corresponding to 0.1%. This distribution shows that most behavioral inconsistencies introduced by refactoring are localized, affecting individual test cases rather than causing complete execution breakdowns. In practical terms, this means that the refactored test suite typically remains executable, but exhibits deviations in specific scenarios such as altered assertions, modified control paths, or changes in timing behavior. The dominance of intra-suite failures indicates that LLMs generally preserve the overall execution structure of the test suite, while introducing subtle inconsistencies that are sufficient to cause failures at the test-case level.

Taken together, these observations highlight the existence of a structural and semantic gap in automated test refactoring. Although LLMs demonstrate strong capability in removing test smells, as evidenced by the SRR of 76.7%, this smell removal capability does not inherently guarantee behavioral preservation. In this context, the results show that detector-based success should not be interpreted as a proxy for semantic validity, since refactorings that satisfy smell detection criteria may still alter execution behavior. Consequently, behavioral preservation must be treated as an independent and essential requirement, reinforcing the need for dynamic validation when applying automated refactoring techniques in practical software engineering scenarios.

5.4 Coverage Reduction as an Independent Quality Signal

The coverage analysis provides an additional perspective on refactoring quality by revealing effects that are not fully captured by SRR or PPR. As presented in Section 4.3, coverage variation was computed by comparing coverage before and after refactoring, with each experiment classified using a tolerance threshold of ± 0.5 percentage points: an experiment is considered improved if at least one of the four coverage metrics increases beyond this threshold, degraded if at least one metric decreases beyond it and none increases, and preserved otherwise. Under this criterion, coverage remains preserved in 97.0% of experiments, improves in 1.3%, and presents meaningful reduction in only 1.7%. These values indicate that coverage behavior is overwhelmingly stable, with degradation confined to a small minority of cases. Even though the mean deltas are positive across all measured dimensions, including statements (+0.840 pp), branches (+0.754 pp), functions (+0.878 pp), and lines (+0.837 pp), the classification results show that aggregate gains coexist with a small number

of localized coverage reductions. This distinction is important because a refactoring may preserve the observable execution outcome while still reducing the set of program elements exercised by the tests. Therefore, coverage reduction should be interpreted as an independent quality signal, since it captures changes in test adequacy that are not necessarily reflected by smell removal or behavioral preservation alone.

This independence becomes clearer when coverage behavior is analyzed by smell type. The strongest coverage risk appears in *Exception Handling*, which presents the highest degradation rate at 4.7%, followed by *Overcommented Test* at 3.1% and *Anonymous Test* at 2.2%. These values are above the global degradation rate of 1.7%, indicating that coverage reduction is not uniformly distributed across smell categories. In particular, smells whose refactoring affects exception propagation, comment or statement organization, or test structure are more likely to alter the set of exercised execution paths after refactoring. This result helps explain why observable correctness does not necessarily imply equivalent test adequacy. A test suite may continue to pass after refactoring while exercising fewer statements, branches, or functions, which reduces the execution reach of the test suite even in the absence of visible behavioral failure. Because coverage models are grounded in the measurement of exercised program elements and are commonly used as practical indicators of test adequacy (Ammann and Offutt, 2016; Spillner and Linz, 2021; Meinke, 2021), these reductions remain relevant for assessing refactoring quality.

At the same time, coverage analysis also shows that refactoring can improve test adequacy in specific contexts. *Duplicate Assert* is the clearest example, with positive coverage deltas of approximately +4.5 percentage points across the measured metrics and no cases of meaningful degradation (0.0%). This indicates that refactoring redundant assertions can, in some cases, make the test structure more direct while preserving or expanding the exercised program elements. A similar pattern is observed for *Unknown Test*, which presents a mean statement coverage gain of approximately +4.2 percentage points, alongside a low degradation rate of 1.7%. This result suggests that refactorings aimed at introducing or clarifying explicit assertions may increase the effective reach of the test suite by exercising program elements that were not adequately covered before.

Coverage behavior at the model level reveals that test adequacy is generally preserved across different LLMs, but not in a uniform or consistent manner. All evaluated models present positive mean statement coverage variation, ranging from +0.619 percentage points for *CodeLlama 34B* to +0.909 for *Qwen 2.5 Coder 32B*, indicating that, on average, refactoring does not systematically degrade coverage. Under the adopted threshold, degradation rates by model range from 1.0% for *Qwen 2.5 Coder 32B* to 2.7% for *CodeLlama 34B*, showing that coverage stability, while consistently high, is not perfectly uniform across models. This suggests that the observed differences reflect not only variation in average performance, but also differences in how consistently each model preserves the execution reach of the test suite. This variability becomes more pronounced when considering the interaction

between model and prompting strategy. While most model and prompt combinations remain within a narrow range of positive variation, suggesting broadly stable behavior, specific configurations deviate substantially from this pattern. The most notable case is *CodeLlama 34B* under CoT, which presents a sharp decrease in mean statement coverage, indicating that certain models may become less stable when exposed to more complex prompting structures. This result suggests that coverage degradation is not solely a function of the targeted smell, but is also influenced by the interaction between model capability and the type of refactoring induced by the prompt. In particular, weaker or less robust models appear more susceptible to producing refactorings that alter the execution reach of the test, even when the intended modification is primarily related to smell removal.

Overall, coverage behavior remains overwhelmingly stable at the aggregate level, although it varies across individual cases and across models and prompting strategies. While the vast majority of cases preserve or slightly improve coverage, a small subset, corresponding to 1.7% of experiments, introduces measurable coverage reduction, particularly in smell categories and configurations that affect exception propagation or test structure. This demonstrates that test adequacy is sensitive to the nature of the changes applied, even when smell removal and behavioral preservation are achieved. In this sense, coverage variation exposes effects that remain undetected by smell removal and behavioral preservation, revealing situations in which the test suite loses execution reach without affecting observable outcomes. As a result, coverage should be considered an independent and necessary dimension in the evaluation of automated refactoring, since it captures whether the resulting tests continue to exercise the program with the same level of adequacy.

5.5 Model Capability as the Primary Driver of Refactoring Outcomes

The comparative analysis across models and prompting strategies indicates that variation in refactoring outcomes is primarily driven by differences between models rather than by the choice of prompting strategy. As observed in the results, the gap in Smell Removal Rate between the best-performing and lowest-performing models exceeds 50 percentage points, with models such as *Gemini 2.5 Pro* achieving values above 90%, while *CodeLlama 34B* remains below 40%. A similar pattern is observed in the integrated success criterion, where stronger models consistently achieve substantially higher success rates across prompt configurations. This magnitude of variation suggests that model capability is the dominant factor in determining whether refactoring attempts satisfy smell removal, behavioral preservation, and adequacy-related constraints.

This difference reflects not only variation in overall performance, but also in the ability to consistently satisfy multiple requirements simultaneously. Stronger models maintain high smell removal effectiveness while preserving behavior and limiting the

introduction of new smells, indicating a greater capacity to coordinate changes across different dimensions of the refactoring task. In contrast, lower-performing models exhibit higher rates of partial success, where one objective is achieved at the expense of others, particularly in scenarios that require more extensive modifications. This behavior suggests that refactoring performance depends on the ability to balance competing constraints, rather than optimizing a single objective in isolation. The impact of model capability becomes more evident as task complexity increases. As discussed in the previous sections, smell categories that require broader changes or interact with execution semantics present lower overall success rates. In these cases, performance degradation is not uniform across models, but disproportionately affects those with lower capability. This indicates that more demanding refactoring scenarios amplify existing differences between models, reinforcing that performance is constrained not only by the task itself, but by the capacity of the model to handle dependencies and contextual interactions.

In contrast, the effect of prompting strategy remains comparatively limited at the aggregate level. Our empirical results indicate that differences between *Zero-shot*, *Few-shot*, and CoT are relatively small when compared to the variation observed across models, indicating that prompting alone does not substantially alter overall performance. Instead, its effect appears to be conditional on model capability. Stronger models may benefit from more structured prompts, showing incremental gains in specific configurations, while weaker models do not exhibit consistent improvement. This suggests that prompting can refine or guide the behavior of capable models, but is insufficient to compensate for limitations in model architecture, training, or reasoning ability.

5.6 Smell-Dependent Risk and New-Smell Introduction

As reported in Section 4.4, the target smell was removed in 2,760 cases, corresponding to 76.7% of the analyzed attempts, while 484 refactorings, or 13.4%, introduced at least one new smell after refactoring. Across the dataset, a total of 1,532 new smell instances were detected, corresponding to an average of 0.426 added smells per refactoring attempt. These results show that smell removal frequently coexists with the introduction of new smells, indicating that refactoring is not strictly cumulative in terms of test quality. In practical terms, a refactoring may eliminate the targeted smell while simultaneously degrading other parts of the test code. This distinction highlights that smell removal and new-smell introduction represent related but independent outcomes, and that evaluating refactoring quality requires considering both dimensions jointly.

This trade-off varies substantially across smell categories. *Unknown Test* achieves one of the highest removal rates, with 332 successful removals out of 360 attempts, corresponding to 92.2%, but it also introduces new smells in 110 cases, or 30.6%

of its experiments. A similar pattern appears for *Sleepy Test*, which combines a high removal rate of 81.3% with a new-smell introduction rate of 28.7%. These results show that high removal effectiveness does not necessarily imply low risk of adding new smells. In both categories, the models frequently achieve the intended correction, but the correction is accompanied by a comparatively high incidence of additional test smells. This indicates that some smell removals are productive but unstable, since they improve the targeted dimension while increasing the likelihood of additional smells.

In contrast, *Exception Handling* presents a much more stable profile. It reaches a removal rate of 91.9%, with 331 successful removals out of 360 attempts, while introducing new smells in only 0.6% of experiments. This combination of high effectiveness and low new-smell introduction suggests that some smell categories can be refactored in a more constrained and predictable way. *Anonymous Test* and *SubOptimal Assert* also exhibit relatively low new-smell introduction rates, with 1.9% and 3.9%, respectively, despite moderate removal effectiveness. At the opposite extreme, *Verbose Statement* combines the lowest removal rate, 33.6%, with a non-negligible new-smell introduction rate of 15.8%. This profile indicates that difficulty is not limited to failing to remove the original smell. In some cases, the same refactoring attempt may both fail to remove the targeted smell and introduce additional test quality problems.

The taxonomy of newly introduced smells further shows that new-smell introduction is highly concentrated rather than evenly distributed, as illustrated in Figure 4.11. *Duplicate Assert* accounts for 605 newly introduced instances, representing 39.5% of all added smells and appearing in 16.8% of experiments. *Conditional Test Logic* follows with 452 added instances, corresponding to 29.5% of all new smells and 12.6% of experiments. Together, these two smell categories account for nearly 70% of all newly introduced smells observed in the dataset. This concentration indicates that the most common added smells are related to redundant assertions and conditional logic, rather than being randomly distributed across smell types. Figure 4.11 further clarifies this pattern by showing how specific refactoring actions are associated with the introduction of particular secondary smells. In particular, refactoring *Conditional Test Logic* introduces *Duplicate Assert* in 487 instances, indicating a strong directional relationship between these categories. This result shows that new-smell introduction is not only concentrated, but also structured, with certain refactoring paths consistently leading to the same types of secondary issues.

Model-level results add another layer to this pattern by showing that the incidence of new-smell introduction varies substantially across models. *GPT-5.2* introduces new smells in 22.7% of experiments, the highest rate among the evaluated models, followed by *Gemini 2.5 Pro* at 20.6% and *Claude Sonnet 4.6* at 19.1%. Among open-weight models, *DeepSeek-V3.2* presents a rate of 14.8%, while *Qwen 2.5 Coder 32B* reaches 12.5%. In contrast, *CodeLlama 34B* exhibits the lowest new-smell introduction rate at 6.3%. This distribution shows that models differ not only in effectiveness, but also in how aggressively they modify the test structure. This

contrast indicates that stronger smell removal capability does not automatically imply cleaner refactored code. Models with higher smell removal effectiveness tend to apply broader or more invasive modifications, which increases the likelihood of introducing new smells. Conversely, models with lower new-smell introduction rates may appear more stable, but this behavior is partially explained by their reduced ability to perform extensive refactorings. Therefore, new-smell analysis is necessary to distinguish between models that achieve smell removal with controlled modifications and those that introduce fewer new smells primarily because they perform less substantial changes.

Taken together, these results show that new-smell introduction is a systematic component of LLM-based refactoring outcomes. The evidence from RQ4 indicates that the risk of introducing new smells depends on the targeted smell, the type of refactoring performed, and the model generating the refactoring. This finding is consistent with empirical studies showing that test refactoring may introduce unintended consequences and therefore requires careful validation (Soares et al., 2023; Damasceno et al., 2022). In this context, refactoring quality should be interpreted as a balance between removing the original smell and avoiding the introduction of additional smells. A successful automated refactoring process should therefore minimize not only smell persistence, but also the introduction of new test smells.

5.7 Threats to Validity

We discuss threats to the validity of our empirical evaluation following the framework proposed by Wohlin et al. (2012), while grounding each dimension in the specific characteristics of LLM-based test smell refactoring and the experimental design adopted in this study.

Construct validity. Construct validity concerns how accurately the selected metrics represent the underlying concepts of interest (Wohlin et al., 2012). In this study, smell removal is operationalized through automated test smell detection tools, which serve as proxies for test design quality. However, these tools rely on rule-based or heuristic detection mechanisms and therefore may not fully capture context-dependent design deficiencies or more subtle test quality problems. As a consequence, the notion of smell removal is inherently tied to the behavior and limitations of the detection tools, meaning that a refactoring classified as successful may still retain residual design problems not captured by the detectors. This introduces a potential mismatch between the measured outcome and the actual quality of the test. To mitigate this limitation, the evaluation incorporates multiple complementary dimensions, including behavioral preservation and coverage variation, which provide additional perspectives on refactoring quality. By combining these metrics, the study reduces dependence on a single operationalization and offers a more comprehensive approximation of the underlying construct. Nevertheless, the reliance on automated detection remains an inherent constraint, particularly in cases where test quality depends on semantic or contextual aspects that are difficult to formalize.

Internal validity. Internal validity refers to the extent to which the observed effects can be attributed to the studied factors rather than to uncontrolled variables (Wohlin et al., 2012). The experimental design isolates each smell instance and applies refactoring independently, which improves comparability across models and prompting strategies by controlling for confounding factors. However, this isolation also reduces the availability of broader project context, which may influence the correctness and adequacy of refactorings in real-world scenarios. As a result, some generated changes may not fully reflect the decisions that would be made in a more contextualized development environment. In addition, LLM outputs are inherently non-deterministic, meaning that repeated executions may produce different refactoring solutions for the same input. This variability introduces uncertainty in the observed outcomes and may affect the stability of the results. Another important threat arises from hallucinated modifications, where the model generates syntactically valid but semantically incorrect changes that are not aligned with the intended refactoring objective. To mitigate these risks, all refactorings are validated through execution, ensuring that behavioral preservation is explicitly verified. Furthermore, the scale of the experiment, involving a large number of instances and configurations, reduces the influence of isolated variations, although non-determinism and context limitations cannot be entirely eliminated.

External validity. External validity concerns the extent to which the findings can be generalized beyond the studied context (Wohlin et al., 2012). The evaluation is conducted on JavaScript repositories using the Jest testing framework, which reflects a widely adopted ecosystem but does not encompass the diversity of programming languages, testing frameworks, and industrial practices. Differences in language features, testing paradigms, and project scale may influence how refactoring behaves in other contexts, potentially limiting direct generalization. In addition, the study focuses on a predefined set of ten test smell categories, which, although representative of common issues, do not cover the full spectrum of possible test design problems. Another important consideration relates to the selection of LLMs. While the study includes both proprietary and open-weight models with diverse architectures and training characteristics, the results are tied to the specific models evaluated and cannot be assumed to generalize to all existing or future LLMs, especially given the rapid pace of advancement in this field. However, the experimental framework itself, including the evaluation pipeline, metrics, and validation procedures, is not specific to a particular language or model, and can be replicated in other environments. Therefore, while the quantitative outcomes may vary, the methodological approach remains transferable and can support future studies in different contexts.

Conclusion validity. Conclusion validity addresses the extent to which the conclusions drawn from the data are reliable and adequately supported by the analysis (Wohlin et al., 2012). In this study, aggregate metrics such as smell removal, behavioral preservation, and coverage variation provide a high-level summary of performance, but may obscure fine-grained variability across smell types, models, and prompting strategies. For instance, global averages may suggest stable perfor-

mance while masking localized failures, regressions, or coverage degradation. To address this limitation, the analysis incorporates disaggregated evaluations, failure categorization, and interaction analysis, enabling a more detailed and nuanced interpretation of the results. Another relevant aspect concerns the interpretative nature of some conclusions, particularly when explaining the mechanisms underlying successful or unsuccessful refactorings. While these interpretations are grounded in empirical observations, they inevitably involve analytical judgment and may be influenced by the researchers' perspective. To mitigate this threat, interpretations are consistently supported by explicit data and are framed cautiously to avoid overgeneralization. Although some level of subjectivity is unavoidable in empirical analysis, the combination of quantitative evidence and structured reasoning strengthens the reliability of the conclusions.

Overall, these threats delimit the scope of the findings but do not compromise their validity within the defined experimental context. The combination of multi-dimensional evaluation, large-scale experimentation, and systematic analysis contributes to reducing potential biases and supports the robustness of the results.

5.8 Implications for Research and Practice

The findings of this study suggest that LLM-based test refactoring is effective but inherently non-uniform. While models demonstrate strong capabilities in localized refactorings, their reliability decreases as tasks require deeper semantic reasoning or the simultaneous satisfaction of multiple quality constraints. This indicates that refactoring success is conditional rather than universal. From a research perspective, the results highlight the importance of adopting multi-dimensional evaluation frameworks. Assessing refactoring quality requires the integration of smell removal, behavioral preservation, and coverage impact, as each dimension captures distinct aspects of refactoring quality. These findings reinforce that improvements in one dimension do not necessarily translate into global success, motivating further investigation into techniques that can better align smell removal and semantic correctness.

From a practical perspective, our findings indicate that LLMs can be reliably applied to well-defined, localized refactoring tasks, particularly those involving explicit and pattern-based changes. However, more complex refactorings, especially those affecting execution semantics or requiring broader restructuring, demand additional validation. This reinforces the need for automated validation pipelines that combine execution testing, smell detection, and coverage monitoring. Ultimately, LLMs should be positioned as assistive tools rather than fully autonomous refactoring agents. Their current capabilities enable substantial productivity gains, but their limitations require human oversight and systematic validation to ensure safe and reliable integration into software development workflows.

5.9 Conclusion of the Discussion

The analysis developed throughout this chapter shows that the effectiveness of LLM-based test smell refactoring is inherently multi-dimensional and constrained by the need to satisfy multiple quality requirements simultaneously. While LLMs demonstrate strong capability in removing test smells, achieving a Smell Removal Rate above 76%, this capability does not directly translate into reliable refactoring outcomes when additional constraints are considered. The reduction observed from smell removal to behavioral preservation and ultimately to the integrated success criterion indicates that refactoring quality cannot be captured by isolated metrics, but instead emerges from the interaction between smell removal, semantic preservation, and test adequacy. In this context, the Refactoring Success Rate provides a more realistic assessment, revealing that a significant portion of refactorings considered successful under partial evaluation fails to meet the combined requirements expected in practice.

This limitation is systematically related to the nature of the refactoring task. Smell removal effectiveness varies according to the complexity of the required changes, with localized and well-defined refactorings achieving higher success rates than those requiring broader reorganization. At the same time, the analysis of behavioral preservation reveals a structural and semantic gap, where refactorings that successfully remove smells may still alter execution behavior, particularly in cases involving control flow, timing, or exception handling. This gap is reinforced by coverage results, which show that even behavior-preserving refactorings may reduce the execution reach of the test suite. Together, these findings demonstrate that observable correctness is not sufficient to guarantee test adequacy, and that refactoring quality depends on how changes interact with execution semantics and coverage.

Refactoring outcomes are also strongly influenced by model capability. Differences across models are substantially larger than those observed across prompting strategies, indicating that model characteristics are the primary driver of performance. Stronger models are better able to satisfy multiple constraints simultaneously, although they may introduce more aggressive changes that increase the likelihood of new-smell introduction. Conversely, weaker models tend to exhibit lower effectiveness and reduced stability, particularly in more complex scenarios. This pattern shows that prompting strategies alone are insufficient to compensate for limitations in model capability and that performance is fundamentally constrained by the model's ability to handle dependencies and contextual interactions.

The analysis of newly introduced smells further shows that refactoring is inherently associated with trade-offs. The introduction of new smells in a non-negligible portion of refactorings indicates that smell removal is not strictly cumulative, and that eliminating one issue may generate others. The concentration of new-smell introduction in specific smell categories, as well as the structured relationships observed in the interaction matrix, shows that these effects follow identifiable patterns rather than occurring randomly. As a result, refactoring quality should be interpreted in

terms of its net impact on the test code, considering both improvements and newly introduced problems.

Taken together, these findings indicate that LLM-based test refactoring is effective as a local refactoring mechanism, but limited as a globally consistent solution. Current models perform well in targeted, localized changes, but struggle to simultaneously preserve behavior, maintain coverage, and avoid new-smell introduction in more complex scenarios. This suggests that automated refactoring operates under a constrained solution space, where satisfying multiple quality dimensions requires a level of coordination that is not consistently achieved by current approaches. Consequently, evaluating refactoring quality requires integrated criteria that reflect these combined requirements, as single-metric assessments tend to overestimate effectiveness by ignoring the interaction between dimensions and the accumulation of partial failures.

Chapter 6

Conclusion

This dissertation examined the effectiveness of Large Language Models (LLMs) in the automated refactoring of JavaScript test smells through a large-scale empirical study grounded in multiple complementary quality dimensions. By jointly considering smell removal, behavioral preservation, coverage variation, newly introduced smells, and an integrated success criterion, the analysis moves beyond isolated performance indicators and frames refactoring as a complex software engineering activity governed by interacting constraints. This perspective enables a more precise understanding of both the capabilities and the limitations of current LLM-based approaches to automated test improvement.

6.1 Integrated Synthesis of Findings

Our results indicated that LLMs performed test smell removal at a substantial scale, achieving a global Smell Removal Rate (SRR) of 76.7% across 3,600 refactoring attempts distributed over ten test smell categories. However, this effectiveness was not uniformly distributed across scenarios. For instance, *Unknown Test*, *Exception Handling*, and *Magic Number* achieved removal rates of 92.2%, 91.9%, and 91.1%, respectively, whereas *Verbose Statement* reached only 33.6%. In practice, the highest effectiveness was observed in scenarios where the required changes were localized and explicit, allowing models to apply pattern-based modifications with relatively high consistency. These results highlighted the strength of LLMs as local refactoring mechanisms, capable of identifying and rewriting recurring test quality problems when the refactoring could be expressed through constrained and well-defined changes.

Yet, smell removal alone did not characterize refactoring quality. Although the global Pass Preservation Rate (PPR) reached 86.3%, the integrated Refactoring Success Rate (RSR) decreased to 64.9% when smell removal, behavioral preservation, and coverage preservation were jointly evaluated. This reduction of more than 20 percentage points demonstrated that gains observed in one dimension did not

necessarily extend to others, and that successful smell removal did not guarantee semantic correctness or coverage adequacy. The divergence between these metrics revealed that test quality is inherently multi-dimensional and cannot be reduced to a single indicator. Instead, reliable automated refactoring depends on simultaneously satisfying multiple constraints, each imposing distinct structural and semantic requirements on the generated changes.

A particularly relevant aspect of this misalignment was the separation between smell removal and semantic preservation. While many smell categories achieved high removal effectiveness, preserving the intended behavior proved substantially more difficult in scenarios involving execution semantics. For example, *Exception Handling* achieved an SRR of 91.9%, but its PPR decreased to 68.1%, representing the lowest preservation value among all evaluated smells. Similarly, *Sleepy Test* reached a preservation rate of only 74.2%, considerably below the global PPR. These results indicated that refactorings involving control flow, timing dependencies, or exception propagation were considerably more prone to behavioral inconsistencies, even when the targeted smell was removed. This disconnect exposed limitations in the ability of current LLMs to reason about the behavioral consequences of code modifications, particularly when correctness depends on implicit execution semantics rather than purely structural patterns.

Differences in refactoring performance were also strongly shaped by the characteristics of the smells being addressed. Smells resolved through localized edits consistently presented higher success rates, while those requiring coordinated modifications exhibited substantially lower effectiveness. *Unknown Test*, *Magic Number*, and *Exception Handling* remained above 90% removal effectiveness, whereas *Conditional Test Logic* and *Duplicate Assert* decreased to 71.1% and 67.2%, respectively. At the lower end of the spectrum, *Verbose Statement* reached only 33.6%. These differences established the existence of a refactoring complexity spectrum in which increasing structural and semantic demands progressively reduced the reliability of automated refactoring. As the required changes became more distributed across the test structure, the probability of incomplete or inconsistent modifications increased substantially.

Another important observation concerned the emergence of newly introduced smells. Although smell removal was achieved in most experiments, 13.4% of refactorings introduced at least one additional smell after refactoring, resulting in a total of 1,532 newly introduced smell instances. These new smells were not randomly distributed. *Duplicate Assert* alone accounted for 39.5% of all introduced smells, followed by *Conditional Test Logic* with 29.5%. Moreover, the interaction analysis showed that refactoring *Conditional Test Logic* introduced *Duplicate Assert* in 487 instances, revealing recurrent patterns of new-smell introduction rather than isolated failures. Consequently, assessing refactoring quality requires considering not only whether a target smell is removed, but also whether the resulting test avoids the introduction of additional smells after modification.

Performance differences were further influenced by the underlying characteristics of the evaluated models. Variation across models was substantially larger than that observed across prompting strategies. *Gemini 2.5 Pro* achieved Smell Removal Rates above 92%, while *CodeLlama 34B* remained below 40%. Similar differences were observed in the integrated success criterion, where stronger models consistently outperformed weaker ones across all prompt configurations. In contrast, the variation introduced by prompting strategies remained comparatively small, with *Zero-shot*, *Few-shot*, and CoT producing relatively similar aggregate results. These findings indicated that intrinsic model characteristics, such as architecture, training data, and reasoning capability, played a substantially more important role than prompt engineering alone. While structured prompting may provide incremental improvements in specific scenarios, it does not compensate for underlying limitations in model capability.

6.2 Scientific Contributions

This dissertation contributes to Software Engineering across empirical, methodological, conceptual, and practical dimensions, particularly at the intersection of AI for Software Engineering, Software Testing, and Software Maintenance. The main contributions are summarized as follows:

- **Empirical evidence on LLM-based test smell refactoring in JavaScript.** The study evaluates the effectiveness of six LLMs and three prompting strategies across ten JavaScript test smell categories. The results provide empirical evidence on how LLM-generated refactorings behave in JavaScript test maintenance scenarios, revealing differences in effectiveness across models, prompts, and smell categories, as well as limitations that emerge when generated changes are assessed beyond the direct removal of the targeted smell.
- **A multi-dimensional and reproducible workflow for evaluating LLM-assisted test refactoring.** The proposed evaluation framework jointly analyzes smell removal, behavioral preservation, coverage stability, and new-smell introduction, enabling refactoring quality to be assessed from multiple complementary perspectives rather than through a single outcome measure. This framework was implemented as a reproducible experimental workflow that integrates smell detection, prompt generation, refactoring execution, behavioral validation, coverage analysis, and post-refactoring smell detection into a unified process. As a result, the workflow supports controlled experimentation across different models, prompting strategies, and smell categories while preserving comparability among experimental conditions.
- **A literature-grounded catalog of JavaScript test smells and refactoring strategies.** The research organizes definitions, manifestations, detection characteristics, JavaScript examples, and refactoring guidance for the smell

categories investigated in the study, drawing on prior work on test smells and test refactoring. The catalog also serves as a knowledge source for the prompt templates employed throughout the experimental process, helping to standardize the instructions provided to the LLMs across models and prompting strategies. In this role, it reduces ambiguity in prompt construction and supports a more consistent operationalization of each smell category during the refactoring experiments.

- **Characterization of refactoring outcomes and implications for software maintenance.** The results reveal systematic differences in effectiveness across smell categories, models, and prompting strategies, as well as recurring patterns involving behavioral regressions, coverage variation, and new-smell introduction. These findings indicate scenarios in which LLMs can support test maintenance tasks and clarify the conditions under which their outputs require additional validation. In practical terms, the study reinforces the importance of combining LLM-generated refactorings with complementary validation mechanisms, such as automated testing, coverage analysis, and post-refactoring quality assessment.

6.3 Future Work

The findings of this dissertation open several directions for future investigation, particularly in addressing the limitations identified throughout the multi-dimensional evaluation of LLM-based refactoring. A first natural extension concerns the generalization of the experimental context. While this study focuses on JavaScript repositories using the Jest framework, evaluating additional programming languages and testing ecosystems would allow assessing whether the observed patterns, such as the structural and semantic gap or the refactoring complexity spectrum, remain consistent across different technological environments. Languages with stronger typing systems, different assertion mechanisms, or distinct testing abstractions may impose different constraints on automated refactoring, potentially affecting both smell removal and behavioral preservation. Similarly, extending the analysis to other categories of test smells may help determine whether the limitations observed in this dissertation are specific to the investigated smells or reflect broader characteristics of LLM-driven software maintenance.

Another important direction involves improving the semantic reasoning capabilities of LLMs in the context of automated test refactoring. The results consistently show that preserving behavior remains one of the central challenges, particularly in scenarios involving control flow, timing dependencies, and exception handling. In this context, future studies may investigate models specifically trained or fine-tuned for test understanding, incorporating information such as execution traces, assertion intent, or specification-oriented signals. In addition, integrating execution-aware feedback mechanisms, including dynamic analysis or mutation testing, may provide stronger behavioral signals capable of helping models validate and refine

generated refactorings according to their execution outcomes rather than relying exclusively on modifications to the test code.

A related direction concerns extending the validation mechanisms adopted in this dissertation toward more tightly integrated feedback-driven refactoring workflows. Although the current research already incorporates execution-based validation, coverage analysis, and post-refactoring smell detection, these analyses are performed after the refactoring has been generated. Future approaches may investigate how behavioral and smell-related feedback can be incorporated directly into the generation process itself, allowing models to iteratively refine refactorings according to execution outcomes, coverage variation, or detected inconsistencies. Rather than treating validation as a separate post-processing stage, this type of feedback-oriented workflow could enable LLMs to dynamically adjust generated refactorings before producing a final output. Such an approach may be particularly beneficial for complex smells requiring coordinated modifications across multiple parts of the test suite, where the probability of incomplete or behaviorally inconsistent refactorings becomes substantially higher.

Future work may also benefit from incorporating human-centered evaluation processes involving software engineering and testing specialists, enabling a deeper qualitative understanding of the practical strengths and limitations of LLM-assisted refactoring. Although this dissertation analyzes smell removal, behavioral preservation, coverage variation, and newly introduced smells through automated procedures, important aspects related to developer perception remain insufficiently explored. In this context, qualitative investigations involving software maintenance professionals, test engineers, and domain experts could complement the quantitative evidence presented in this study, helping determine which refactoring patterns are considered genuinely beneficial in practice and which modifications, despite being technically valid, may still be viewed as undesirable during maintenance activities. Such analyses could further clarify how practitioners perceive the trade-offs between smell removal and behavioral risk, contributing to a more comprehensive understanding of how LLM-generated refactorings fit within real-world review processes and software evolution workflows.

Finally, extending the evaluation framework itself represents another important avenue for future research. Incorporating additional quality dimensions, such as maintainability indicators, readability metrics, or long-term evolution impact, may provide a more comprehensive understanding of automated refactoring outcomes. Likewise, longitudinal analyses that observe the effects of LLM-generated refactorings over time could help determine whether the improvements achieved initially remain beneficial as projects evolve. Investigating how these refactorings influence future maintenance effort, defect introduction, or test suite evolution may contribute to bridging the gap between controlled experimental evaluation and practical adoption in real-world software engineering environments.

Referências

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Alteschmidt, J., Altman, S., Anadkat, S., et al. (2024). Gpt-4 technical report.
- Alfadel, M., Costa, D. E., and Shihab, E. (2023). Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering*, 28(3):59.
- Aljedaani, W., Peruma, A., Aljohani, A., Alotaibi, M., Mkaouer, M. W., Ouni, A., Newman, C. D., Ghallab, A., and Ludi, S. (2021). Test smell detection tools: A systematic mapping study. In *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*, pages 170–180.
- Amannejad, Y., Garousi, V., Irving, R., and Sahaf, Z. (2014). A search-based approach for cost-effective software test automation decision support and an industrial case study. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops*, pages 302–311.
- Ammann, P. and Offutt, J. (2016). *Introduction to software testing*. Cambridge University Press.
- Amorim, L., da Costa, I. M., Alves, L., and Figueiredo, E. (2025). Bad smell detection using google gemini. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 1637–1642. IEEE.
- Bach, S. H., Sanh, V., Yong, Z.-X., Webson, A., Raffel, C., Nayak, N. V., Sharma, A., Kim, T., Bari, M. S., Fevry, T., et al. (2022). Promptsource: An integrated development environment and repository for natural language prompts. *arXiv preprint arXiv:2202.01279*.
- Bavota, G., Qusef, A., Oliveto, R., De Lucia, A., and Binkley, D. (2012). An empirical analysis of the distribution of unit test smells and their impact on software maintenance. In *2012 28th IEEE International Conference on Software Maintenance (ICSM)*, pages 56–65.
- Bavota, G., Qusef, A., Oliveto, R., De Lucia, A., and Binkley, D. (2015). Are test smells really harmful? an empirical study. *Empirical Software Engineering*, 20:1052–1094.

- Bayram, A., Menekse Dalveren, G. G., and Derawi, M. (2025). Comparative analysis of ai models for python code generation: A humaneval benchmark study. *Applied Sciences*, 15(18).
- Breugelmans, M. and Van Rompaey, B. (2008). Testq: Exploring structural and maintenance characteristics of unit test suites. In *WASDeTT-1: 1st International Workshop on Advanced Software Development Tools and Techniques*, page 11. Citeseer.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Nee-lakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Burnstein, I., Suwanassart, T., and Carlson, R. (1996). Developing a testing maturity model for software test process evaluation and improvement. In *Proceedings International Test Conference 1996. Test and Design Validity*, pages 581–589.
- Cass, S. (2025). The top programming languages 2025. *IEEE Spectrum*. Accessed: 2026-01-25.
- Chen, Z., Embury, S. M., and Vigo, M. (2023). Who is afraid of test smells? assessing technical debt from developer actions. In *Testing Software and Systems*, pages 160–175, Cham. Springer Nature Switzerland.
- Choi, J., An, G., and Yoo, S. (2024). Iterative refactoring of real-world open-source programs with large language models. In *Search-Based Software Engineering*, pages 49–55.
- Ciniselli, M., Cooper, N., Pascarella, L., Mastropaolo, A., Aghajani, E., Poshy-vanyk, D., Di Penta, M., and Bavota, G. (2021). An empirical study on the usage of transformer models for code completion. *IEEE Transactions on Software Engineering*, 48(12):4818–4837.
- Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. (2025). Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Cordeiro, J., Noei, S., and Zou, Y. (2024). An empirical study on the code refactoring capability of large language models. *arXiv preprint arXiv:2411.02320*.
- Dabic, O., Aghajani, E., and Bavota, G. (2021). Sampling projects in github for msr studies. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pages 560–564.

- Damasceno, H., Bezerra, C., Campos, D., Machado, I., and Coutinho, E. (2023). Test smell refactoring revisited: What can internal quality attributes and developers' experience tell us? *Journal of Software Engineering Research and Development*, 11(1):13–1.
- Damasceno, H., Bezerra, C., Coutinho, E., and Machado, I. (2022). Analyzing test smells refactoring from a developers perspective. In *Proceedings of the 21st Brazilian Symposium on Software Quality*, pages 28–37.
- De Bleser, J., Di Nucci, D., and De Roover, C. (2019). Socrates: Scala radar for test smells. In *Proceedings of the Tenth ACM SIGPLAN Symposium on Scala*, pages 22–26.
- Dobslaw, F., Feldt, R., Yoon, J., and Yoo, S. (2025). Challenges in testing large language model based software: A faceted taxonomy. *ACM Transactions on Software Engineering and Methodology*.
- Donato, B., Mariani, L., Micucci, D., and Riganelli, O. (2025). Studying how configurations impact code generation in llms: The case of chatgpt. In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, pages 442–453.
- Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., and Zhang, J. M. (2023). Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53.
- Fernandes, D., Machado, I., and Maciel, R. (2022). Tempy: Test smell detector for python. In *Proceedings of the 36th Brazilian Symposium on Software Engineering*, pages 214–219.
- Fowler, M. (2018). *Refactoring: improving the design of existing code*. Addison-Wesley Professional.
- Gao, Y., Hu, X., Yang, X., and Xia, X. (2025). Automated unit test refactoring. *Proc. ACM Softw. Eng.*, 2(FSE).
- Garousi, V. and Küçük, B. (2018). Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software*, 138:52–81.
- Gillioz, A., Casas, J., Mugellini, E., and Khaled, O. A. (2020). Overview of the transformer-based models for nlp tasks. In *2020 15th Conference on Computer Science and Information Systems (FedCSIS)*, pages 179–183.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.

- Greif, S. and Burel, E. (2025). The state of javascript 2025: Testing - jest. <https://2025.stateofjs.com/en-US/libraries/testing/>. Online; accessed 20 January 2026. Survey run by Devographics from Sep 24 to Dec 15, 2025. Results published on Dec 16, 2025.
- Herculano, W. B. R., Alves, E. L. G., and Mongiovi, M. (2022). Generated tests in the context of maintenance tasks: A series of empirical studies. *IEEE Access*, 10:121418–121443.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. (2024). Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*.
- Islam, R. and Ahmed, I. (2024). Gemini-the most powerful llm: Myth or truth. In *2024 5th Information Communication Technologies Conference (ICTC)*, pages 303–308.
- Jorge, D., Machado, P., and Andrade, W. (2021). Investigating test smells in javascript test code. In *Proceedings of the 6th Brazilian Symposium on Systematic and Automated Software Testing*, pages 36–45.
- Jorge, D. N. (2023). *An investigation on Test Smells in JavaScript test code*. Phd thesis, Federal University of Campina Grande, Paraíba, Brazil.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022). Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.
- Lambiase, S., Cupito, A., Pecorelli, F., De Lucia, A., and Palomba, F. (2020). Just-in-time test smell detection and refactoring: The darts project. In *Proceedings of the 28th International Conference on Program Comprehension*, pages 441–445.
- Leon, M. (2026). Gpt-5 and open-weight large language models: Advances in reasoning, transparency, and control. *Information Systems*, 136:102620.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Lago, A. D., Hubert, T., Choy, P., de Masson d’Autume, C., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Goyal, S., Cherepanov, A., Molloy, J., Mankowitz, D. J., Robson, E. S., Kohli, P., de Freitas, N., Kavukcuoglu, K., and Vinyals, O. (2022). Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.
- Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. (2025a). Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.

- Liu, B., Jiang, Y., Zhang, Y., Niu, N., Li, G., and Liu, H. (2024). An empirical study on the potential of llms in automated software refactoring. *arXiv preprint arXiv:2411.04444*.
- Liu, B., Jiang, Y., Zhang, Y., Niu, N., Li, G., and Liu, H. (2025b). Exploring the potential of general purpose LLMs in automated software refactoring: an empirical study. *Automated Software Engineering*, 32(1):26.
- Liu, V. and Chilton, L. B. (2022). Design guidelines for prompt engineering text-to-image generative models. In *Proceedings of the 2022 CHI conference on human factors in computing systems*, pages 1–23.
- Lucas, K., Gheyi, R., Soares, E., Ribeiro, M., and Machado, I. (2024). Evaluating large language models in detecting test smells. In *Brazilian Symposium on Software Engineering (SBES)*, pages 672–678.
- Magalhães, N. M., de Souza Campos Junior, H., Araújo, M. A. P., and de Oliveira Neves, V. (2017). An automated refactoring approach to remove unnecessary complexity in source code. In *Proceedings of the 2nd Brazilian Symposium on Systematic and Automated Software Testing*, New York, NY, USA. Association for Computing Machinery.
- Martinez, M., Etien, A., Ducasse, S., and Fuhrman, C. (2020). Rtj: a java framework for detecting and refactoring rotten green test cases. In *2020 IEEE/ACM 42nd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 69–72.
- Martinez, S., Xu, L., Elnaggar, M., and Alomar, E. A. (2025). Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software*, page 112762.
- Martins, L., Ghaleb, T. A., Costa, H., and Machado, I. (2024). A comprehensive catalog of refactoring strategies to handle test smells in java-based systems. *Software Quality Journal*, 32(2):641–679.
- Marvin, G., Hellen, N., Jjingo, D., and Nakatumba-Nabende, J. (2023). Prompt engineering in large language models. In *International conference on data intelligence and cognitive informatics*, pages 387–402. Springer.
- Mastropaolo, A., Pascarella, L., Guglielmi, E., Ciniselli, M., Scalabrino, S., Oliveto, R., and Bavota, G. (2023). On the robustness of code generation techniques: An empirical study on github copilot. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2149–2160.
- Meinke, K. (2021). Code coverage and test automation: State of the art. *arXiv preprint arXiv:2108.11723*.

- Melo, R., Simões, P., Gheyi, R., d’Amorim, M., Ribeiro, M., Soares, G., Almeida, E., and Soares, E. (2025). Agentic slms: Hunting down test smells.
- Mendoza, I., Filho, F. S., Medeiros, G., Paes, A., and Neves, V. (2024). Comparative analysis of large language model tools for automated test data generation from bdd. In *Proceedings of the 38th Brazilian Symposium on Software Engineering*, pages 280–290.
- Meneses, M. S. C. (2025). Catspy: a test smells catalog for python. Bachelor’s Thesis, Federal University of Ceará. Bachelor’s Thesis.
- Mens, T. and Tourwe, T. (2004). A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139.
- Meszaros, G. (2007). *xUnit test patterns: Refactoring test code*. Pearson Education.
- Meszaros, G., Smith, S. M., and Andrea, J. (2003). The test automation manifesto. In *Extreme Programming and Agile Methods – XP/Agile Universe 2003*, pages 73–81.
- Misu, M. R. H., Li, J., Bhattiprolu, A., Liu, Y., Almeida, E., and Ahmed, I. (2023). Test smell: A parasitic energy consumer in software testing. *arXiv preprint arXiv:2310.14548*.
- Myers, G. J., Sandler, C., and Badgett, T. (2011). *The art of software testing*. John Wiley & Sons.
- Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., and Mian, A. (2024). A comprehensive overview of large language models.
- Nia, A. B., Dindoost, M., and Bader, D. A. (2025). Evaluating efficiency and novelty of llm-generated code for graph analysis. In *2025 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–7.
- Oliveira, J., Mateus, L., Amaral, G., Virgínio, T., Bezerra, C., Machado, I., and Rocha, L. (2025). Identifying and addressing test smells in javascript: A developer-centric study. In *Proceedings of the Brazilian Symposium on Software Engineering (SBES), Research Track*.
- Oliveira, J., Mateus, L., Virgínio, T., and Rocha, L. (2024). Snuts.js: Sniffing nasty unit test smells in javascript. In *Proceedings of the 38th Brazilian Symposium on Software Engineering*, pages 720–726.
- Opdyke, W. F. (1992). *Refactoring object-oriented frameworks*. University of Illinois at Urbana-Champaign.
- Oppenlaender, J. (2024). A taxonomy of prompt modifiers for text-to-image generation. *Behaviour & Information Technology*, 43(15):3763–3776.

- Ouédraogo, W. C., Li, Y., Dang, X., Tang, X., Koyuncu, A., Klein, J., Lo, D., and Bissyandé, T. F. (2024). Test smells in llm-generated unit tests. *arXiv preprint arXiv:2410.10628*.
- Ouédraogo, W. C., Li, Y., Dang, X., Zhou, X., Koyuncu, A., Klein, J., Lo, D., and Bissyandé, T. F. (2025). Beyond surface similarity: Evaluating llm-based test refactorings with structural and semantic awareness. *arXiv preprint arXiv:2506.06767*.
- Palomba, F., Di Nucci, D., Panichella, A., Oliveto, R., and De Lucia, A. (2016). On the diffusion of test smells in automatically generated test code: an empirical study. In *Proceedings of the 9th International Workshop on Search-Based Software Testing*, pages 5–14, New York, NY, USA. Association for Computing Machinery.
- Peruma, A., Almalki, K., Newman, C. D., Mkaouer, M. W., Ouni, A., and Palomba, F. (2019). On the distribution of test smells in open source android applications: an exploratory study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*, page 193–202.
- Peruma, A., Almalki, K., Newman, C. D., Mkaouer, M. W., Ouni, A., and Palomba, F. (2020a). Tsdetect: An open source test smells detection tool. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, pages 1650–1654.
- Peruma, A., Newman, C. D., Mkaouer, M. W., Ouni, A., and Palomba, F. (2020b). An exploratory study on the refactoring of unit test files in android applications. In *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, pages 350–357.
- Pizzini, A., Reinehr, S., and Malucelli, A. (2023). Sentinel: A process for automatic removing of test smells. In *Proceedings of the 22nd Brazilian Symposium on Software Quality*, pages 80–89.
- Pomian, D., Bellur, A., Dilhara, M., Kurbatova, Z., Bogomolov, E., Sokolov, A., Bryksin, T., and Dig, D. (2024). Em-assist: Safe automated extractmethod refactoring with llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 582–586, New York, NY, USA. Association for Computing Machinery.
- Pontillo, V., d’Aragona, D. A., Pecorelli, F., Nucci, D. D., Ferrucci, F., and Palomba, F. (2024). Machine learning-based test smell detection. *Empirical Software Engineering*, 29(2):55.
- Pressman, R. S. and Maxim, B. R. (2021). *Software Engineering, 9th edition*. McGraw Hill.

- Pucho, A., Ferreira, A., Cirilo, E., and Cafeo, B. (2025). Refactoring python code with llm-based multi-agent systems: An empirical study in ml software projects. In *Proceedings of the 39th Brazilian Symposium on Software Engineering*, pages 678–684.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- Ramchurn, S. D., Stein, S., and Jennings, N. R. (2021). Trustworthy human-ai partnerships. *Iscience*, 24(8).
- Ramler, R. and Wolfmaier, K. (2006). Economic perspectives in test automation: balancing automated and manual testing with opportunity cost. In *Proceedings of the 2006 International Workshop on Automation of Software Test*, pages 85–91.
- Ramos, D., Santana, R., and Machado, I. (2025). Dantes: Automating detection and refactoring of test smells. In *Proceedings of the 39th Brazilian Symposium on Software Engineering*, pages 1003–1009.
- Reichart, S. (2007). *Assessing test quality*. PhD thesis, Master’s thesis, Universitat Bern.
- Reichhart, S., Gîrba, T., and Ducasse, S. (2007). Rule-based assessment of test quality. *J. Object Technol.*, 6(9):231–251.
- Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al. (2023). Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
- Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., and Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *CoRR*.
- Santana, R., Martins, L., Rocha, L., Virgínio, T., Cruz, A., Costa, H., and Machado, I. (2020). Raide: a tool for assertion roulette and duplicate assert identification and refactoring. In *Proceedings of the 34th Brazilian Symposium on Software Engineering*, pages 374–379.
- Santana, R., Martins, L., Virgínio, T., Rocha, L., Costa, H., and Machado, I. (2024). An empirical evaluation of raide: A semi-automated approach for test smells detection and refactoring. *Science of Computer Programming*, 231:103013.
- Santana, R. d. S. (2021). Raide: a semi-automated approach for test smells identification and refactoring. Master’s thesis, Federal University of Bahia, Salvador, Brazil.
- Santana Jr, E. G., Junior, J. P. S., Almeida, E. P., Ahmed, I., Neto, P. A. d. M. S., and de Almeida, E. S. (2025). Evaluating llms effectiveness in detecting and correcting test smells: An empirical study. *arXiv preprint arXiv:2506.07594*.

- Siddiq, M. L., Da Silva Santos, J. C., Tanvir, R. H., Ulfat, N., Al Rifat, F., and Carvalho Lopes, V. (2024). Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, pages 313–322. ACM.
- Silva, A. C. (2022). Identification and characterization of test smells in javascript. *Institute of Exact Sciences and Informatics – Pontifical Catholic University*, 138:52–81.
- Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., McLaughlin, A., Low, A., Ostrow, A., Ananthram, A., et al. (2025). Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*.
- Soares, E., Ribeiro, M., Amaral, G., Gheyi, R., Fernandes, L., Garcia, A., Fonseca, B., and Santos, A. (2020). Refactoring test smells: A perspective from open-source developers. In *Proceedings of the 5th Brazilian Symposium on Systematic and Automated Software Testing*, pages 50–59.
- Soares, E., Ribeiro, M., Gheyi, R., Amaral, G., and Santos, A. (2023). Refactoring test smells with junit 5: Why should developers keep up-to-date? *IEEE Transactions on Software Engineering*, 49(3):1152–1170.
- Soares, E., Ribeiro, M., and Santos, A. (2024). A multimethod study of test smells: Cataloging removal and new types. In *Proceedings of the 23rd Brazilian Symposium on Software Quality*, pages 676–686.
- Soares, E. A. (2023). *A Multimethod Study of Test Smells: Cataloging, Removal, and New Types*. Phd thesis, Federal University of Pernambuco, Recife, Brazil.
- Sommerville, I. (2011). *Software Engineering, 9/E*. Pearson Education India.
- Spillner, A. and Linz, T. (2021). *Software testing foundations: A study guide for the certified tester exam-foundation level-ISTQB® compliant*. dpunkt. verlag.
- Sriwilailak, C., Higo, Y., Lapvikai, P., Ragkhitwetsagul, C., and Choetkiertikul, M. (2024). Autorepairability of chatgpt and gemini: A comparative study. In *2024 31st Asia-Pacific Software Engineering Conference (APSEC)*, pages 442–446.
- Suryanarayana, G., Sharma, T., and Samarthayam, G. (2015). Software process versus design quality: Tug of war? *IEEE Software*, 32(04):7–11.
- Tapader, M. R., Rahman, M. M., Shiplu, A. I., Amin, M. F. I., and Watanobe, Y. (2025). Code refactoring with llm: A comprehensive evaluation with few-shot settings. *arXiv preprint arXiv:2511.21788*.
- Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. (2025). Gemini: A family of highly capable multimodal models.

- Team, G. C. (2026). Copilotchat. Accessed: 2026-01-20.
- Tufano, M., Palomba, F., Bavota, G., Di Penta, M., Oliveto, R., De Lucia, A., and Poshyvanyk, D. (2016). An empirical investigation into the nature of test smells. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, pages 4–15.
- Vahabzadeh, A., Fard, A. M., and Mesbah, A. (2015). An empirical study of bugs in test code. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 101–110.
- van Deursen, A., Moonen, L., van den Bergh, A., and Kok, G. (2001). Refactoring test code. In *2nd International Conference on Extreme Programming and Flexible Processes in Software Engineering (XP2001)*, pages 92–95.
- Virgínio, T., Martins, L., Santana, R., Cruz, A., Rocha, L., Costa, H., and Machado, I. (2021). On the test smells detection: an empirical study on the jnose test accuracy. *Journal of Software Engineering Research and Development*, 9(1):8:1–8:14.
- Wang, T., Golubev, Y., Smirnov, O., Li, J., Bryksin, T., and Ahmed, I. (2021). Pynose: A test smell detector for python. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 593–605.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. (2021). Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., and Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with chatgpt. In *Proceedings of the 30th Conference on Pattern Languages of Programs*.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in software engineering*. Springer Science & Business Media.
- Xue, Z., Zhang, X., Gao, Z., Hu, X., Gao, S., Xia, X., and Li, S. (2026). Clean code, better models: Enhancing llm performance with smell-cleaned dataset. *ACM Trans. Softw. Eng. Methodol.*

Appendix A

Catalog

A.1 Anonymous Test

A.1.1 Definition

The *Anonymous Test* smell occurs when a test method is assigned a vague, generic, or otherwise meaningless name that fails to convey the purpose of the test in its context (Aljedaani et al., 2021; Herculano et al., 2022; Soares et al., 2024). Such names do not describe the behavior under test, the conditions in which it operates, or the expected outcome, making the test intent unclear and open to multiple interpretations (Oliveira et al., 2024).

A.1.2 Consequences

This smell reduces test readability and maintainability by obscuring the intent of the test case and increasing the cognitive effort required to understand the test suite (Herculano et al., 2022). Since test names are a fundamental part of tests as executable documentation, an Anonymous Test weakens the communicative role of the test suite and forces developers to inspect the test implementation to infer its intent (Reichhart et al., 2007; Soares et al., 2024).

A.1.3 Detection

Detection of Anonymous Test focuses on analyzing test names to assess whether they meaningfully describe the behavior being verified (Reichhart et al., 2007). Automated approaches typically rely on syntactic and lexical analysis, such as identifying overly short names, generic terms, or low semantic content. In JavaScript testing frameworks (e.g., Jest or Mocha), this analysis applies to the descriptive strings passed to `it()`, `test()`, or `describe()` blocks, which serve as the effective test identifiers (Oliveira et al., 2024).

A.1.4 Example in JavaScript

Listing 4 shows a Jest test affected by the Anonymous Test smell. The test name is generic and does not specify what aspect of date handling is being verified or what outcome is expected.

Listing 4: Anonymous Test – Example Demonstrating the Smell

```
1 it('should handle date', () => {  
2   const event = new Event({ start: '2026-02-12' });  
3  
4   const formatted = event.formatDate();  
5  
6   expect(formatted).toBe('February 12, 2026');  
7 });
```

A.1.5 Refactoring Strategies

Refactoring an Anonymous Test consists of renaming the test to clearly and explicitly describe the verified behavior (Reichhart et al., 2007; Oliveira et al., 2024). Effective test names typically summarize the relevant precondition, the operation under test, and the expected result, improving the role of tests as readable and self-documenting artifacts (Soares et al., 2024).

Listing 5 presents the refactored version of the test from Listing 4, where the new name explicitly states the input condition and the expected formatting behavior.

Listing 5: Anonymous Test – Refactored Example

```
1 it('formats ISO date string to month day, year format', () => {  
2   const event = new Event({ start: '2026-02-12' });  
3  
4   const formatted = event.formatDate();  
5  
6   expect(formatted).toBe('February 12, 2026');  
7 });
```

When a test exercises multiple behaviors, it should be decomposed into smaller, focused tests, each with a precise and descriptive name. Although most empirical studies on Anonymous Test focus on Java-based or generated tests, the underlying principle of meaningful test naming is language- and framework-independent (Al-jedaani et al., 2021; Reichhart et al., 2007; Herculano et al., 2022; Soares et al., 2024).

A.2 Conditional Test Logic

A.2.1 Definition

The *Conditional Test Logic* test smell is characterized by the presence of control-flow constructs such as conditional statements or loops within a test case, causing the

test to alter its execution path or assertions based on runtime conditions instead of expressing a single fixed expectation (Meszaros, 2007; Wang et al., 2021). By embedding decision logic inside the test body, the test deviates from its role as a direct and unambiguous specification of expected behavior, allowing multiple behavioral scenarios to be exercised within a single test case (Meszaros, 2007; Wang et al., 2021).

A.2.2 Consequences

Conditional Test Logic reduces test clarity and maintainability by obscuring which behavior is being validated in a given execution, since the asserted outcome depends on internal branching rather than on the test name and structure (Meszaros, 2007). Empirical studies in Python and JavaScript test suites associate this smell with increased comprehension and maintenance effort, as developers must reason about conditional paths to diagnose failures, and with reduced diagnostic precision as tests evolve or system behavior changes (Wang et al., 2021; Silva, 2022; Oliveira et al., 2024).

A.2.3 Detection

This smell is typically detected by identifying explicit control-flow constructs such as `if/else` statements, `switch` blocks, or loops inside test methods or test blocks (Meszaros, 2007). Static analysis approaches for Python and JavaScript flag these constructs as indicators that a single test may be covering multiple behavioral scenarios instead of asserting one deterministic outcome (Wang et al., 2021; Oliveira et al., 2024).

A.2.4 Example in JavaScript

Listing 6 illustrates the Conditional Test Logic smell in a Jest test. The conditional statement determines which assertion is executed, making the verified behavior dependent on runtime branching.

Listing 6: Conditional Test Logic - Example Demonstrating the Smell

```
1 it('should process user status correctly', () => {  
2   const status = getUserStatus(user);  
3  
4   if (status === 'active') {  
5     expect(user.isEnabled).toBe(true);  
6   } else {  
7     expect(user.isEnabled).toBe(false);  
8   }  
9 });
```

A.2.5 Refactoring Strategies

Refactoring Conditional Test Logic consists of eliminating internal branching by decomposing a conditional test into multiple focused test cases, each validating a single behavioral scenario with fixed assertions (Meszaros, 2007; Wang et al., 2021). In Listing 6, the invariant assertion on `user.isEnabled` is obscured by conditional selection of the expected value. Separating the scenarios makes the verified behavior explicit. Listing 7 presents the refactored version, where distinct `it()` blocks independently validate the `active` and non-active cases, preserving the original intent while improving clarity and behavioral specification (Meszaros, 2007; Wang et al., 2021; Silva, 2022; Oliveira et al., 2024).

Listing 7: Conditional Test Logic - Refactored Example

```

1  it('should enable the user when status is active', () => {
2      const status = 'active';
3      const user = createUserWithStatus(status);
4
5      expect(user.isEnabled).toBe(true);
6  });
7
8  it('should disable the user when status is not active', () => {
9      const status = 'inactive';
10     const user = createUserWithStatus(status);
11
12     expect(user.isEnabled).toBe(false);
13 });

```

A.3 Duplicate Assert

A.3.1 Definition

The *Duplicate Assert* test smell occurs when a test method contains multiple assertions that verify the same or semantically equivalent condition. These assertions do not improve fault detection capability and instead increase test size and complexity (Peruma et al., 2019; Soares et al., 2024). This smell commonly arises from copy-and-paste practices or incremental test evolution without refactoring, resulting in redundant verification logic that obscures test intent and degrades test quality (Damasceno et al., 2022).

A.3.2 Consequences

Duplicate Assert negatively affects test readability and diagnosability. When a failure occurs, developers must inspect multiple equivalent assertions to determine the relevant condition, increasing debugging effort and cognitive load (Peruma et al., 2019). Redundant assertions also raise maintenance costs, since changes in expected behavior require consistent updates across duplicated checks, increasing the risk

of inconsistencies. Empirical studies report that tests affected by this smell are perceived as harder to understand and refactor, particularly in large test suites (Soares et al., 2024; Damasceno et al., 2022).

A.3.3 Detection

This smell is present when a single test case repeatedly verifies the same property or outcome without providing additional semantic value. Structurally, it can be detected by identifying repeated assertion statements that use the same assertion method with identical or equivalent operands, such as multiple equality checks on the same attribute or return value (Peruma et al., 2019). Automated detection techniques typically rely on syntactic similarity or abstract syntax tree matching to locate duplicated assertion patterns within a test method (Soares et al., 2024). In JavaScript tests, this commonly manifests as repeated `expect()` calls targeting the same value.

A.3.4 Example in JavaScript

Listing 8 illustrates a Jest test affected by the Duplicate Assert smell. The test retrieves a user profile and redundantly verifies the email format multiple times, even though the asserted property remains unchanged across objects.

Listing 8: Duplicate Assert - Example Demonstrating the Smell

```
1 it('should return user with valid email', async () => {  
2   const user = await api.fetchUser(1);  
3  
4   expect(user.email).toMatch(/@company\.com$/);  
5  
6   const profile = await user.getProfile();  
7   expect(profile.contact.email).toMatch(/@company\.com$/);  
8  
9   const normalized = profile.normalize();  
10  expect(normalized.email).toMatch(/@company\.com$/);  
11 });
```

A.3.5 Refactoring Strategies

Refactoring Duplicate Assert consists of removing redundant assertions and retaining a single assertion that clearly expresses the expected behavior (Peruma et al., 2019). Empirical studies recommend eliminating duplicated assertions when they do not convey additional information, as their presence only increases maintenance effort (Damasceno et al., 2022). Tool-supported approaches, such as RAIDE, automate this process by identifying and removing duplicated assertions to improve test maintainability (Santana et al., 2020).

In Listing 8, the email format is an invariant property of the user domain and is propagated unchanged through derived objects. Asserting this property on the

original `user` object is therefore sufficient, since subsequent assertions verify the same condition and do not provide additional fault detection value.

Listing 9 presents the refactored version, in which redundant assertions are removed. The test retains a single email format check, while the remaining assertions focus on distinct properties that were previously obscured by duplication.

Listing 9: Duplicate Assert - Refactored Example

```
1 it('should return user with valid email', async () => {  
2   const user = await api.fetchUser(1);  
3  
4   expect(user.email).toMatch(/@company\.com$/);  
5  
6   const profile = await user.getProfile();  
7   const normalized = profile.normalize();  
8  
9   expect(profile.contact.verified).toBe(true);  
10  expect(normalized.displayName).toBeDefined();  
11 });
```

When multiple properties must be verified, assertions should target distinct and semantically meaningful conditions rather than repeating the same check. If duplication occurs across multiple tests, shared verification logic can be extracted into helper functions or custom matchers (Soares et al., 2024). Although most empirical evidence originates from Java-based environments, the principle of eliminating assertion redundancy directly applies to JavaScript test suites.

A.4 Exception Handling

A.4.1 Definition

The *Exception Handling* test smell, also referred to as *Expected Exception Test* by Meszaros (2007), occurs when the outcome of a test case explicitly depends on a production method throwing an exception and this behavior is verified through manual exception handling constructs such as `try/catch` blocks or explicit `throw` statements (Jorge, 2023; Wang et al., 2021; Peruma et al., 2019). This smell is characterized by the presence of imperative exception handling logic within the test body, typically introduced when developers attempt to validate exceptional behavior directly in the test code (Jorge, 2023). Instead of relying on these language-level constructs, exception verification should be expressed using framework-specific assertion mechanisms designed for this purpose (Jorge, 2023; Peruma et al., 2019; Soares, 2023).

A.4.2 Consequences

Manual exception handling in test cases introduces unnecessary complexity and coupling between test logic and control-flow mechanisms (Jorge, 2023). This imperative

style is often verbose and error-prone, particularly when failure-triggering statements such as `fail()` are omitted, leading to tests that may pass incorrectly (Soares, 2023). Additionally, approaches based on traditional `@ExpectedException` annotations hinder diagnostic precision by obscuring which specific operation triggered the exception (Soares et al., 2020). Empirical studies report that developers frequently perceive such mechanisms as unsafe and remain unaware of safer, more expressive alternatives provided by modern testing frameworks (Soares et al., 2020). Overall, this smell reduces test readability and maintainability by intertwining exception verification with procedural error handling.

A.4.3 Detection

Detection of Exception Handling relies on identifying imperative exception management constructs within test cases. In JavaScript test suites, this is achieved by traversing the abstract syntax tree and detecting `TryStatement` and `ThrowStatement` nodes inside test blocks (Jorge, 2023). In Python, the smell is identified when tests depend on explicit exception handling instead of framework-provided constructs such as `assertRaises()` (Wang et al., 2021). In JUnit-based Java tests, detection targets manual `try/catch` blocks combined with explicit failure statements, as well as the deprecated `@ExpectedException` annotation (Peruma et al., 2019; Soares, 2023).

A.4.4 Example in JavaScript

Listing 10 illustrates the Exception Handling smell in a Jest test. The test attempts to verify exceptional behavior using a manual `try/catch` block and an explicit error throw, which obscures the intent of the test and introduces unnecessary control-flow logic.

Listing 10: Exception Handling - Example Demonstrating the Smell

```
1 it('should reject invalid email format', async () => {  
2   const user = new User();  
3  
4   try {  
5     await user.setEmail('not-an-email');  
6     throw new Error('Expected validation error');  
7   } catch (error) {  
8     expect(error.message).toContain('Invalid email format');  
9   }  
10 });
```

A.4.5 Refactoring Strategies

Refactoring Exception Handling consists of replacing manual, imperative exception management with declarative, framework-specific mechanisms for exception verification (Meszaros, 2007; Peruma et al., 2019; Soares, 2023). Prior studies consistently

recommend using built-in assertions to validate exceptional behavior, as they automatically manage failure conditions and improve the clarity of test intent (Meszaros, 2007; Peruma et al., 2019). This transformation removes boilerplate `try/catch` logic and enables precise specification of expected exception types and messages (Soares, 2023).

Listing 11 shows the refactored version of the test in Listing 10. The imperative exception handling logic is replaced with Jest’s declarative `rejects` matcher, which explicitly asserts that the returned promise fails with the expected error.

Listing 11: Exception Handling - Refactored Example

```
1 it('should reject invalid email format', async () => {  
2   const user = new User();  
3  
4   await expect(user.setEmail('not-an-email'))  
5     .rejects  
6     .toThrow('Invalid email format');  
7 });
```

In Java environments, the corresponding refactoring replaces manual exception handling with JUnit 5’s `assertThrows` assertion, which improves readability and diagnostic accuracy (Soares, 2023). In Python, explicit exception handling should be replaced by `assertRaises()` context managers (Wang et al., 2021). Although the example focuses on JavaScript, the principle of preferring declarative, framework-level exception assertions over imperative control-flow constructs applies consistently across xUnit-style testing frameworks (Meszaros, 2007; Peruma et al., 2019; Soares, 2023).

A.5 Magic Number

A.5.1 Definition

The *Magic Number* test smell occurs when numeric or literal values with implicit meaning are embedded directly in test code without explanatory context. This smell is characterized by the use of hard-coded constants whose semantic role is not immediately evident, thereby obscuring test intent (Peruma et al., 2019). Empirical studies conducted in Java-based testing environments show that Magic Numbers reduce test comprehensibility, as readers must infer the meaning of such values from surrounding code rather than from explicit naming (Soares et al., 2020). As a result, tests affected by this smell require additional cognitive effort to understand and validate.

A.5.2 Consequences

Magic Numbers primarily affect test maintainability and fault diagnosis. When a test fails, developers must first infer the meaning of unexplained literals before

reasoning about the failure, which increases diagnosis time and the likelihood of misinterpretation (Peruma et al., 2020b). In addition, duplicated or inconsistently used literals across tests complicate maintenance, since changes in expected behavior require locating and updating multiple hard-coded values (Damasceno et al., 2022). Empirical studies on developer perception further indicate that tests containing Magic Numbers are considered less readable and more difficult to refactor, especially in large test suites (Peruma et al., 2019).

A.5.3 Detection

Detection of the Magic Number smell can be performed conceptually and structurally. Conceptually, the smell is present when an assertion depends on a literal whose semantic meaning is not self-explanatory. Structurally, automated approaches identify numeric or string literals used directly in assertions or control logic, excluding neutral values such as zero or one (Soares et al., 2020). In Java-based studies, static analysis techniques flag assertion parameters that are not symbolic constants or named variables (Peruma et al., 2020b). In JavaScript test code, this typically manifests as direct comparisons against unexplained values in `expect()` statements (Oliveira et al., 2024).

A.5.4 Example in JavaScript

Listing 12 illustrates the Magic Number smell in a Jest test. The numeric literal used in the assertion does not convey its semantic role, making the expected behavior implicit.

Listing 12: Magic Number - Example Demonstrating the Smell

```
1 it('should apply the correct discount', () => {  
2   const price = calculateFinalPrice(200);  
3  
4   expect(price).toBe(170);  
5 });
```

A.5.5 Refactoring Strategies

Refactoring Magic Number consists of replacing unexplained literals with named constants or variables that explicitly convey intent (Peruma et al., 2020b). Studies conducted in Java-based contexts identify this transformation as a core refactoring operation for mitigating the smell, as it improves readability and reduces duplication without altering test behavior (Damasceno et al., 2022; Soares et al., 2020).

In Listing 12, the assertion verifies that the final price equals `170`, which implicitly represents the discounted result for an input of `200`. Although the assertion correctly checks the expected behavior, the literal obscures the test intent. Refactoring therefore targets the representation of the expected value rather than the assertion

structure. Listing 13 presents the refactored version, in which the opaque literal is replaced with a named constant that captures its semantic role.

Listing 13: Magic Number - Refactored Example

```
1 it('should apply the correct discount', () => {  
2   const originalPrice = 200;  
3   const expectedDiscountedPrice = 170;  
4  
5   const price = calculateFinalPrice(originalPrice);  
6  
7   expect(price).toBe(expectedDiscountedPrice);  
8 });
```

This refactoring removes the Magic Number identified in Listing 12 while preserving test semantics and fault detection capability. By making the expected outcome explicit, the refactored test improves readability and maintainability and aligns with the refactoring intent reported in prior studies (Peruma et al., 2019; Damasceno et al., 2022).

A.6 Overcommented Test

A.6.1 Definition

The *Overcommented Test* smell is characterized by the presence of an excessive number of inline comments within a test case, where comments redundantly describe straightforward test setup or assertions instead of conveying essential information (Reichart, 2007). In this context, comments restate what is already evident from the test code, violating the principle that tests should act as self-explanatory, executable specifications and introducing a *Descriptive Encapsulation Break* by exposing unnecessary narrative about implementation details (Reichart, 2007). Empirical studies in JavaScript test suites report this smell when comments add noise without semantic value, duplicating the meaning already expressed through well-named variables, functions, and assertions (Oliveira et al., 2024).

A.6.2 Consequences

Overcommented Tests increase cognitive load by visually cluttering test code and diverting attention from the actual verification logic, thereby reducing readability and maintainability (Reichart, 2007). Empirical evidence indicates that such comments are prone to becoming outdated as code evolves, creating discrepancies between comments and behavior and complicating debugging and maintenance activities (Reichart, 2007). Since tests are intended to serve as executable documentation, excessive commentary weakens their communicative role and forces developers to mentally filter redundant information in order to understand the test intent (Oliveira

et al., 2024). The presence of this smell also signals insufficient code expressiveness, as the need for extensive commentary often correlates with unclear naming or unnecessarily complex test structure (Reichart, 2007).

A.6.3 Detection

Detection of Overcommented Tests relies on identifying disproportionate comment density within test code. In Java-based studies, a test is flagged when the number of comment lines exceeds the combined number of assertions and referenced fixture variables (Reichart, 2007). JavaScript-oriented approaches adapt this idea through Abstract Syntax Tree analysis, counting all comment nodes associated with a test function and reporting a smell when this count exceeds a fixed threshold (Oliveira et al., 2024). Conceptually, the smell manifests in tests where comments visually dominate the code or merely paraphrase adjacent statements.

A.6.4 Example in JavaScript

Listing 14 shows an Overcommented Test written in Jest. The inline comments explain each trivial step of the test, despite the logic and intent already being explicit in the code.

Listing 14: Overcommented Test - Example Demonstrating the Smell

```
1 // test/models/calculator.test.js
2 describe('Calculator', () => {
3   it('should correctly add two numbers together', () => {
4     // Create a new calculator instance
5     const calc = new Calculator();
6     // Define the first operand
7     const a = 5;
8     // Define the second operand
9     const b = 10;
10
11    // Invoke the add method with the two operands
12    const result = calc.add(a, b);
13
14    // Verify that the result equals the expected sum
15    expect(result).toBe(15);
16  });
17 });
```

A.6.5 Refactoring Strategies

Refactoring Overcommented Tests focuses on removing redundant comments and relying on expressive code structure and naming to convey intent (Reichart, 2007). In Listing 14, the inline comments merely restate obvious actions such as object creation, variable definition, method invocation, and assertion, without adding domain or behavioral insight. Listing 15 presents the refactored version, where com-

ments are eliminated and clarity is preserved through a descriptive test name and straightforward assertions, while maintaining identical test semantics. This refactoring approach is consistent with empirical observations in both Java and JavaScript contexts, which report improved readability and maintainability when unnecessary commentary is removed and intent is expressed directly in code (Reichart, 2007; Oliveira et al., 2024).

Listing 15: Overcommented Test - Refactored Example

```
1 describe('Calculator', () => {  
2   it('adds two numbers correctly', () => {  
3     const calc = new Calculator();  
4     const result = calc.add(5, 10);  
5  
6     expect(result).toBe(15);  
7   });  
8 });
```

A.7 Sleepy Test

A.7.1 Definition

The *Sleepy Test* test smell occurs when a test case contains function calls that temporarily interrupt threads or pause execution in order to simulate external events such as asynchronous calls, delays, or pauses in processing (Jorge, 2023). Developers introduce this smell when they pause the execution of statements in a test method for a fixed duration, typically to simulate or wait for an external event, and then resume execution (Peruma et al., 2019; Santana, 2021). This practice has been reported across multiple languages and frameworks, including JavaScript, Java, JUnit for Android, and Python (Meneses, 2025).

A.7.2 Consequences

Sleepy Test negatively impacts both test suite performance and reliability. Explicitly suspending execution using time-based delays can lead to unexpected results, since the processing time of a task may vary across environments, hardware configurations, and execution loads (Peruma et al., 2019; Santana, 2021). As a consequence, tests become nondeterministic and prone to flakiness when real-time waiting mechanisms are used (Jorge, 2023). In addition, tests that rely on fixed delays to wait for events or responses tend to be unnecessarily slow, increasing overall execution time without improving fault detection capability (Meneses, 2025). This behavior reduces developer confidence in the test suite, as failures may occur intermittently and cannot be reliably reproduced.

A.7.3 Detection

The smell is syntactically detectable through the presence of explicit sleep or delay functions within test bodies or helper methods. In Java and Android tests, detection is performed by identifying calls to the `Thread.sleep()` method inside test methods (Santana, 2021). In JavaScript environments, detection targets calls to the `setTimeout()` function or equivalent delay-based constructs (Jorge, 2023). More subtle instances involve polling loops that repeatedly check conditions combined with fixed waiting intervals. Advanced detection approaches analyze execution traces to identify time-based waiting patterns that can be replaced with event-driven synchronization.

A.7.4 Example in JavaScript

Listing 16 illustrates a Jest test affected by the Sleepy Test smell. The test verifies that a notification is sent after a user updates their profile, but relies on a fixed delay to wait for the asynchronous operation to complete.

Listing 16: Sleepy Test - Example Demonstrating the Smell

```
1 it('should send notification after profile update', async () => {  
2   const user = new User('john@example.com');  
3  
4   user.updateProfile({ name: 'John Doe' });  
5  
6   // Fixed delay to wait for async notification  
7   await new Promise(resolve => setTimeout(resolve, 2000));  
8  
9   expect(notificationService.sent).toHaveBeenCalledWith(  
10     user.id,  
11     'profile_updated'  
12   );  
13 });
```

A.7.5 Refactoring Strategies

Refactoring Sleepy Tests consists of eliminating explicit sleep calls and replacing them with synchronization mechanisms that do not depend on real time (Santana, 2021). Testing libraries provide dedicated abstractions for handling asynchronous behavior, and these should be preferred over raw timing functions (Jorge, 2023). Mock objects and test doubles can also be used to simulate the behavior of external components, removing the need to wait for actual events (Santana, 2021).

In JavaScript environments, refactoring typically involves replacing fixed delays with event-driven synchronization or controlled time manipulation. Listing 17 presents the refactored version of the test shown in Listing 16. Instead of waiting for an arbitrary duration, the test synchronizes on a concrete event emitted by the notification service.

Listing 17: Sleepy Test - Refactored Example

```

1 it('should send notification after profile update', async () => {
2   const user = new User('john@example.com');
3
4   const notificationPromise = new Promise((resolve) => {
5     notificationService.on('sent', (userId, type) => {
6       resolve({ userId, type });
7     });
8   });
9
10  user.updateProfile({ name: 'John Doe' });
11
12  const { userId, type } = await notificationPromise;
13
14  expect(userId).toBe(user.id);
15  expect(type).toBe('profile_updated');
16 });

```

For tests that depend on timers or scheduled callbacks, fake timer mechanisms offer a deterministic alternative. Listing 18 demonstrates this approach using the same notification scenario.

Listing 18: Sleepy Test - Refactored with Fake Timers

```

1 beforeEach(() => {
2   jest.useFakeTimers();
3 });
4
5 afterEach(() => {
6   jest.useRealTimers();
7 });
8
9 it('should send notification after profile update', () => {
10  const user = new User('john@example.com');
11
12  user.updateProfile({ name: 'John Doe' });
13
14  jest.advanceTimersByTime(2000);
15
16  expect(notificationService.sent).toHaveBeenCalledWith(
17    user.id,
18    'profile_updated'
19  );
20 });

```

In Java environments, the recommended transformation replaces `Thread.sleep()` with synchronization utilities such as `CountDownLatch`, `CompletableFuture`, or mocking frameworks that simulate asynchronous behavior without introducing real delays. Although the examples focus on JavaScript, the principle of removing time-based waiting in favor of event-driven or controlled-time synchronization applies across programming languages and testing frameworks (Peruma et al., 2019; Santana, 2021).

A.8 Suboptimal Assertion

A.8.1 Definition

The *Suboptimal Assertion* test smell occurs when a test employs an assertion that does not explicitly or precisely encode the intended verification, even though a more semantically appropriate assertion construct is available (Meneses, 2025). In this situation, the assertion expresses the expected behavior at an unnecessarily low level of abstraction, resulting in a test oracle whose semantics are implicit rather than directly stated (Wang et al., 2021). This smell typically manifests through the use of generic boolean checks or broad comparisons that fail to clearly represent the expected behavior of the system under test (Meneses, 2025; Wang et al., 2021).

A.8.2 Consequences

Suboptimal Assertions reduce the effectiveness of tests during failure analysis and maintenance. When such a test fails, the feedback produced by the assertion is often vague, which forces developers to inspect surrounding code to infer which expectation was violated and increases the effort required for fault localization (Meneses, 2025). Since the validated property is not explicitly stated, this imprecision weakens the role of tests as executable specifications and decreases confidence in the test suite. Empirical catalog analyses further associate this smell with reduced test comprehensibility and a higher risk of misinterpreting failures during regression testing, particularly in large and evolving codebases (Wang et al., 2021; Silva, 2022).

A.8.3 Detection

Suboptimal Assertion can be detected conceptually when assertions validate only broad conditions, such as truth values or generic equality, instead of domain-relevant properties or outcomes. Structurally, detection approaches analyze the use of low-level assertion primitives in contexts where more expressive assertions are applicable (Wang et al., 2021). Cross-language analyses indicate that these detection principles extend to JavaScript test code, enabling systematic identification through static analysis techniques (Silva, 2022).

A.8.4 Example in JavaScript

Listing 19 shows a Jest test affected by the Suboptimal Assertion smell. The test verifies the result of an input validation function, but the assertion checks only a boolean value and does not explicitly state which validation property is expected to hold.

Listing 19: Suboptimal Assertion - Example Demonstrating the Smell

```
1 it('should validate the user input', () => {  
2   const isValid = validateInput('user@email.com');
```

```
3
4     expect(isValid).toBe(true);
5 });
```

A.8.5 Refactoring Strategies

Prior research recommends refactoring Suboptimal Assertions by replacing generic or low-level checks with assertions that explicitly encode the expected behavior (Meneses, 2025; Wang et al., 2021). Strengthening assertions by aligning them with domain semantics improves both readability and diagnostic precision.

In Listing 19, the assertion verifies only that the validation result is `true`, which represents a generic boolean outcome of the validation process. Although this assertion is sufficient to detect a failure, it provides limited information about why the input is considered valid and obscures the test intent. Refactoring therefore focuses on improving the expressiveness of the assertion rather than modifying its structure. Listing 20 presents a refactored version in which the assertion directly reflects the intended validation property.

Listing 20: Suboptimal Assertion - Refactored Example

```
1 it('should validate the user input', () => {
2     const validationResult = validateInput('user@email.com');
3
4     expect(validationResult).toBeValid();
5 });
```

This refactoring replaces the generic boolean comparison identified in Listing 19 with a semantically expressive assertion while preserving the original test intent and fault detection capability. By making the validated property explicit, the refactored test improves clarity and maintainability, in accordance with existing refactoring guidelines (Meneses, 2025; Silva, 2022).

A.9 Unknown Test

A.9.1 Definition

The *Unknown Test* test smell occurs when a test method is executed without explicit assertion statements, being characterized by the complete absence of assertions in the test case body (Jorge, 2023; Meneses, 2025; Wang et al., 2021). Assertion statements describe expected conditions for a test method and allow readers to infer the test purpose by examining what is being verified (Peruma et al., 2019). However, a test method may be written without any assertions, in which case the testing framework reports the test as passing if no exception is thrown during execution (Peruma et al., 2019). This smell commonly arises when developers forget to implement assertions or inadvertently remove existing ones during test modification (Jorge, 2023).

A.9.2 Consequences

Unknown Test negatively impacts test understandability, effectiveness, and maintainability. The absence of explicit assertions makes it difficult for developers to understand the objective of the test, as there are no clearly defined conditions that specify correct behavior (Peruma et al., 2019; Meneses, 2025). As a result, such tests lose their value as executable documentation and as verification mechanisms (Wang et al., 2021). Furthermore, assertion-less tests reduce test effectiveness and complicate maintenance, since developers cannot determine whether the test validates correct behavior or merely executes code paths (Meneses, 2025). These tests may silently pass even when the system under test is faulty, creating false confidence in the reliability of the test suite.

A.9.3 Detection

Detection of the Unknown Test smell occurs when no assertion is identified after scanning the nodes of a test case body (Jorge, 2023). Structurally, a test method is classified as affected when it contains no explicit assertion statements (Meneses, 2025). In JavaScript testing frameworks such as Jest and Mocha, this corresponds to test functions that do not invoke `expect()`, `assert.*()`, or custom matcher functions. Automated detection techniques traverse the abstract syntax tree of each test method and verify the presence of assertion nodes; if none are found, the test is flagged as an Unknown Test.

A.9.4 Example in JavaScript

Listing 21 illustrates a Jest test affected by the Unknown Test smell. The test invokes a user update operation but contains no assertions to verify the outcome. The test passes as long as no exception is thrown, regardless of whether the update was performed correctly.

Listing 21: Unknown Test - Example Demonstrating the Smell

```
1 it('should update user profile', async () => {  
2   const user = new User('john@example.com');  
3  
4   await user.updateProfile({  
5     name: 'John Doe',  
6     department: 'Engineering'  
7   });  
8  
9   // No assertions present  
10 });
```

A.9.5 Refactoring Strategies

Refactoring Unknown Test consists of introducing explicit assertions that verify the expected conditions of the code being tested (Meneses, 2025). Adding meaningful

assertions ensures that the test actually validates behavior and clarifies the test objective for future readers (Meneses, 2025). The refactoring process begins by identifying the intended behavior that the test should verify, which may require consulting requirements, domain knowledge, or the test setup and method name.

Listing 22 presents the refactored version of the test shown in Listing 21. The test now includes explicit assertions that verify the profile update was persisted correctly. When the tested operation does not produce a direct state change, developers should instead assert observable side effects such as mock interactions, emitted events, or logged outputs.

Listing 22: Unknown Test - Refactored Example

```
1 it('should update user profile', async () => {  
2   const user = new User('john@example.com');  
3  
4   await user.updateProfile({  
5     name: 'John Doe',  
6     department: 'Engineering'  
7   });  
8  
9   const updatedUser = await User.findById(user.id);  
10  
11   expect(updatedUser.name).toBe('John Doe');  
12   expect(updatedUser.department).toBe('Engineering');  
13   expect(updatedUser.updatedAt).toBeInstanceOf(Date);  
14 });
```

This refactoring shows that resolving the Unknown Test smell requires more than adding arbitrary assertions. Developers must identify the intended behavior and encode explicit, meaningful verification logic. Although existing studies focus on JavaScript, JUnit, and Python environments, the principle of eliminating assertion-less tests applies uniformly across programming languages and testing frameworks (Peruma et al., 2019; Jorge, 2023; Meneses, 2025; Wang et al., 2021).

A.10 Verbose Test

A.10.1 Definition of Verbose Test

The *Verbose Test* test smell occurs when a test method contains an excessive number of statements or lines relative to a single testing objective (Meneses, 2025; Soares et al., 2024). Instead of validating one specific behavior, such tests aggregate multiple responsibilities within a single method, which obscures the test intent. This smell is conceptually aligned with the *Long Method* code smell described by Fowler (2018). In JavaScript test suites, Verbose Test commonly appears as oversized functions inside `it()`, `test()`, or `describe()` blocks, where extensive setup, execution, and assertion logic are combined, reducing the clarity of the tested behavior (Oliveira et al., 2024).

A.10.2 Consequences

Verbose Test reduces test readability, maintainability, and diagnostic effectiveness (Breugelmans and Van Rompaey, 2008; Meneses, 2025; Soares et al., 2024). Excessively long test methods hinder the identification of the behavior under test and complicate failure analysis.

A.10.3 Detection

Detection of Verbose Test is commonly based on structural metrics applied to test methods. Existing catalogs and tools define threshold values for acceptable test size, such as 30 lines in CATSPY (Meneses, 2025) and 45 lines in TEMPY (Fernandes et al., 2022). In JavaScript, tools such as SNUTS.js analyze the abstract syntax tree of test files to identify oversized functions passed to testing constructs, using configurable limits on line count, number of statements, or logical assertions (Oliveira et al., 2024).

A.10.4 Example in JavaScript

Listing 23 presents a Jest test affected by the Verbose Test smell. The test validates multiple distinct behaviors of an order processing workflow within a single test method, resulting in an oversized and conceptually overloaded test.

Listing 23: Verbose Test – Example Demonstrating the Smell

```

1  it('should process order correctly', async () => {
2      const product1 = new Product('P001', 'Laptop', 1299.99);
3      const product2 = new Product('P002', 'Mouse', 29.99);
4      const customer = new Customer('C001', 'john@example.com');
5      const address = new Address('123 Main St', 'New York', '10001');
6      const payment = new Payment('credit_card', '4242424242424242');
7
8      const order = new Order('ORD-001', customer, address);
9      order.addItem(product1, 1);
10     order.addItem(product2, 2);
11     order.applyDiscount('SUMMER2026', 0.1);
12     await order.calculateTax('NY');
13     await order.processPayment(payment);
14     await notification.send(order, customer);
15
16     expect(order.subtotal).toBe(1359.97);
17     expect(order.discount).toBe(135.99);
18     expect(order.tax).toBe(97.58);
19     expect(order.total).toBe(1321.56);
20     expect(order.status).toBe('PAID');
21     expect(notification.sent).toHaveBeenCalledWith(
22         customer.email,
23         'order_confirmation'
24     );
25 });

```

A.10.5 Refactoring Strategies

Refactoring Verbose Test focuses on decomposing large test methods into smaller, focused test cases, each validating a single behavior (Meneses, 2025; Soares et al., 2024). This approach follows the *Extract Method* refactoring pattern applied to test code, improving clarity and reducing cognitive load (Fowler, 2018).

Listing 24 shows the refactored version of the test from Listing 23. The original test is split into multiple concise test cases, each targeting a distinct aspect of the order processing logic, thereby improving readability and fault localization.

Listing 24: Verbose Test – Refactored Example

```
1 describe('Order processing', () => {
2   let product1, product2, customer, address;
3
4   beforeEach(() => {
5     product1 = new Product('P001', 'Laptop', 1299.99);
6     product2 = new Product('P002', 'Mouse', 29.99);
7     customer = new Customer('C001', 'john@example.com');
8     address = new Address('123 Main St', 'New York', '10001');
9   });
10
11   it('calculates subtotal correctly', () => {
12     const order = new Order('ORD-001', customer, address);
13     order.addItem(product1, 1);
14     order.addItem(product2, 2);
15
16     expect(order.subtotal).toBe(1359.97);
17   });
18
19   it('applies discount correctly', () => {
20     const order = new Order('ORD-001', customer, address);
21     order.addItem(product1, 1);
22     order.addItem(product2, 2);
23     order.applyDiscount('SUMMER2026', 0.1);
24
25     expect(order.discount).toBe(135.99);
26     expect(order.total).toBe(1321.56);
27   });
28
29   it('calculates tax based on location', async () => {
30     const order = new Order('ORD-001', customer, address);
31     order.addItem(product1, 1);
32     order.addItem(product2, 2);
33
34     await order.calculateTax('NY');
35
36     expect(order.tax).toBeCloseTo(97.58, 2);
37   });
38 });
```

Appendix B

Repositories

Table B.1: Technical Characteristics of Selected Projects

| Repository | Stars | JS
Usage(%) | Type |
|--------------------------------------|-------|----------------|-------------|
| redux-offline/redux-offline | 6.1k | 100.00% | library |
| moment/luxon | 16.4k | 100.00% | library |
| trekhleb/javascript-algorithms | 195k | 99.99% | education |
| netflix/falcor | 10.6k | 99.94% | library |
| jantimon/html-webpack-plugin | 10.7k | 99.93% | plugin |
| infernojs/inferno | 16.4k | 99.89% | framework |
| anuraghazra/github-readme-stats | 78.5k | 99.87% | tool |
| CodeGenieApp/serverless-express | 5.3k | 99.76% | library |
| winstonjs/winston | 24.4k | 99.20% | library |
| node-formidable/formidable | 7.2k | 98.29% | library |
| javve/list.js | 11.2k | 97.57% | library |
| react-grid-layout/react-grid-layout | 22.1k | 97.49% | library |
| nock/nock | 13.1k | 96.00% | library |
| pqina/filepond | 16.3k | 95.86% | library |
| verlok/vanilla-lazyload | 7.9k | 83.64% | library |
| svg/svgo | 22.3k | 97.84% | tool |
| facebook/jscodeshift | 10k | 96.86% | tool |
| shutterstock/rickshaw | 6.5k | 91.53% | library |
| bootstrap-vue/bootstrap-vue-dev | 14.5k | 97.84% | framework |
| welldone-software/why-did-you-render | 12.4k | 97.64% | library |
| tj/commander.js | 28k | 95.48% | library |
| swagger-api/swagger-editor | 9.4k | 94.74% | tool |
| jaywcjlove/hotkeys-js | 5.0k | 93.12% | library |
| shipshapecode/tether | 8.5k | 91.65% | library |
| dustinbrett/daedalus | 12.4k | 88.79% | application |
| shutterstock/rickshaw | 6.5k | 88.66% | library |
| katex/katex | 19.8k | 85.83% | library |
| necolas/react-native-web | 22.1k | 83.02% | framework |